

```

F01:0001 """EgoSmplx: automatic fisheye body fitting and fixed-rule MANO fusion."""
F01:0002 __version__ = "1.0.0"
F02:0001 """Command-line interface: check, run, sample."""
F02:0002 import argparse
F02:0003 from pathlib import Path
F02:0004 from egosmplx_pipeline.configuration import load_config, preflight
F02:0005 from egosmplx_pipeline.io import write
F02:0008 def main():
F02:0009     parser = argparse.ArgumentParser(description='EgoSmplx V1.0 automatic fisheye pose pipeline')
F02:0010     sub = parser.add_subparsers(dest='command', required=True)
F02:0011     for name in ['check', 'run']:
F02:0012         cmd = sub.add_parser(name)
F02:0013         cmd.add_argument('--config', type=Path, required=True)
F02:0014         cmd.add_argument('--cached-predictions', action='store_true')
F02:0015         if name == 'run':
F02:0016             cmd.add_argument('--resume', action='store_true')
F02:0017     sample = sub.add_parser('sample')
F02:0018     sample.add_argument('--video', type=Path, required=True)
F02:0019     sample.add_argument('--calibration', type=Path, required=True)
F02:0020     sample.add_argument('--camera', required=True)
F02:0021     sample.add_argument('--count', type=int, default=4)
F02:0022     sample.add_argument('--output', type=Path, required=True)
F02:0023     args = parser.parse_args()
F02:0024     if args.command == 'check':
F02:0025         config, rows = load_config(args.config)
F02:0026         print(preflight(config, rows, args.cached_predictions))
F02:0027     elif args.command == 'run':
F02:0028         from egosmplx_pipeline.pipeline import run
F02:0029         print(run(args.config, args.cached_predictions, args.resume))
F02:0030     else:
F02:0031         from egosmplx_pipeline.sampling import sample_video
F02:0032         print(sample_video(args.video, args.calibration, args.camera, args.count, args.output))
F02:0035 if __name__ == '__main__':
F02:0036     main()
F03:0001 """Portable JSON configuration and fail-fast input checks."""
F03:0002 import os
F03:0003 import re
F03:0004 from pathlib import Path
F03:0005 from egosmplx_pipeline.io import read, sha
F03:0007 PACKAGE_ROOT = Path(__file__).resolve().parents[1]
F03:0010 def expand(value, base):
F03:0011     if isinstance(value, dict):
F03:0012         return {k: expand(v, base) for k, v in value.items()}
F03:0013     if isinstance(value, list):
F03:0014         return [expand(v, base) for v in value]
F03:0015     if isinstance(value, str):
F03:0016         value = value.replace('${PACKAGE_ROOT}', str(PACKAGE_ROOT))
F03:0017         value = os.path.expandvars(value)
F03:0018         if re.search(r'\${[^\}]+\}', value):
F03:0019             raise ValueError('Unresolved environment variable: ' + value)

```

```

F03:0020     return value
F03:0023 def absolute(path, base):
F03:0024     p = Path(path).expanduser()
F03:0025     return str((base / p).resolve() if not p.is_absolute() else p.resolve())
F03:0028 def load_config(path):
F03:0029     path = Path(path).resolve()
F03:0030     config = expand(read(path), path.parent)
F03:0031     for key, value in config['paths'].items():
F03:0032         if value:
F03:0033             config['paths'][key] = absolute(value, path.parent)
F03:0034     config['manifest'] = absolute(config['manifest'], path.parent)
F03:0035     config['output'] = absolute(config['output'], path.parent)
F03:0036     fit = config.setdefault('fitting', {})
F03:0037     for key, default in [('rigid_steps', 1800), ('pose_steps', 1600)]:
F03:0038         fit.setdefault(key, default)
F03:0039     if 'silhouette_weight' in fit:
F03:0040         raise ValueError('The reference profile applies contours after wrist refinement; remove fitting.silhouette_weight')
F03:0041     if min(fit['rigid_steps'], fit['pose_steps']) < 1:
F03:0042         raise ValueError('Invalid fitting settings')
F03:0043     manifest = read(config['manifest'])
F03:0044     frames = expand(manifest['frames'], Path(config['manifest']).parent)
F03:0045     ids = set()
F03:0046     for row in frames:
F03:0047         identifier = row['id']
F03:0048         if not re.fullmatch(r'[A-Za-z0-9_-]+', identifier) or identifier in ids:
F03:0049             raise ValueError('Frame IDs must be unique safe filenames: ' + identifier)
F03:0050         ids.add(identifier)
F03:0051         for key in ['image', 'calibration']:
F03:0052             row[key] = absolute(row[key], Path(config['manifest']).parent)
F03:0053         for key, value in row.get('predictions', {}).items():
F03:0054             row['predictions'][key] = absolute(value, Path(config['manifest']).parent)
F03:0055     if not frames:
F03:0056         raise ValueError('Empty input manifest')
F03:0057     return config, frames
F03:0060 def preflight(config, frames, cached):
F03:0061     """Check required files and image/observation provenance before any output."""
F03:0062     import cv2
F03:0063     required = ['body_python', 'egosmplx_repo', 'egosmplx_config']
F03:0064     if not cached:
F03:0065         required += ['pose_python', 'wilor_python', 'sapiens_repo', 'wilor_repo',
F03:0066                     'egosmplx_checkpoint', 'pose_checkpoint', 'seg_checkpoint',
F03:0067                     'wilor_checkpoint', 'wilor_config', 'wilor_detector']
F03:0068     for key in required:
F03:0069         if key not in config['paths'] or not Path(config['paths'][key]).exists():
F03:0070             raise FileNotFoundError('Required configured path: ' + key)
F03:0071     for row in frames:
F03:0072         image = cv2.imread(row['image'])
F03:0073         if image is None:
F03:0074             raise ValueError('Unreadable image: ' + row['image'])

```

```

F03:0075         if image.shape[1::-1] != (1280, 720):
F03:0076             raise ValueError('Reference profile requires 1280 x 720 images: ' + row['id'])
F03:0077         calibration = read(row['calibration'])
F03:0078         if calibration['size'] != list(image.shape[1::-1]):
F03:0079             raise ValueError('Image and calibration sizes differ: ' + row['id'])
F03:0080         if cached:
F03:0081             for key in ['egosmplx', 'sapiens', 'segmentation', 'wilor']:
F03:0082                 if not Path(row.get('predictions', {})).get(key, '').exists() or key not in row.get(
F03:0082+ 'predictions', {}):
F03:0083                     raise FileNotFoundError(row['id'] + ': missing cached prediction ' + key)
F03:0084                 points = read(row['predictions']['sapiens'])
F03:0085                 if points['image_sha256'] != sha(row['image']):
F03:0086                     raise ValueError('Stale Sapiens2 observation: ' + row['id'])
F03:0087                 for name in ['labels.npy', 'probabilities.npz']:
F03:0088                     if not (Path(row['predictions']['segmentation']) / name).is_file():
F03:0089                         raise FileNotFoundError('Incomplete segmentation: ' + row['id'])
F03:0090         if not cached:
F03:0091             for name in ['pose', 'seg']:
F03:0092                 expected = config.get('checkpoint_sha256', {}).get(name)
F03:0093                 if expected is None or not re.fullmatch('[a-f0-9]{64}', expected):
F03:0094                     raise ValueError('Configure checkpoint_sha256.' + name)
F03:0095         return dict(passed=True, frame_count=len(frames), cached_predictions=cached,
F03:0096                     scope='paths, input sizes, cache image hashes; models are loaded strictly by workers
F03:0096+ ')
F04:0001 """Sequential multi-environment controller with input hashing and verified resume."""
F04:0002 import copy
F04:0003 import datetime
F04:0004 import fcntl
F04:0005 import hashlib
F04:0006 import os
F04:0007 from pathlib import Path
F04:0008 import shutil
F04:0009 import subprocess
F04:0010 import sys
F04:0011 from egosmplx_pipeline.configuration import PACKAGE_ROOT, load_config, preflight
F04:0012 from egosmplx_pipeline.io import read, write, sha
F04:0015 def command(args, log, cwd, environment):
F04:0016     print('RUN', log.name, flush=True)
F04:0017     with log.open('w') as stream:
F04:0018         process = subprocess.run(list(map(str, args)), cwd=str(cwd), env=environment,
F04:0019                                 stdout=stream, stderr=subprocess.STDOUT)
F04:0020     if process.returncode:
F04:0021         raise RuntimeError('Worker failed; inspect ' + str(log))
F04:0024 def environment(config):
F04:0025     result = os.environ.copy()
F04:0026     result.update(OMP_NUM_THREADS='2', MKL_NUM_THREADS='2', PYTHONDONTWRITEBYTECODE='1',
F04:0027                  CUDA_VISIBLE_DEVICES=str(config.get('gpu', '0')),
F04:0028                  EGOSMPLX_CONFIG=config['paths']['egosmplx_config'],
F04:0029                  EGOSMPLX_REPO_ROOT=config['paths']['egosmplx_repo'])
F04:0030     result['PYTHONPATH'] = os.pathsep.join([str(PACKAGE_ROOT), config['paths'].get('wilor_repo', '')])

```

```

F04:0030+ ])
F04:0031     return result
F04:0034 def observe(config, rows, output, env):
F04:0035     """Invoke independent external networks. No manual annotation input."""
F04:0036     p = config['paths']
F04:0037     observations = output / 'observations'
F04:0038     observations.mkdir(exist_ok=False)
F04:0039     staged = observations / 'images'
F04:0040     staged.mkdir()
F04:0041     observed_rows = copy.deepcopy(rows)
F04:0042     for row in observed_rows:
F04:0043         image = Path(row['image'])
F04:0044         target = staged / (row['id'] + image.suffix.lower())
F04:0045         shutil.copy2(image, target)
F04:0046         row['image'] = str(target)
F04:0047     manifest = observations / 'manifest.json'
F04:0048     write(manifest, dict(frames=observed_rows))
F04:0049     for row in observed_rows:
F04:0050         directory = observations / row['id']
F04:0051         directory.mkdir()
F04:0052         command([p['body_python'], '-m', 'egosmplx_pipeline.egosmplx_predict', '--image-dir', staged
F04:0052+ , '--image-glob', Path(row['image']).name,
F04:0053                     '--fisheye-calibration', row['calibration'], '--config-path', p['egosmplx_config'],
F04:0054                     '--checkpoint-path', p['egosmplx_checkpoint'], '--output-dir', directory / 'network
F04:0054+ ',
F04:0055                     '--smplx-input-mode', 'det_undistort_contiguous', '--contiguous-roi-mode', 'aspect_
F04:0055+ pad'],
F04:0056                     directory / 'network.log', p['egosmplx_repo'], env)
F04:0057         candidates = sorted((directory / 'network').rglob(row['id'] + '*_smplx.npz'))
F04:0058         # Some backend versions return the timestamp directory beside the request.
F04:0059         if not candidates:
F04:0060             candidates = sorted(directory.glob('network*/smplx/' + row['id'] + '*_smplx.npz'))
F04:0061         if len(candidates) != 1:
F04:0062             raise RuntimeError('Expected one wearer prediction, found ' + str(len(candidates)))
F04:0063         args = [p['pose_python'], '-m', 'egosmplx_pipeline.sapiens_pose', '--repo-root', p['sapiens_
F04:0063+ repo'],
F04:0064                     '--image', row['image'], '--output-dir', directory / 'sapiens',
F04:0065                     '--checkpoint', p['pose_checkpoint'], '--checkpoint-sha256', config['checkpoint_sha2
F04:0065+ 56']['pose']]
F04:0066         if p.get('sapiens_dependencies'):
F04:0067             args += ['--dependencies', p['sapiens_dependencies']]
F04:0068         command(args, directory / 'sapiens.log', PACKAGE_ROOT, env)
F04:0069         row['predictions'] = dict(egosmplx=str(candidates[0]), sapiens=str(directory / 'sapiens/keyp
F04:0069+ oints.json'))
F04:0070         command([p['wilor_python'], '-m', 'egosmplx_pipeline.wilor_predict', '--manifest', manifest,
F04:0071                     '--output-dir', observations / 'wilor', '--checkpoint', p['wilor_checkpoint'],
F04:0072                     '--config', p['wilor_config'], '--detector', p['wilor_detector']],
F04:0073                     observations / 'wilor.log', p['wilor_repo'], env)
F04:0074         args = [p['pose_python'], '-m', 'egosmplx_pipeline.sapiens_seg', '--repo-root', p['sapiens_repo'
F04:0074+ ],

```

```

F04:0075         '--manifest', manifest, '--output-dir', observations / 'segmentation',
F04:0076         '--checkpoint', p['seg_checkpoint'], '--checkpoint-sha256', config['checkpoint_sha256'] [
F04:0076+ 'seg']]
F04:0077     if p.get('sapiens_dependencies'):
F04:0078         args += ['--dependencies', p['sapiens_dependencies']]
F04:0079     command(args, observations / 'segmentation.log', PACKAGE_ROOT, env)
F04:0080     for row in observed_rows:
F04:0081         row['predictions'].update(wilor=str(observations / 'wilor/npz' / (row['id'] + '_wilor.npz'))
F04:0081+ ,
F04:0082                                     segmentation=str(observations / 'segmentation/frames' / row['id'])
F04:0082+ )
F04:0083     write(output / 'automatic_manifest.json', dict(frames=observed_rows))
F04:0084     return observed_rows
F04:0087 def fingerprint(config, rows):
F04:0088     import json
F04:0089     hashes = {}
F04:0090     for row in rows:
F04:0091         for key in ['image', 'calibration']:
F04:0092             hashes[row['id'] + ':' + key] = sha(row[key])
F04:0093         for key, value in row.get('predictions', {}).items():
F04:0094             if key == 'segmentation':
F04:0095                 for name in ['labels.npy', 'probabilities.npz']:
F04:0096                     hashes[row['id'] + ':' + name] = sha(Path(value) / name)
F04:0097             else:
F04:0098                 hashes[row['id'] + ':' + key] = sha(value)
F04:0099     code = {str(p.relative_to(PACKAGE_ROOT)): sha(p) for p in (PACKAGE_ROOT / 'egosmplx_pipeline').r
F04:0099+ glob('*.py')}
F04:0100     return hashlib.sha256(json.dumps(dict(config=config, inputs=hashes, code=code), sort_keys=True).
F04:0100+ encode()).hexdigest()
F04:0103 def run(config_path, cached=False, resume=False):
F04:0104     config, rows = load_config(config_path)
F04:0105     check = preflight(config, rows, cached)
F04:0106     output = Path(config['output'])
F04:0107     if output.exists() and not resume:
F04:0108         raise FileExistsError('Output exists: choose a new directory or --resume')
F04:0109     output.mkdir(parents=True, exist_ok=True)
F04:0110     with (output / '.lock').open('w') as lock:
F04:0111         fcntl.flock(lock, fcntl.LOCK_EX | fcntl.LOCK_NB)
F04:0112         env = environment(config)
F04:0113         write(output / 'preflight.json', check)
F04:0114         signature = fingerprint(config, rows)
F04:0115         state_path = output / 'run_state.json'
F04:0116         if state_path.exists() and read(state_path)['fingerprint'] != signature:
F04:0117             raise ValueError('Inputs, code, or configuration changed: use a new output directory')
F04:0118         write(state_path, dict(fingerprint=signature, state='running', cached_predictions=cached))
F04:0119         if not cached:
F04:0120             ready = output / 'automatic_manifest.json'
F04:0121             if resume and ready.exists():
F04:0122                 rows = read(ready)['frames']
F04:0123             else:

```

```

F04:0124         if (output / 'observations').exists():
F04:0125             stamp = datetime.datetime.now().strftime('%Y%m%dT%H%M%S%f')
F04:0126             (output / 'observations').rename(output / ('incomplete_observations_' + stamp))
F04:0127             rows = observe(config, rows, output, env)
F04:0128         records = []
F04:0129         for index, row in enumerate(rows):
F04:0130             destination = output / 'frames' / row['id']
F04:0131             validation = destination / 'validation.json'
F04:0132             if resume and validation.exists():
F04:0133                 report = read(validation)
F04:0134                 valid = report.get('passed') and all((destination / f).is_file() and sha(destination
F04:0134+ / f) == h
F04:0135                                     for f, h in report['artifact_sha256'].items())
F04:0136                 if valid:
F04:0137                     records.append(report)
F04:0138                     print('VERIFIED_RESUME', row['id'], flush=True)
F04:0139                     continue
F04:0140             if destination.exists():
F04:0141                 stamp = datetime.datetime.now().strftime('%Y%m%dT%H%M%S%f')
F04:0142                 destination.rename(destination.with_name(destination.name + '_incomplete_' + stamp))
F04:0143                 job_path = output / 'jobs' / (row['id'] + '.json')
F04:0144                 write(job_path, dict(config=config, row=row, output=str(destination)))
F04:0145                 command([config['paths']['body_python'], '-m', 'egosmplx_pipeline.frame', '--job', job_p
F04:0145+ ath],
F04:0146                     output / (row['id'] + '.log'), PACKAGE_ROOT, env)
F04:0147                 records.append(read(validation))
F04:0148                 write(output / 'progress.json', dict(completed=index+1, total=len(rows), last_frame=row[
F04:0148+ 'id']))
F04:0149             if not all(r.get('pipeline_profile') == 'session_hand6_full_pipeline_20260926_v1' for r in r
F04:0149+ ecords):
F04:0150                 job_path = output / 'jobs/reference.json'
F04:0151                 write(job_path, dict(config=config, rows=rows, output=str(output)))
F04:0152                 command([config['paths']['body_python'], '-m', 'egosmplx_pipeline.reference.pipeline', '
F04:0152+ --job', job_path],
F04:0153                     output / 'reference.log', PACKAGE_ROOT, env)
F04:0154                 records = [read(output / 'frames' / row['id'] / 'validation.json') for row in rows]
F04:0155                 from egosmplx_pipeline.report import build
F04:0156                 build(output, records)
F04:0157                 write(state_path, dict(fingerprint=signature, state='complete', frame_count=len(records), ca
F04:0157+ ched_predictions=cached))
F04:0158         return output
F05:0001 """Execute the two-stage body fit and prepare baseline fusion for refinement."""
F05:0002 import argparse
F05:0003 import json
F05:0004 from pathlib import Path
F05:0005 import sys
F05:0006 import cv2
F05:0007 import numpy as np
F05:0008 from egosmplx_pipeline.io import read, write, load, sha, rmse
F05:0011 def canonicalize(parameters, model, camera):

```

```

F05:0012     from egosmplx_pipeline.fusion import geometry
F05:0013     vertices, joints = geometry(parameters)
F05:0014     pixels = camera.world2camera(joints.astype(np.float64)).astype(np.float32)
F05:0015     parameters.update(smplx_mesh_cam=vertices, smplx_joint_cam=joints, smplx_joint_img=pixels,
F05:0016                        smplx_joint_cam_root_relative=joints-joints[:1],
F05:0017                        smplx_body_joints_3d_local=(joints-parameters['transl'].reshape(3))[:25],
F05:0018                        smplx_body_joints_3d_render_cam=joints[:25],
F05:0019                        smplx_body_joints_2d_render_cam=pixels[:25])
F05:0020     return parameters
F05:0023 def invoke(function, arguments):
F05:0024     previous = sys.argv
F05:0025     try:
F05:0026         sys.argv = [function.__module__] + list(map(str, arguments))
F05:0027         function()
F05:0028     finally:
F05:0029         sys.argv = previous
F05:0032 def run(config, row, output):
F05:0033     from egosmplx_pipeline.bootstrap import setup
F05:0034     model, Camera = setup(config['paths']['egosmplx_repo'])
F05:0035     from egosmplx_pipeline import fusion, body_rigid, body_pose
F05:0036     from egosmplx_pipeline.matching import assign
F05:0037     from egosmplx_pipeline.render import render, write_obj, read_obj
F05:0038     fusion.initialize(model)
F05:0039     camera = Camera(row['calibration'])
F05:0040     image = cv2.imread(row['image'])
F05:0041     if image is None or tuple(camera.img_size) != tuple(image.shape[1::-1]):
F05:0042         raise ValueError('Image or calibration dimensions invalid')
F05:0043     output.mkdir(parents=True, exist_ok=False)
F05:0044     predictions = row['predictions']
F05:0045     targets = read(predictions['sapiens'])
F05:0046     image_digest = sha(row['image'])
F05:0047     if targets['image_sha256'] != image_digest:
F05:0048         raise ValueError('Automatic target image hash differs from input')
F05:0049     raw = load(predictions['egosmplx'])
F05:0050     for key in ['global_orient', 'transl']:
F05:0051         if key + '_network' not in raw:
F05:0052             raise ValueError('Raw prediction requires explicit original network field: ' + key)
F05:0053         raw[key] = raw[key + '_network'].copy()
F05:0054     # Retain reconstruction inputs and original network rigid outputs only.
F05:0055     keep = fusion.PARAMS + ['transl', 'global_orient_network', 'transl_network']
F05:0056     raw = canonicalize({k: raw[k] for k in keep}, model, camera)
F05:0057     raw_dir = output / 'raw'
F05:0058     raw_dir.mkdir()
F05:0059     np.savez_compressed(raw_dir / 'params.npz', **raw)
F05:0060     write_obj(raw_dir / 'mesh.obj', raw['smplx_mesh_cam'], model.face)
F05:0061     cv2.imwrite(str(raw_dir / 'wireframe.jpg'), render(image, raw['smplx_mesh_cam'].astype(np.float64), model.face, camera, .12))
F05:0062     common = ['—repo-root', config['paths']['egosmplx_repo'], '—image', row['image'], '—calibrati
F05:0062+ on', row['calibration']]
F05:0063     invoke(body_rigid.main, common + ['—source-npz', raw_dir / 'params.npz', '—keypoints', predict

```

```

F05:0063+ ions['sapiens'],
F05:0064                                     '--output-dir', output / 'rigid', '--steps', config['fitting']['
F05:0064+ rigid_steps'], '--device', 'cpu'])
F05:0065     invoke(body_pose.main, common + ['--source-npz', output / 'rigid/optimized_params.npz',
F05:0066                                     '--automatic-keypoints', predictions['sapiens'],
F05:0067                                     '--output-dir', output / 'body_fit', '--steps', config['fitting']
F05:0067+ ['pose_steps']]))
F05:0068     body = canonicalize(load(output / 'body_fit/automatic/refined_params.npz'), model, camera)
F05:0069     body_dir = output / 'body'
F05:0070     body_dir.mkdir()
F05:0071     np.savez_compressed(body_dir / 'params.npz', **body)
F05:0072     write_obj(body_dir / 'mesh.obj', body['smp1x_mesh_cam'], model.face)
F05:0073     cv2.imwrite(str(body_dir / 'wireframe.jpg'), render(image, body['smp1x_mesh_cam'].astype(np.floa
F05:0073+ t64), model.face, camera, .12))
F05:0074     hands = load(predictions['wilor'])
F05:0075     if 'image_sha256' in hands and str(hands['image_sha256']) != image_digest:
F05:0076         raise ValueError('WiLoR image hash differs from input')
F05:0077     if 'focal_length_px' not in hands:
F05:0078         if 'cached_wilor_focal_px' not in config:
F05:0079             raise ValueError('Legacy WiLoR cache requires an explicit verified focal length')
F05:0080             hands['focal_length_px'] = np.array(config['cached_wilor_focal_px'])
F05:0081             hands['image_size_wh'] = np.array(image.shape[1::-1])
F05:0082     if not np.array_equal(hands['image_size_wh'], image.shape[1::-1]):
F05:0083         raise ValueError('WiLoR image size mismatch')
F05:0084     lookup = {q['name']: q for q in targets['keypoints']}
F05:0085     matches = assign(hands, lookup, body['smp1x_joint_img'], *image.shape[1::-1])
F05:0086     mesh, hand_reports = fusion.stitch(body, hands, matches, camera)
F05:0087     fused = output / 'fused'
F05:0088     fused.mkdir()
F05:0089     np.savez_compressed(fused / 'mesh.npz', **mesh)
F05:0090     np.savez_compressed(fused / 'wilor_predictions.npz', **hands)
F05:0091     write_obj(fused / 'mesh.obj', mesh['vertices_cam'], mesh['faces'])
F05:0092     write(fused / 'recipe.json', dict(matches=matches, blend_rings=7, body_parameter_path='../body/p
F05:0092+ arams.npz',
F05:0093                                     manual_annotations_used=False, local_surface_optimizer=False,
F05:0094                                     standard_smp1x_parameter_only_reconstruction=False))
F05:0095     cv2.imwrite(str(fused / 'wireframe.jpg'), render(image, mesh['vertices_cam'].astype(np.float64),
F05:0095+ mesh['faces'], camera, .12))
F05:0096     cv2.imwrite(str(fused / 'skeleton.jpg'), fusion.draw_geometry(image, camera, mesh))
F05:0097     # Reload actual saved artifacts, including the fusion inputs and recipe.
F05:0098     reloaded_body = load(body_dir / 'params.npz')
F05:0099     v, j = fusion.geometry(reloaded_body)
F05:0100     body_error = float(np.max(np.abs(v-reloaded_body['smp1x_mesh_cam'])))
F05:0101     rebuilt, _ = fusion.stitch(reloaded_body, load(fused / 'wilor_predictions.npz'), read(fused / 'r
F05:0101+ ecipe.json')['matches'], camera)
F05:0102     stored = load(fused / 'mesh.npz')
F05:0103     fusion_error = float(np.max(np.abs(rebuilt['vertices_cam']-stored['vertices_cam'])))
F05:0104     obj_vertices, obj_faces = read_obj(fused / 'mesh.obj')
F05:0105     obj_error = float(np.max(np.abs(obj_vertices-stored['vertices_cam'])))
F05:0106     if max(body_error, fusion_error, obj_error) > 1e-5 or not np.array_equal(obj_faces, stored['face

```



```

F05:0106+ s']]):
F05:0107         raise RuntimeError('Saved geometry failed reload validation')
F05:0108         indices = body['stage2_selected_joint_indices'].astype(int)
F05:0109         truth = body['stage2_target_joint_pixels']
F05:0110         body_errors = np.linalg.norm(body['smplx_joint_img'][indices]-truth, axis=1)
F05:0111         hand_errors = []
F05:0112         for side, match in matches.items():
F05:0113             k = fusion.SIDES.index(side)
F05:0114             projected = camera.world2camera(mesh['hand_joints_cam'][k].astype(np.float64))
F05:0115             hand_errors.extend(np.linalg.norm(projected-hands['joints_2d'][match['candidate']], axis=1).
F05:0115+ tolist())
F05:0116         report = dict(id=row['id'], passed=True, matched_hands=len(matches), matches=matches,
F05:0117             body_target_count=len(indices), body_error_px=body_errors.tolist(), hand_error_px=
F05:0117+ hand_errors,
F05:0118             body_rmse_px=rmse(body['smplx_joint_img'][indices], truth),
F05:0119             hand_rmse_px=float(np.sqrt(np.mean(np.square(hand_errors)))) if hand_errors else N
F05:0119+ one,
F05:0120             body_reload_max_abs_m=body_error, fusion_reload_max_abs_m=fusion_error,
F05:0121             obj_reload_max_abs_m=obj_error, minimum_fused_vertex_z_m=float(mesh['vertices_cam'
F05:0121+ ][:,2].min()),
F05:0122             hand_reports=hand_reports, manual_annotations_used=False,
F05:0123             accuracy_reference='automatic predictions, not human ground truth',
F05:0124             input_image_sha256=image_digest, calibration_sha256=sha(row['calibration']))
F05:0125         report['artifact_sha256'] = {str(p.relative_to(output)): sha(p) for p in output.rglob('*') if p.
F05:0125+ is_file()}
F05:0126         write(output / 'validation.json', report)
F05:0127         print('FRAME_COMPLETE', row['id'], report['body_rmse_px'], report['hand_rmse_px'], flush=True)
F05:0128         return report
F05:0131 def main():
F05:0132     parser = argparse.ArgumentParser(description=__doc__)
F05:0133     parser.add_argument('--job', type=Path, required=True)
F05:0134     args = parser.parse_args()
F05:0135     job = read(args.job)
F05:0136     run(job['config'], job['row'], Path(job['output']))
F05:0139 if __name__ == '__main__':
F05:0140     main()
F06:0001 """Initialize the separately installed EgoSMPLX backend in a worker process."""
F06:0002 import os
F06:0003 import sys
F06:0004 from pathlib import Path
F06:0006 def setup(repo):
F06:0007     repo = Path(repo).resolve()
F06:0008     for sub in ['', 'main', 'common', 'data']:
F06:0009         sys.path.insert(0, str(repo / sub))
F06:0010     os.environ['EGOSMPLX_REPO_ROOT'] = str(repo)
F06:0011     from config import cfg
F06:0012     path = os.environ.get('EGOSMPLX_CONFIG', str(repo / 'main/config/config_ft_egopw_sceneego_egowho
F06:0012+ 1body_unrealego_egofishbody.py'))
F06:0013     cfg.get_config_fromfile(path)
F06:0014     from egosmplx_pipeline import camera as calibrated_camera

```

```

F06:0015     sys.modules['FishEyeCalibrated'] = calibrated_camera
F06:0016     from egosmplx_pipeline.camera import FishEyeCameraCalibrated
F06:0017     from egosmplx_pipeline.projection import stable_world2camera
F06:0018     FishEyeCameraCalibrated.world2camera_pytorch = stable_world2camera
F06:0019     from utils.human_models import smpl_x
F06:0020     import torch
F06:0021     torch.set_num_threads(2)
F06:0022     torch.backends.cuda.matmul.allow_tf32 = False
F06:0023     torch.backends.cudnn.allow_tf32 = False
F06:0024     return smpl_x, FishEyeCameraCalibrated
F07:0001     """Strict JSON/NPZ I/O and reproducibility metadata."""
F07:0002     import hashlib
F07:0003     import json
F07:0004     from pathlib import Path
F07:0005     import numpy as np
F07:0008     def read(path):
F07:0009         return json.loads(Path(path).read_text(encoding='utf-8'))
F07:0012     def write(path, value):
F07:0013         path = Path(path)
F07:0014         path.parent.mkdir(parents=True, exist_ok=True)
F07:0015         temporary = path.with_suffix(path.suffix + '.tmp')
F07:0016         temporary.write_text(json.dumps(value, indent=2, ensure_ascii=False, allow_nan=False) + '\n', en
F07:0016+ coding='utf-8')
F07:0017         temporary.replace(path)
F07:0020     def sha(path):
F07:0021         h = hashlib.sha256()
F07:0022         with Path(path).open('rb') as handle:
F07:0023             for block in iter(lambda: handle.read(8 * 1024**2), b''):
F07:0024                 h.update(block)
F07:0025         return h.hexdigest()
F07:0028     def load(path):
F07:0029         with np.load(path, allow_pickle=False) as data:
F07:0030             result = {k: data[k].copy() for k in data.files}
F07:0031         for key, value in result.items():
F07:0032             if np.issubdtype(value.dtype, np.number) and not np.isfinite(value).all():
F07:0033                 raise ValueError('Nonfinite NPZ field: ' + key)
F07:0034         return result
F07:0037     def arr(tensor):
F07:0038         return tensor.detach().cpu().numpy()
F07:0041     def rmse(prediction, target):
F07:0042         """Root mean squared 2D Euclidean distance, not coordinate-wise RMSE."""
F07:0043         prediction, target = np.asarray(prediction), np.asarray(target)
F07:0044         if prediction.shape != target.shape or prediction.ndim != 2 or prediction.shape[1] != 2:
F07:0045             raise ValueError('Expected matching N x 2 arrays')
F07:0046         if not len(prediction):
F07:0047             return None
F07:0048         return float(np.sqrt(np.mean(np.sum((prediction - target)**2, axis=1))))
F08:0001     """Decode evenly spaced video frames; do not claim stereo synchronization."""
F08:0002     from pathlib import Path
F08:0003     import re

```

```

F08:0004 import cv2
F08:0005 from egosmplx_pipeline.io import write, sha
F08:0008 def sample_video(video, calibration, camera, count, output):
F08:0009     if count < 1 or not re.fullmatch('[A-Za-z0-9_-]+', camera):
F08:0010         raise ValueError('Positive frame count and safe camera name required')
F08:0011     output = Path(output)
F08:0012     if output.exists():
F08:0013         raise FileExistsError(output)
F08:0014     cap = cv2.VideoCapture(str(video))
F08:0015     try:
F08:0016         n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
F08:0017         fps = cap.get(cv2.CAP_PROP_FPS)
F08:0018         if not cap.isOpened() or n < count or fps <= 0:
F08:0019             raise ValueError('Video cannot provide requested frames')
F08:0020         output.mkdir(parents=True)
F08:0021         rows = []
F08:0022         for k in range(count):
F08:0023             index = min(n-1, int((k+.5) * n/count))
F08:0024             cap.set(cv2.CAP_PROP_POS_FRAMES, index)
F08:0025             ok, image = cap.read()
F08:0026             if not ok:
F08:0027                 raise RuntimeError('Failed decoding frame ' + str(index))
F08:0028             identifier = camera + '_f' + str(index).zfill(8)
F08:0029             target = output / (identifier + '.png')
F08:0030             if not cv2.imwrite(str(target), image):
F08:0031                 raise IOError(target)
F08:0032             rows.append(dict(id=identifier, camera=camera, image=target.name,
F08:0033                             calibration=str(Path(calibration).resolve()), image_sha256=sha(target),
F08:0034                             frame_index_zero_based=index, approximate_video_seconds=index/fps))
F08:0035         write(output / 'manifest.json', dict(frames=rows, exact_stereo_synchronization_claimed=False
F08:0035+ ))
F08:0036         return output / 'manifest.json'
F08:0037     finally:
F08:0038         cap.release()
F09:0001 #!/usr/bin/env python3
F09:0002 """Single-person Sapiens2 inference using the installed official pipeline.
F09:0004 An explicit box or the full image is used; no manual keypoint annotations are
F09:0005 read. Output keypoints are in original image pixels, with uncalibrated scores.
F09:0006 """
F09:0007 import argparse
F09:0008 import hashlib
F09:0009 import json
F09:0010 import sys
F09:0011 import time
F09:0012 from pathlib import Path
F09:0015 def main():
F09:0016     p = argparse.ArgumentParser(description=__doc__)
F09:0017     p.add_argument('--repo-root', type=Path, required=True, help='Sapiens2 checkout')
F09:0018     p.add_argument('--dependencies', type=Path)
F09:0019     p.add_argument('--checkpoint', type=Path, required=True)

```

```

F09:0020     p.add_argument('--checkpoint-sha256', required=True)
F09:0021     p.add_argument('--image', type=Path, required=True)
F09:0022     p.add_argument('--output-dir', type=Path, required=True)
F09:0023     p.add_argument('--size', choices=['1b', '5b'], default='1b')
F09:0024     p.add_argument('--bbox', nargs=4, type=float, help='Original image xyxy: default full image')
F09:0025     a = p.parse_args()
F09:0026     root = a.repo_root.resolve()
F09:0027     repo = root
F09:0028     sys.path.insert(0, str(repo))
F09:0029     if a.dependencies:
F09:0030         sys.path.insert(0, str(a.dependencies))
F09:0031     import cv2
F09:0032     import numpy as np
F09:0033     import torch
F09:0034     from safetensors.torch import load_file
F09:0035     from sapiens.engine.config import Config
F09:0036     from sapiens.engine.datasets import Compose
F09:0037     from sapiens.pose.datasets import parse_pose_metainfo
F09:0038     from sapiens.pose.datasets.codecs.udp_heatmap import UDPHeatmap
F09:0039     from sapiens.registry import MODELS
F09:0041     im = cv2.imread(str(a.image))
F09:0042     if im is None:
F09:0043         raise ValueError(f'Unreadable image: {a.image}')
F09:0044     h, w = im.shape[:2]
F09:0045     bbox = a.bbox or [0, 0, w, h]
F09:0046     if not (0 <= bbox[0] < bbox[2] <= w and 0 <= bbox[1] < bbox[3] <= h):
F09:0047         raise ValueError('bbox must lie inside the original image')
F09:0048     a.output_dir.mkdir(parents=True, exist_ok=False)
F09:0049     weight = a.checkpoint
F09:0050     digest = hashlib.sha256()
F09:0051     with weight.open('rb') as f:
F09:0052         for block in iter(lambda: f.read(8 * 1024**2), b''):
F09:0053             digest.update(block)
F09:0054     if digest.hexdigest() != a.checkpoint_sha256:
F09:0055         raise ValueError('Pose checkpoint SHA-256 mismatch')
F09:0056     config = repo / f'sapiens/pose/configs/keypoints308/shutterstock_goliath_3po/sapiens2_{a.size}_k
F09:0056+ eypoints308_shutterstock_goliath_3po-1024x768.py'
F09:0057     cfg = Config.fromfile(config)
F09:0058     cfg.model['backbone'].pop('init_cfg', None)
F09:0059     print('Building official pose model', flush=True)
F09:0060     model = MODELS.build(cfg.model)
F09:0061     state = load_file(str(weight), device='cpu')
F09:0062     model.load_state_dict(state, strict=True)
F09:0063     tensor_count = len(state)
F09:0064     del state
F09:0065     model = model.cuda().eval()
F09:0066     preprocessor = MODELS.build(cfg.data_preprocessor).cuda()
F09:0067     pipeline = Compose(cfg.test_pipeline)
F09:0068     meta = parse_pose_metainfo(dict(from_file=str(repo / 'sapiens/pose/configs/_base_/keypoints308.p
F09:0068+ y'))))

```

```

F09:0069     names = [meta['keypoint_id2name'][i] for i in range(308)]
F09:0070     codec = UDPHeatmap(**{k: v for k, v in cfg.codec.items() if k != 'type'})
F09:0071     data = preprocessor(pipeline(dict(img=im, bbox=np.array([bbox], dtype=np.float32), bbox_score=np
F09:0071+ .ones(1, dtype=np.float32))))
F09:0072     inputs, sample = data['inputs'], data['data_samples']
F09:0073     torch.cuda.synchronize()
F09:0074     start = time.time()
F09:0075     with torch.inference_mode(), torch.autocast('cuda', dtype=torch.bfloat16):
F09:0076         heatmaps = model(inputs)
F09:0077         if cfg.val_cfg.get('flip_test', False):
F09:0078             flipped = model(inputs.flip(-1)).flip(-1)[: , meta['flip_indices']]
F09:0079             heatmaps = (heatmaps + flipped) / 2
F09:0080     heatmaps = heatmaps.float().cpu().numpy()
F09:0081     xy, scores = codec.decode(heatmaps[0])
F09:0082     xy = xy / sample['meta']['input_size'] * sample['meta']['bbox_scale'] + sample['meta']['bbox_cen
F09:0082+ ter'] - 0.5 * sample['meta']['bbox_scale']
F09:0083     xy, scores = xy[0], scores[0]
F09:0084     assert xy.shape == (308, 2) and np.isfinite(xy).all() and np.isfinite(scores).all()
F09:0085     joints = [dict(index=i, name=n, x=float(xy[i, 0]), y=float(xy[i, 1]), score=float(scores[i]), in
F09:0085+ side_image=bool(0 <= xy[i, 0] < w and 0 <= xy[i, 1] < h)) for i, n in enumerate(names)]
F09:0086     payload = dict(model=f'Sapiens2-{a.size.upper()}', image_path=str(a.image.resolve()),
F09:0087                     image_sha256=hashlib.sha256(a.image.read_bytes()).hexdigest(),
F09:0088                     image_width=w, image_height=h, joint_set='sociopticon308',
F09:0089                     coordinate_space='original_image_pixels', bbox_xyxy=bbox,
F09:0090                     visibility='unknown; heatmap scores are not calibrated visibility',
F09:0091                     keypoints=joints, inference_seconds=time.time() - start)
F09:0092     (a.output_dir / 'keypoints.json').write_text(json.dumps(payload, indent=2) + '\n')
F09:0093     np.savez_compressed(a.output_dir / 'heatmaps.npz', heatmap=heatmaps[0])
F09:0094     body = im.copy()
F09:0095     for j in joints:
F09:0096         if j['name'] in {'nose', 'left_shoulder', 'right_shoulder', 'left_elbow', 'right_elbow', 'le
F09:0096+ ft_wrist', 'right_wrist', 'left_hip', 'right_hip', 'left_knee', 'right_knee', 'left_ankle', 'right_a
F09:0096+ nkle'} and j['inside_image']:
F09:0097             pt = (round(j['x']), round(j['y']))
F09:0098             color = (0, 220, 0) if j['score'] >= 0.3 else (0, 150, 255)
F09:0099             cv2.circle(body, pt, 5, color, -1, cv2.LINE_AA)
F09:0100             cv2.putText(body, f"{j['name']} {j['score']:.2f}", (pt[0] + 5, pt[1] - 6), cv2.FONT_HERSHEY
F09:0100+ SIMPLEX, .4, color, 1, cv2.LINE_AA)
F09:0101     cv2.imwrite(str(a.output_dir / 'body_keypoints.jpg'), body)
F09:0102     manifest = dict(checkpoint=str(weight), checkpoint_sha256=digest.hexdigest(),
F09:0103                     configured_sha256_verified=True, strict_load=True, checkpoint_tensors=tensor_cou
F09:0103+ nt,
F09:0104                     config=str(config), python=sys.executable, torch=torch.__version__,
F09:0105                     input_size_hw=[1024, 768], precision='bf16',
F09:0106                     flip_test=cfg.val_cfg.get('flip_test', False), manual_labels_used=False,
F09:0107                     gpu=torch.cuda.get_device_name(), peak_gpu_memory_bytes=torch.cuda.max_memory_al
F09:0107+ located())
F09:0108     (a.output_dir / 'manifest.json').write_text(json.dumps(manifest, indent=2) + '\n')
F09:0109     print(json.dumps(payload, indent=2))
F09:0112 if __name__ == '__main__':

```

```

F09:0113     main()
F10:0001 from pathlib import Path
F10:0002 import sys,json,time,hashlib
F10:0003 import argparse
F10:0004 parser=argparse.ArgumentParser(description='Strict Sapiens2 29-class segmentation')
F10:0005 parser.add_argument('--repo-root',type=Path,required=True)
F10:0006 parser.add_argument('--dependencies',type=Path)
F10:0007 parser.add_argument('--checkpoint',type=Path,required=True)
F10:0008 parser.add_argument('--checkpoint-sha256',required=True)
F10:0009 parser.add_argument('--manifest',type=Path,required=True)
F10:0010 parser.add_argument('--output-dir',type=Path,required=True)
F10:0011 args=parser.parse_args()
F10:0012 REPO=args.repo_root;OUT=args.output_dir
F10:0013 OUT.mkdir(parents=True,exist_ok=False)
F10:0014 sys.path.insert(0,str(REPO))
F10:0015 if args.dependencies:sys.path.insert(0,str(args.dependencies))
F10:0016 import cv2,numpy as np,torch
F10:0017 import torch.nn.functional as F
F10:0018 from sapiens.engine.config import Config
F10:0019 from sapiens.engine.datasets import Compose
F10:0020 from sapiens.registry import MODELS
F10:0021 from sapiens.dense.visualizers import SegVisualizer
F10:0022 from sapiens.dense.datasets import DOME_CLASSES_29
F10:0023 from safetensors.torch import load_file
F10:0025 def write(p,x):p.write_text(json.dumps(x,indent=2,ensure_ascii=False))
F10:0026 def sha(p):
F10:0027     h=hashlib.sha256()
F10:0028     with p.open('rb') as f:
F10:0029         for b in iter(lambda:f.read(16*1024**2),b''):h.update(b)
F10:0030     return h.hexdigest()
F10:0032 def main():
F10:0033     cfgpath=REPO/'sapiens/dense/configs/seg/shutterstock_goliath/sapiens2_1b_seg_shutterstock_goliath-1
F10:0033+ 024x768.py';weight=args.checkpoint
F10:0034     digest=sha(weight);assert digest==args.checkpoint_sha256, 'Segmentation checkpoint SHA-256 mismatch
F10:0034+ '
F10:0035     cfg=Config.fromfile(cfgpath);cfg.model['backbone'].pop('init_cfg',None)
F10:0036     print('Building model',flush=True);model=MODELS.build(cfg.model);state=load_file(str(weight));model
F10:0036+ .load_state_dict(state,strict=True);count=len(state);del state
F10:0037     model=model.cuda().eval();prep=MODELS.build(cfg.data_preprocessor).cuda();pipe=Compose(cfg.test_pip
F10:0037+ eline)
F10:0038     viz=SegVisualizer(with_labels=False);palette=viz.class_palette[:29,:-1].copy()
F10:0039     write(OUT/'classes.json',{int(k):v for k,v in DOME_CLASSES_29.items()})
F10:0040     rows=json.loads(args.manifest.read_text())['frames'];records=[]
F10:0041     for r in rows:
F10:0042         t=time.time();d=OUT/'frames'/r['id'];d.mkdir(parents=True,exist_ok=True);im=cv2.imread(r['image'])
F10:0042+ ;assert im is not None
F10:0043         data=prep(pipe(dict(img=im)));inputs=data['inputs']
F10:0044         with torch.inference_mode(),torch.autocast('cuda',dtype=torch.bfloat16):logits=model(inputs)
F10:0045         logits=F.interpolate(logits.float(),size=im.shape[:2],mode='bilinear',align_corners=False);prob=lo
F10:0045+ gits.softmax(dim=1)[0].cpu().numpy();labels=prob.argmax(0).astype(np.uint8);confidence=prob.max(0)

```

```

F10:0046  assert prob.shape==(29,*im.shape[:2]) and np.isfinite(prob).all() and np.max(np.abs(prob.sum(0)-1)
F10:0046+ )<1e-5
F10:0047  np.save(d/'labels.npy',labels);cv2.imwrite(str(d/'labels.png'),labels);np.savez_compressed(d/'prob
F10:0047+ abilities.npz',probabilities=prob.astype(np.float16),class_ids=np.arange(29),coordinate_space=np.arr
F10:0047+ ay('original_image_pixels'))
F10:0048  cv2.imwrite(str(d/'original.jpg'),im);cv2.imwrite(str(d/'all_parts.jpg'),viz._visualize_segmentati
F10:0048+ on(im,labels));cv2.imwrite(str(d/'part_colors.png'),palette[labels])
F10:0049  focus=im.copy();region=np.isin(labels,[6,7,11,15,16,20]);focus[region]=(im[region]*.5+palette[lab
F10:0049+ els[region]]*.5).astype(np.uint8)
F10:0050  boundary=im.copy();stats={}
F10:0051  for k in [6,7,11,15,16,20]:
F10:0052      mask=(labels==k).astype(np.uint8);contours,_=cv2.findContours(mask,cv2.RETR_EXTERNAL,cv2.CHAIN_AP
F10:0052+ PROX_SIMPLE);color=tuple(int(x) for x in palette[k]);cv2.drawContours(boundary,contours,-1,color,2)
F10:0053      cv2.imwrite(str(d/'mask_{k:02d}.png'),mask*255)
F10:0054      ys,xs=np.where(mask);stats[k]=dict(name=DOME_CLASSES_29[k]['name'],pixels=int(mask.sum()),mean_so
F10:0054+ ftmax=float(confidence[mask>0].mean()) if mask.any() else None,high_confidence_pixels=int(((mask>0)&
F10:0054+ (confidence>=.8)).sum()),bbox_xyxy=[int(xs.min()),int(ys.min()),int(xs.max()+1),int(ys.max()+1)] if
F10:0054+ len(xs) else None)
F10:0055  cv2.imwrite(str(d/'arms_hands.jpg'),focus);cv2.imwrite(str(d/'boundaries.jpg'),boundary);cv2.imwri
F10:0055+ te(str(d/'confidence.jpg'),cv2.applyColorMap((confidence*255).astype(np.uint8),cv2.COLORMAP_VIRIDIS)
F10:0055+ )
F10:0056  rec=dict(id=r['id'],camera=r.get('camera','unknown'),image=r['image'],image_sha256=sha(Path(r['ima
F10:0056+ ge'])),original_hw=list(im.shape[:2]),network_input_shape=list(inputs.shape),inference_seconds=time.
F10:0056+ time()-t,parts=stats,softmax_is_not_calibrated_accuracy=True)
F10:0057  write(d/'stats.json',rec);records.append(rec);print('DONE',r['id'],round(time.time()-t,2),{x:stats
F10:0057+ [x]['pixels'] for x in stats},flush=True)
F10:0058  write(OUT/'manifest.json',dict(frames=records,checkpoint=str(weight),checkpoint_sha256=digest,confi
F10:0058+ gured_sha256_verified=True,strict_load=True,tensor_count=count,config=str(cfgpath),config_sha256=sha
F10:0058+ (cfgpath),python=sys.executable,torch=torch.__version__,precision='bfloat16 autocast; float32 interp
F10:0058+ olation/softmax',preprocessing='official test_pipeline: 1024x768 direct resize keep_ratio=False; BGR
F10:0058+ to RGB; official normalization',postprocessing='bilinear logits resize to original H,W align_corner
F10:0058+ s=False; softmax; argmax',manual_annotations_used=False,mesh_optimization_run=False))
F10:0059 if __name__=='__main__':main()
F11:0001 #!/usr/bin/env python3
F11:0002 """Run WiLoR hand detection/reconstruction on a configurable image manifest."""
F11:0004 import argparse
F11:0005 import json
F11:0006 from pathlib import Path
F11:0008 import cv2
F11:0009 import numpy as np
F11:0010 import torch
F11:0011 from scipy.spatial.transform import Rotation
F11:0013 from wilor.datasets.vitdet_dataset import ViTDetDataset
F11:0014 from wilor.models import load_wilor
F11:0015 from wilor.utils import recursive_to
F11:0016 from wilor.utils.camera import cam_crop_to_full
F11:0017 from wilor.utils.yolo_loader import load_yolo_detector
F11:0020 def parse_args():
F11:0021     ap = argparse.ArgumentParser(description=__doc__)
F11:0022     ap.add_argument("--manifest", type=Path, required=True)

```

```

F11:0023     ap.add_argument("--output-dir", type=Path, required=True)
F11:0024     ap.add_argument("--checkpoint", type=Path, required=True)
F11:0025     ap.add_argument("--config", type=Path, required=True)
F11:0026     ap.add_argument("--detector", type=Path, required=True)
F11:0027     ap.add_argument("--confidence", type=float, default=0.20)
F11:0028     ap.add_argument("--rescale-factor", type=float, default=2.0)
F11:0029     return ap.parse_args()
F11:0032 def project(points, translation, focal_length, image_size_wh):
F11:0033     width, height = map(float, image_size_wh)
F11:0034     xyz = points + translation[None]
F11:0035     uv = xyz[:, :2] / np.maximum(xyz[:, 2:3], 1e-6)
F11:0036     uv *= float(focal_length)
F11:0037     uv += np.asarray([width / 2.0, height / 2.0], np.float32)
F11:0038     return uv
F11:0041 def write_obj(path, vertices, faces):
F11:0042     with path.open("w", encoding="utf-8") as handle:
F11:0043         for vertex in vertices:
F11:0044             handle.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
F11:0045         for face in faces:
F11:0046             handle.write(f"f {face[0] + 1} {face[1] + 1} {face[2] + 1}\n")
F11:0049 def main():
F11:0050     args = parse_args()
F11:0051     if args.output_dir.exists():
F11:0052         raise FileExistsError(args.output_dir)
F11:0053     for name in ("npz", "json", "mesh", "overlay"):
F11:0054         (args.output_dir / name).mkdir(parents=True, exist_ok=True)
F11:0056     rows = json.loads(args.manifest.read_text())['frames']
F11:0057     images = [Path(r['image']) for r in rows]
F11:0058     if len({p.stem for p in images}) != len(images):
F11:0059         raise ValueError('WiLoR input stems must be unique; runner stages images by frame id')
F11:0061     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
F11:0062     model, model_cfg = load_wilor(str(args.checkpoint), str(args.config))
F11:0063     detector = load_yolo_detector(str(args.detector))
F11:0064     model = model.to(device).eval()
F11:0065     detector = detector.to(device)
F11:0066     faces = np.asarray(model.mano.faces, np.int32)
F11:0067     report = {"method": "WiLoR detector + MANO regression", "frames": []}
F11:0069     for image_path in images:
F11:0070         image = cv2.imread(str(image_path), cv2.IMREAD_COLOR)
F11:0071         result = detector(image, conf=args.confidence, verbose=False)[0]
F11:0072         boxes = result.boxes.xyxy.detach().cpu().numpy().astype(np.float32)
F11:0073         scores = result.boxes.conf.detach().cpu().numpy().astype(np.float32)
F11:0074         right = result.boxes.cls.detach().cpu().numpy().astype(np.float32)
F11:0075         frame = {"image": str(image_path), "detections": []}
F11:0076         overlay = image.copy()
F11:0077         focal_value = float(model_cfg.EXTRA.FOCAL_LENGTH / model_cfg.MODEL.IMAGE_SIZE * max(image.sh
F11:0077+ ape[:2]))
F11:0078         if len(boxes) == 0:
F11:0079             np.savez_compressed(args.output_dir / 'npz' / f'{image_path.stem}_wilor.npz',
F11:0080                                 detector_confidence=np.empty(0, np.float32), is_right=np.empty(0, np.f

```



```

F11:0080+ 1oat32),
F11:0081                                     joints_2d=np.empty((0,21,2),np.float32), mano_faces=faces,
F11:0082                                     image_size_wh=np.array(image.shape[1::-1]), focal_length_px=np.array
F11:0082+ (focal_value),
F11:0083                                     image_sha256=np.array(__import__('hashlib').sha256(image_path.read_b
F11:0083+ ytes()).hexdigest()))
F11:0084         cv2.imwrite(str(args.output_dir / 'overlay' / f'{image_path.stem}_wilor.jpg'), overlay)
F11:0085         report['frames'].append(frame)
F11:0086         continue
F11:0088         dataset = ViTDetDataset(model_cfg, image, boxes, right, rescale_factor=args.rescale_factor)
F11:0089         loader = torch.utils.data.DataLoader(dataset, batch_size=16, shuffle=False, num_workers=0)
F11:0090         arrays = {key: [] for key in (
F11:0091             "bbox_xyxy", "detector_confidence", "is_right", "global_orient_aa",
F11:0092             "hand_pose_aa", "betas", "cam_t", "joints_3d_local", "joints_2d",
F11:0093             "vertices_3d_local",
F11:0094         )}
F11:0095         offset = 0
F11:0096         for batch in loader:
F11:0097             batch = recursive_to(batch, device)
F11:0098             with torch.no_grad():
F11:0099                 output = model(batch)
F11:0100                 pred_mano = output["pred_mano_params"]
F11:0101                 pred_cam = output["pred_cam"].clone()
F11:0102                 pred_cam[:, 1] *= 2 * batch["right"] - 1
F11:0103                 image_size = batch["img_size"].float()
F11:0104                 focal = model_cfg.EXTRA.FOCAL_LENGTH / model_cfg.MODEL.IMAGE_SIZE * image_size.max()
F11:0105                 translations = cam_crop_to_full(
F11:0106                     pred_cam, batch["box_center"].float(), batch["box_size"].float(),
F11:0107                     image_size, focal,
F11:0108                 ).detach().cpu().numpy()
F11:0109                 vertices_batch = output["pred_vertices"].detach().cpu().numpy()
F11:0110                 joints_batch = output["pred_keypoints_3d"].detach().cpu().numpy()
F11:0111                 hand_rot = pred_mano["hand_pose"].detach().cpu().numpy()
F11:0112                 global_rot = pred_mano["global_orient"].detach().cpu().numpy()
F11:0113                 betas = pred_mano["betas"].detach().cpu().numpy()
F11:0114                 right_batch = batch["right"].detach().cpu().numpy()
F11:0115                 size_batch = image_size.detach().cpu().numpy()
F11:0116                 for index in range(len(vertices_batch)):
F11:0117                     source_index = offset + index
F11:0118                     is_right = bool(right_batch[index] > 0.5)
F11:0119                     vertices = vertices_batch[index].copy()
F11:0120                     joints = joints_batch[index].copy()
F11:0121                     mirror = 1.0 if is_right else -1.0
F11:0122                     vertices[:, 0] *= mirror
F11:0123                     joints[:, 0] *= mirror
F11:0124                     uv = project(joints, translations[index], float(focal), size_batch[index])
F11:0125                     global_aa = Rotation.from_matrix(global_rot[index, 0]).as_rotvec().astype(np.float32)
F11:0125+ )
F11:0126                     hand_aa = Rotation.from_matrix(hand_rot[index]).as_rotvec().astype(np.float32)
F11:0127                     arrays["bbox_xyxy"].append(boxes[source_index])

```

```

F11:0128         arrays["detector_confidence"].append(scores[source_index])
F11:0129         arrays["is_right"].append(float(is_right))
F11:0130         arrays["global_orient_aa"].append(global_aa)
F11:0131         arrays["hand_pose_aa"].append(hand_aa)
F11:0132         arrays["betas"].append(betas[index])
F11:0133         arrays["cam_t"].append(translations[index])
F11:0134         arrays["joints_3d_local"].append(joints)
F11:0135         arrays["joints_2d"].append(uv)
F11:0136         arrays["vertices_3d_local"].append(vertices)
F11:0137         rec = {
F11:0138             "index": source_index,
F11:0139             "is_right": is_right,
F11:0140             "confidence": float(scores[source_index]),
F11:0141             "bbox_xyxy": boxes[source_index].tolist(),
F11:0142             "joints_2d": uv.tolist(),
F11:0143             "global_orient_aa": global_aa.tolist(),
F11:0144             "hand_pose_aa": hand_aa.tolist(),
F11:0145             "betas": betas[index].tolist(),
F11:0146             "cam_t": translations[index].tolist(),
F11:0147         }
F11:0148         frame["detections"].append(rec)
F11:0149         color = (0, 220, 255) if is_right else (255, 180, 0)
F11:0150         p1 = tuple(np rint(boxes[source_index, :2]).astype(int))
F11:0151         p2 = tuple(np rint(boxes[source_index, 2:]).astype(int))
F11:0152         cv2.rectangle(overlay, p1, p2, color, 2)
F11:0153         cv2.putText(overlay, f"{'R' if is_right else 'L'} {scores[source_index]:.2f}", p1,
F11:0154             cv2.FONT_HERSHEY_SIMPLEX, 0.65, color, 2, cv2.LINE_AA)
F11:0155         for point in uv:
F11:0156             if np.isfinite(point).all():
F11:0157                 cv2.circle(overlay, tuple(np rint(point).astype(int)), 2, color, -1, cv2.LINE_AA)
F11:0157+ E_AA)
F11:0158         write_obj(
F11:0159             args.output_dir / "mesh" / f"{image_path.stem}_hand{source_index}_{'R' if is_right
F11:0159+ ht else 'L'}.obj",
F11:0160             vertices + translations[index][None], faces,
F11:0161         )
F11:0162         offset += len(vertices_batch)
F11:0164         np.savez_compressed(
F11:0165             args.output_dir / "npz" / f"{image_path.stem}_wilor.npz",
F11:0166             **{key: np.asarray(value) for key, value in arrays.items()},
F11:0167             mano_faces=faces,
F11:0168             image_size_wh=np.array(image.shape[1::-1]), focal_length_px=np.array(focal_value),
F11:0169             image_sha256=np.array(__import__('hashlib').sha256(image_path.read_bytes()).hexdigest())
F11:0169+ ,
F11:0170         )
F11:0171         (args.output_dir / "json" / f"{image_path.stem}_wilor.json").write_text(
F11:0172             json.dumps(frame, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
F11:0173         )
F11:0174         cv2.imwrite(str(args.output_dir / "overlay" / f"{image_path.stem}_wilor.jpg"), overlay)
F11:0175         report["frames"].append(frame)

```

```

F11:0176         print(image_path.name, len(frame["detections"]))
F11:0178         (args.output_dir / "wilor_report.json").write_text(
F11:0179             json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
F11:0180         )
F11:0183 if __name__ == "__main__":
F11:0184     main()
F12:0001 """Joint mappings, bounded pose increments, and rigid rotation math. No annotation reader."""
F12:0002 import numpy as np
F12:0003 import torch
F12:0005 JOINT_MAPPING = {
F12:0006     "nose": "Nose",
F12:0007     "left_shoulder": "L_Shoulder",
F12:0008     "right_shoulder": "R_Shoulder",
F12:0009     "left_elbow": "L_Elbow",
F12:0010     "right_elbow": "R_Elbow",
F12:0011     "left_wrist": "L_Wrist",
F12:0012     "right_wrist": "R_Wrist",
F12:0013     "left_hip": "L_Hip",
F12:0014     "right_hip": "R_Hip",
F12:0015     "left_knee": "L_Knee",
F12:0016     "right_knee": "R_Knee",
F12:0017     "left_ankle": "L_Ankle",
F12:0018     "right_ankle": "R_Ankle",
F12:0019 }
F12:0022 BODY_POSE_NAMES = (
F12:0023     "L_Hip", "R_Hip", "Spine_1", "L_Knee", "R_Knee", "Spine_2",
F12:0024     "L_Ankle", "R_Ankle", "Spine_3", "L_Foot", "R_Foot", "Neck",
F12:0025     "L_Collar", "R_Collar", "Head", "L_Shoulder", "R_Shoulder",
F12:0026     "L_Elbow", "R_Elbow", "L_Wrist", "R_Wrist",
F12:0027 )
F12:0030 MAX_DELTA_DEG = {
F12:0031     "L_Hip": 18.0, "R_Hip": 18.0,
F12:0032     "Spine_1": 10.0, "Spine_2": 10.0, "Spine_3": 12.0,
F12:0033     "L_Knee": 22.0, "R_Knee": 22.0,
F12:0034     "L_Ankle": 14.0, "R_Ankle": 14.0,
F12:0035     "Neck": 12.0, "Head": 12.0,
F12:0036     "L_Collar": 15.0, "R_Collar": 15.0,
F12:0037     "L_Shoulder": 28.0, "R_Shoulder": 28.0,
F12:0038     "L_Elbow": 25.0, "R_Elbow": 25.0,
F12:0039     "L_Wrist": 15.0, "R_Wrist": 15.0,
F12:0040 }
F12:0043 def save_obj(vertices, faces, path):
F12:0044     with path.open("w", encoding="utf-8") as output:
F12:0045         for vertex in vertices:
F12:0046             output.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
F12:0047         for face in faces:
F12:0048             output.write(f"f {face[0] + 1} {face[1] + 1} {face[2] + 1}\n")
F12:0050 def axis_angle_rotation(axis_angle):
F12:0051     x, y, z = axis_angle.unbind()
F12:0052     zero = torch.zeros_like(x)

```

```

F12:0053     skew = torch.stack((zero, -z, y, z, zero, -x, -y, x, zero)).reshape(3, 3)
F12:0054     theta = torch.linalg.norm(axis_angle)
F12:0055     first = torch.sinc(theta / np.pi)
F12:0056     second = 0.5 * torch.sinc(theta / (2.0 * np.pi)) ** 2
F12:0057     eye = torch.eye(3, dtype=axis_angle.dtype, device=axis_angle.device)
F12:0058     return eye + first * skew + second * (skew @ skew)
F13:0001 #!/usr/bin/env python3
F13:0002 """Fit only SMPL-X global orientation and translation to Sapiens2 2D points.
F13:0004 Reuses the existing calibrated-fisheye rigid-fit helpers. Only automatic Sapiens2 targets are accept
F13:0004+ ed.
F13:0005 The body rotates about the SMPL-X pelvis; saved parameters are regenerated and
F13:0006 checked against the optimized geometry before any result is delivered.
F13:0007 """
F13:0008 import argparse
F13:0009 import hashlib
F13:0010 import json
F13:0011 import os
F13:0012 from pathlib import Path
F13:0014 import cv2
F13:0015 import numpy as np
F13:0016 import torch
F13:0017 import torch.nn.functional as F
F13:0020 def main():
F13:0021     p = argparse.ArgumentParser(description=__doc__)
F13:0022     for name in ['repo-root', 'source-npz', 'calibration', 'keypoints', 'image', 'output-dir']:
F13:0023         p.add_argument('--' + name, type=Path, required=True)
F13:0024     p.add_argument('--score-threshold', type=float, default=0.3)
F13:0025     p.add_argument('--steps', type=int, default=1800)
F13:0026     p.add_argument('--max-orient-delta-deg', type=float, default=60.)
F13:0027     p.add_argument('--device', choices=['cpu', 'cuda'], default='cpu', help='Rigid fitting is small;
F13:0027+ CPU avoids old CUDA numerical differences')
F13:0028     a = p.parse_args()
F13:0029     if not 0 <= a.score_threshold <= 1 or a.steps < 1:
F13:0030         raise ValueError('Invalid score threshold or step count')
F13:0031     os.environ['EGOSMPLX_REPO_ROOT'] = str(a.repo_root.resolve())
F13:0032     from egosmplx_pipeline.bootstrap import setup
F13:0033     setup(a.repo_root)
F13:0034     from egosmplx_pipeline.body_common import axis_angle_rotation, save_obj, JOINT_MAPPING
F13:0035     from FishEyeCalibrated import FishEyeCameraCalibrated
F13:0036     from utils.human_models import smpl_x
F13:0037     from utils.vis import render_mesh_fisheye
F13:0039     image = cv2.imread(str(a.image))
F13:0040     if image is None:
F13:0041         raise ValueError('Unreadable image')
F13:0042     h, w = image.shape[:2]
F13:0043     payload = json.loads(a.keypoints.read_text())
F13:0044     if payload['coordinate_space'] != 'original_image_pixels':
F13:0045         raise ValueError('Sapiens2 targets must use original image pixels')
F13:0046     if [payload['image_width'], payload['image_height']] != [w, h]:
F13:0047         raise ValueError('Target dimensions differ from image')

```

```

F13:0048     if payload['image_sha256'] != hashlib.sha256(a.image.read_bytes()).hexdigest():
F13:0049         raise ValueError('Target image hash mismatch')
F13:0050     camera = FishEyeCameraCalibrated(str(a.calibration.resolve()))
F13:0051     if tuple(map(int, camera.img_size)) != (w, h):
F13:0052         raise ValueError('Calibration dimensions differ from image')
F13:0053     raw = {k: v.copy() for k, v in np.load(a.source_npz).items()}
F13:0054     fixed_keys = ['body_pose', 'betas', 'left_hand_pose', 'right_hand_pose', 'jaw_pose', 'leye_pose'
F13:0054+ , 'reye_pose', 'expression']
F13:0055     for key in fixed_keys + ['global_orient', 'transl', 'smplx_mesh_cam', 'smplx_body_joints_3d_loca
F13:0055+ 1']:
F13:0056         if key not in raw or not np.isfinite(raw[key]).all():
F13:0057             raise ValueError('Missing or nonfinite source field: ' + key)
F13:0058     dev = torch.device(a.device)
F13:0059     # Rigid composition and parameter regeneration require full FP32 mantissas;
F13:0060     # TF32 matrix products can introduce millimeter-scale discrepancies.
F13:0061     torch.backends.cuda.matmul.allow_tf32 = False
F13:0062     torch.backends.cudnn.allow_tf32 = False
F13:0063     t = lambda x: torch.as_tensor(x, dtype=torch.float32, device=dev)
F13:0064     layer = smplx_x.layer['neutral'].to(dev).eval()
F13:0065     fixed = {key: t(raw[key]).reshape(1, -1) for key in fixed_keys}
F13:0066     cam0 = t(raw['transl']).reshape(3)
F13:0067     glob0 = t(raw['global_orient']).reshape(1, 3)
F13:0068     with torch.no_grad():
F13:0069         initial = layer(global_orient=glob0, **fixed)
F13:0070         local = initial.joints[0, smplx_x.joint_idx[:25]].detach()
F13:0071         vertices = initial.vertices[0].detach()
F13:0072         pivot = initial.joints[0, 0].detach()
F13:0073         mesh_error = float((vertices + cam0 - t(raw['smplx_mesh_cam'])).abs().max())
F13:0074         joints_error = float((local - t(raw['smplx_body_joints_3d_local'])).abs().max())
F13:0075     print('Source reconstruction errors (m):', mesh_error, joints_error, flush=True)
F13:0076     # Legacy CUDA inference and full-precision CPU LBS differ by submillimeter
F13:0077     # rounding. Audit this separately from the much tighter final round trip.
F13:0078     if max(mesh_error, joints_error) > 1e-3:
F13:0079         raise ValueError(f'Source parameter reconstruction mismatch: mesh={mesh_error}, joints={join
F13:0079+ ts_error} m')
F13:0081     # Selection is determined solely by model scores and in-image coordinates.
F13:0082     lookup = {j['name']: j for j in payload['keypoints']}
F13:0083     if len(lookup) != len(payload['keypoints']):
F13:0084         raise ValueError('Duplicate keypoint names')
F13:0085     selected, excluded = [], []
F13:0086     for name, model_name in JOINT_MAPPING.items():
F13:0087         j = lookup.get(name)
F13:0088         valid = j is not None and np.isfinite([j['x'], j['y'], j['score']]).all()
F13:0089         valid = valid and 0 <= j['x'] < w and 0 <= j['y'] < h and j['score'] >= a.score_threshold
F13:0090         if valid:
F13:0091             selected.append(dict(name=name, model_name=model_name, index=smplx_x.joints_name.index(mo
F13:0091+ del_name), x=j['x'], y=j['y'], score=j['score']))
F13:0092         else:
F13:0093             excluded.append(dict(name=name, prediction=j, reason='missing, nonfinite, outside image,
F13:0093+ or below score threshold'))

```

```

F13:0094     if len(selected) < 4:
F13:0095         raise ValueError(f'Only {len(selected)} usable automatic targets; need at least four')
F13:0096     target = t([j['x'], j['y']] for j in selected])
F13:0097     if np.linalg.svd(target.cpu().numpy() - target.cpu().numpy().mean(0), compute_uv=False)[1] < 10:
F13:0098         raise ValueError('Automatic targets have insufficient 2D spatial spread')
F13:0099     indices = torch.tensor([j['index'] for j in selected], device=dev)
F13:0100     weights = t([np.clip(j['score'], 0, 1)**2 for j in selected])
F13:0101     cam = cam0.clone().requires_grad_(True)
F13:0102     delta = torch.zeros(3, device=dev, requires_grad=True)
F13:0103     scale = t([w, h])
F13:0104     def geometry():
F13:0105         rotation = axis_angle_rotation(delta)
F13:0106         joints = (local - pivot) @ rotation.t() + pivot + cam
F13:0107         mesh = (vertices - pivot) @ rotation.t() + pivot + cam
F13:0108         pixels = camera.world2camera_pytorch(joints)
F13:0109         return joints, mesh, pixels
F13:0110     with torch.no_grad():
F13:0111         _, _, before = geometry()
F13:0112         before = before.clone()
F13:0113     opt = torch.optim.Adam([{'params': [cam], 'lr': .005}, {'params': [delta], 'lr': .002}])
F13:0114     best_loss, best_state = float('inf'), None
F13:0115     history = []
F13:0116     for step in range(a.steps + 1):
F13:0117         opt.zero_grad()
F13:0118         joints, mesh, pixels = geometry()
F13:0119         residual = (pixels[indices] - target) / scale
F13:0120         robust = F.smooth_l1_loss(residual, torch.zeros_like(residual), beta=.005, reduction='none')
F13:0120+     .mean(1)
F13:0121         reprojection = (robust * weights).sum() / weights.sum()
F13:0122         prior_t = (((cam - cam0) / t([.25, .25, .50]))**2).mean()
F13:0123         loss = reprojection + 1e-4 * prior_t + 2e-4 * delta.square().mean() + F.relu(.02 - mesh[:, 2
F13:0123+ ].min()).square()
F13:0124         if not torch.isfinite(loss):
F13:0125             raise RuntimeError('Nonfinite optimization objective')
F13:0126         value = float(loss.detach())
F13:0127         if value < best_loss:
F13:0128             best_loss, best_state = value, (cam.detach().clone(), delta.detach().clone())
F13:0129         if step % 100 == 0 or step == a.steps:
F13:0130             history.append(dict(step=step, loss=value, reprojection=float(reprojection.detach())))
F13:0131         if step == a.steps:
F13:0132             break
F13:0133         loss.backward()
F13:0134         if not torch.isfinite(cam.grad).all() or not torch.isfinite(delta.grad).all():
F13:0135             raise RuntimeError('Nonfinite rigid-fit gradient')
F13:0136         opt.step()
F13:0137         with torch.no_grad():
F13:0138             norm = delta.norm()
F13:0139             limit = np.deg2rad(a.max_orient_delta_deg)
F13:0140             if norm > limit:
F13:0141                 delta.mul_(limit / norm)

```

```

F13:0142         cam[2].clamp_(.05, 3.)
F13:0143     with torch.no_grad():
F13:0144         cam.copy_(best_state[0])
F13:0145         delta.copy_(best_state[1])
F13:0146         joints, mesh, after = geometry()
F13:0147         delta_rotation = axis_angle_rotation(delta).cpu().numpy()
F13:0148         new_rotation = delta_rotation @ cv2.Rodrigues(raw['global_orient']).astype(np.float64).reshap
F13:0148+ e(3)) [0]
F13:0149         new_global = cv2.Rodrigues(new_rotation.astype(np.float64)) [0].astype(np.float32).reshape(1,
F13:0149+ 3)
F13:0150         regenerated = layer(global_orient=t(new_global), **fixed)
F13:0151         rebuilt_mesh = regenerated.vertices[0] + cam
F13:0152         rebuilt_local = regenerated.joints[0, smp1_x.joint_idx[:25]]
F13:0153         rebuilt_pixels = camera.world2camera_pytorch(rebuilt_local + cam)
F13:0154         regen_mesh_error = float((rebuilt_mesh - mesh).abs().max())
F13:0155         regen_pixel_error = float((rebuilt_pixels - after).abs().max())
F13:0156         if regen_mesh_error > 1e-4 or regen_pixel_error > .05:
F13:0157             print('REGEN_DIAGNOSTIC', dict(
F13:0158                 difference_mean=(rebuilt_mesh - mesh).mean(0).cpu().tolist(),
F13:0159                 difference_std=(rebuilt_mesh - mesh).std(0).cpu().tolist(),
F13:0160                 pivot=pivot.cpu().tolist(), regenerated_pivot=regenerated.joints[0, 0].cpu().tolist(
F13:0160+ ),
F13:0161                 rotation_roundtrip_error=float(np.max(np.abs(cv2.Rodrigues(new_global.astype(np.floa
F13:0161+ t64).reshape(3)) [0] - new_rotation))),
F13:0162                 rotation_orthogonality_error=float(np.max(np.abs(new_rotation.T @ new_rotation - np.
F13:0162+ eye(3))))),
F13:0163                 delta=delta.cpu().tolist(), global_new=new_global.tolist(), flush=True)
F13:0164         raise RuntimeError(f'Regenerated geometry mismatch: {regen_mesh_error} m, {regen_pixel_e
F13:0164+ rror} px')
F13:0165     cpu = lambda x: x.detach().cpu().numpy()
F13:0166     saved = {k: v.copy() for k, v in raw.items()}
F13:0167     saved.update(global_orient=new_global,
F13:0168                 transl=cpu(cam).reshape(1, 3), smp1x_mesh_cam=cpu(rebuilt_mesh),
F13:0169                 smp1x_body_joints_3d_local=cpu(rebuilt_local),
F13:0170                 smp1x_body_joints_3d_render_cam=cpu(rebuilt_local + cam),
F13:0171                 smp1x_body_joints_2d_render_cam=cpu(rebuilt_pixels),
F13:0172                 sapiens2_target_joint_pixels=cpu(target), sapiens2_selected_joint_indices=cpu(indic
F13:0172+ es),
F13:0173                 sapiens2_target_scores=np.array([j['score'] for j in selected], dtype=np.float32),
F13:0174                 global_orient_delta_rotation=delta_rotation)
F13:0175     full_joints = regenerated.joints[0, smp1_x.joint_idx]
F13:0176     saved['smp1x_joint_cam_root_relative'] = cpu(full_joints - full_joints[:1])
F13:0177     if not all(np.array_equal(saved[k], raw[k]) for k in fixed_keys):
F13:0178         raise RuntimeError('A fixed parameter was changed')
F13:0179     def errors(pred, truth):
F13:0180         d = np.linalg.norm(pred - truth, axis=1)
F13:0181         return dict(rmse_px=float(np.sqrt(np.mean(d**2))), mean_px=float(d.mean()), median_px=float(
F13:0181+ np.median(d)))
F13:0182     report = dict(source_npz=str(a.source_npz.resolve()), image=str(a.image.resolve()), calibration=
F13:0182+ str(a.calibration.resolve()),

```

```

F13:0183 sapiens2_keypoints=str(a.keypoints.resolve()), optimized_parameters=['global_orien
F13:0183+ t', 'cam_trans'],
F13:0184 fixed_parameters=fixed_keys, fixed_parameters_bitwise_equal=True, network_was_reru
F13:0184+ n=False,
F13:0185 manual_labels_used_for_optimization=False, score_threshold=a.score_threshold,
F13:0186 score_weight='clip(score,0,1)^2; scores are not calibrated visibility probabilitie
F13:0186+ s',
F13:0187 selected=selected, excluded=excluded, initial_cam_trans_m=cpu(cam0).tolist(),
F13:0188 optimized_cam_trans_m=cpu(cam).tolist(), initial_global_orient=raw['global_orient'
F13:0188+ ].tolist(),
F13:0189 optimized_global_orient=new_global.tolist(), global_orient_delta_deg=float(np.rad2
F13:0189+ deg(np.linalg.norm(cpu(delta)))),
F13:0190 source_reconstruction_max_abs_m=dict(mesh=mesh_error, joints=joints_error),
F13:0191 source_reconstruction_tolerance_m=1e-3, optimization_device=str(dev),
F13:0192 saved_parameter_regeneration_max_abs_m=regen_mesh_error,
F13:0193 saved_parameter_regeneration_max_abs_px=regen_pixel_error,
F13:0194 source_npz_sha256=hashlib.sha256(a.source_npz.read_bytes()).hexdigest(),
F13:0195 raw_to_sapiens2=errors(cpu(before[indices]), cpu(target)),
F13:0196 optimized_to_sapiens2=errors(cpu(rebuilt_pixels[indices]), cpu(target)), history=h
F13:0196+ istory,
F13:0197 minimum_vertex_z_m=float(rebuilt_mesh[:, 2].min()),
F13:0198 geometry_convention='SMPL-X vertices/joints include global orientation; add transla
F13:0198+ ; rotate about SMPL-X pelvis',
F13:0199 preserved_network_fields='Fields containing network or model_output and positionne
F13:0199+ t are original inference diagnostics')
F13:0200 a.output_dir.mkdir(parents=True, exist_ok=False)
F13:0201 np.savez_compressed(a.output_dir / 'optimized_params.npz', **saved)
F13:0202 save_obj(cpu(rebuilt_mesh), smpl_x.face, a.output_dir / 'optimized_mesh.obj')
F13:0203 overlays = []
F13:0204 for name, verts in [('raw', raw['smplx_mesh_cam']), ('optimized', cpu(rebuilt_mesh))]:
F13:0205     overlay = render_mesh_fisheye(image, verts, smpl_x.face, {}, camera, mode='wireframe', mesh_
F13:0205+ color=(255, 255, 255), alpha=.78, proj_trans=None)
F13:0206     cv2.imwrite(str(a.output_dir / f'{name}_wireframe.jpg'), overlay)
F13:0207     panel = cv2.resize(overlay, (640, round(h * 640 / w)))
F13:0208     cv2.rectangle(panel, (0, 0), (640, 38), (25, 25, 25), -1)
F13:0209     cv2.putText(panel, 'Raw EgoSMPLX' if name == 'raw' else 'Sapiens2 -> global orient + transla
F13:0209+ tion', (12, 25), cv2.FONT_HERSHEY_SIMPLEX, .62, (255, 255, 255), 1, cv2.LINE_AA)
F13:0210     overlays.append(panel)
F13:0211     cv2.imwrite(str(a.output_dir / 'comparison.jpg'), np.concatenate(overlays, axis=1))
F13:0212     diagnostic = image.copy()
F13:0213     for j, initial_xy, final_xy in zip(selected, cpu(before[indices]), cpu(rebuilt_pixels[indices]))
F13:0213+ :
F13:0214         target_xy = (round(j['x']), round(j['y']))
F13:0215         cv2.line(diagnostic, target_xy, tuple(np.rint(final_xy).astype(int)), (0, 220, 220), 1, cv2.
F13:0215+ LINE_AA)
F13:0216         for xy, color in [(initial_xy, (255, 100, 0)), (final_xy, (0, 0, 255)), (target_xy, (0, 220,
F13:0216+ 0))]:
F13:0217             cv2.circle(diagnostic, tuple(np.rint(xy).astype(int)), 5, color, -1, cv2.LINE_AA)
F13:0218             cv2.putText(diagnostic, j['name'], (target_xy[0] + 7, target_xy[1] - 7), cv2.FONT_HERSHEY_SI
F13:0218+ MPLEX, .4, (0, 220, 0), 1, cv2.LINE_AA)

```



```

F13:0219     cv2.putText(diagnostic, 'BLUE raw | GREEN Sapiens2 | RED optimized', (15, 30), cv2.FONT_HERSHEY_
F13:0219+ SIMPLEX, .65, (255, 255, 255), 2, cv2.LINE_AA)
F13:0220     cv2.imwrite(str(a.output_dir / 'keypoint_diagnostic.jpg'), diagnostic)
F13:0221     (a.output_dir / 'optimization_report.json').write_text(json.dumps(report, indent=2) + '\n')
F13:0222     print(json.dumps(report, indent=2))
F13:0225 if __name__ == '__main__':
F13:0226     main()
F14:0001 #!/usr/bin/env python3
F14:0002 """Optimize 19 body joints and small rigid changes using automatic points.
F14:0004 Shape, hands, feet and facial parameters stay fixed. No local surface deformation.
F14:0005 """
F14:0006 import argparse
F14:0007 import hashlib
F14:0008 import json
F14:0009 import os
F14:0010 from pathlib import Path
F14:0011 import time
F14:0013 import cv2
F14:0014 import numpy as np
F14:0015 import torch
F14:0016 import torch.nn.functional as F
F14:0019 def main():
F14:0020     p = argparse.ArgumentParser(description=__doc__)
F14:0021     for key in ['repo-root', 'source-npz', 'image', 'calibration', 'automatic-keypoints', 'output-di
F14:0021+ r']:
F14:0022         p.add_argument('--' + key, type=Path, required=True)
F14:0023         p.add_argument('--steps', type=int, default=1600)
F14:0024     a = p.parse_args()
F14:0025     os.environ['EGOSMPLX_REPO_ROOT'] = str(a.repo_root.resolve())
F14:0026     from egosmplx_pipeline.bootstrap import setup
F14:0027     setup(a.repo_root)
F14:0028     from egosmplx_pipeline.body_common import BODY_POSE_NAMES, MAX_DELTA_DEG, JOINT_MAPPING, save_ob
F14:0028+ j
F14:0029     from FishEyeCalibrated import FishEyeCameraCalibrated
F14:0030     from utils.human_models import smpl_x
F14:0031     from utils.vis import render_mesh_fisheye
F14:0033     a.output_dir.mkdir(parents=True, exist_ok=False)
F14:0034     image = cv2.imread(str(a.image))
F14:0035     if image is None:
F14:0036         raise ValueError('Unreadable image')
F14:0037     h, w = image.shape[:2]
F14:0038     camera = FishEyeCameraCalibrated(str(a.calibration))
F14:0039     assert tuple(map(int, camera.img_size)) == (w, h)
F14:0040     raw = {k: v.copy() for k, v in np.load(a.source_npz).items()}
F14:0041     auto = json.loads(a.automatic_keypoints.read_text())
F14:0042     assert auto['coordinate_space'] == 'original_image_pixels'
F14:0043     assert [auto['image_width'], auto['image_height']] == [w, h]
F14:0044     assert auto['image_sha256'] == hashlib.sha256(a.image.read_bytes()).hexdigest()
F14:0045     lookup = {j['name']: j for j in auto['keypoints']}
F14:0046     # Filter automatic observations before optimization.

```

```

F14:0047     names = [n for n in JOINT_MAPPING if n in lookup and np.isfinite([lookup[n][k] for k in ['x', 'y
F14:0047+ ', 'score']])).all()
F14:0048         and lookup[n]['score'] >= .3 and 0 <= lookup[n]['x'] < w and 0 <= lookup[n]['y'] < h]
F14:0049     if len(names) < 4:
F14:0050         raise ValueError('Too few usable automatic targets')
F14:0051     t = lambda x: torch.as_tensor(x, dtype=torch.float32)
F14:0052     indices = torch.tensor([smpl_x.joints_name.index(JOINT_MAPPING[n]) for n in names])
F14:0053     auto_xy = np.array([[lookup[n]['x'], lookup[n]['y']] for n in names], dtype=np.float32)
F14:0054     layer = smpl_x.layer['neutral'].cpu().eval()
F14:0055     layer.requires_grad_(False)
F14:0056     fixed_keys = ['betas', 'left_hand_pose', 'right_hand_pose', 'jaw_pose', 'leye_pose', 'reye_pose'
F14:0056+ , 'expression']
F14:0057     fixed = {k: t(raw[k]).reshape(1, -1) for k in fixed_keys}
F14:0058     body0, glob0, cam0 = t(raw['body_pose']).reshape(21, 3), t(raw['global_orient']).reshape(3), t(r
F14:0058+ aw['transl']).reshape(3)
F14:0059     active = list(MAX_DELTA_DEG)
F14:0060     active_indices = torch.tensor([BODY_POSE_NAMES.index(n) for n in active])
F14:0061     caps = t([np.deg2rad(MAX_DELTA_DEG[n]) for n in active])
F14:0062     wrists = [smpl_x.joints_name.index('L_Wrist'), smpl_x.joints_name.index('R_Wrist')]
F14:0063     def forward(body, glob, cam):
F14:0064         out = layer(body_pose=body.reshape(1, -1), global_orient=glob.reshape(1, 3), **fixed)
F14:0065         local = out.joints[0, smpl_x.joint_idx]
F14:0066         mesh = out.vertices[0] + cam
F14:0067         joints = local + cam
F14:0068         pixels = camera.world2camera_pytorch(joints)
F14:0069         return mesh, local, joints, pixels
F14:0070     with torch.no_grad():
F14:0071         initial = tuple(v.clone() for v in forward(body0, glob0, cam0))
F14:0072         initial_wrists = initial[2][wrists].norm(dim=1)
F14:0073         wrist_floor = initial_wrists * .85
F14:0074         source_error = float((initial[0] - t(raw['smplx_mesh_cam'])).abs().max())
F14:0075         assert source_error < 1e-5
F14:0076     def cpu(x):
F14:0077         return x.detach().numpy()
F14:0078     def error(pred, target):
F14:0079         d = np.linalg.norm(pred - target, axis=1)
F14:0080         return dict(rmse_px=float(np.sqrt(np.mean(d*d))), mean_px=float(d.mean()), median_px=float(n
F14:0080+ p.median(d)), per_joint_px={n: float(v) for n, v in zip(names, d)})
F14:0081     def angle_between(x, y):
F14:0082         rx, ry = cv2.Rodrigues(np.asarray(x, dtype=np.float64))[0], cv2.Rodrigues(np.asarray(y, dtyp
F14:0082+ e=np.float64))[0]
F14:0083         return float(np.rad2deg(np.arccos(np.clip((np.trace(rx @ ry.T)-1)/2, -1, 1))))
F14:0084     settings = dict(steps=a.steps, device='cpu', target_weighting='uniform automatic targets',
F14:0085         body_lr=.0015, camera_lr=.0008, orient_lr=.0005,
F14:0086         pose_prior_weight=.0008, camera_prior_weight=.0002, orient_prior_weight=.0004,
F14:0087         smooth_l1_beta=.003, max_global_axis_angle_increment_deg=8.,
F14:0088         body_axis_angle_increment_limits_deg=MAX_DELTA_DEG,
F14:0089         wrist_distance_floor_fraction=.85, wrist_guard_weight=.002,
F14:0090         wrist_distance_floor_m=cpu(wrist_floor).tolist(), minimum_vertex_z_m=.02,
F14:0091         depth_barrier_onset_m=.03, depth_barrier_scale_m=.02, depth_barrier_weight=.02,

```

```

F14:0092             best_candidate_rule='lowest total objective among candidates satisfying depth gu
F14:0092+ ards: manual evaluation never used')
F14:0094     def fit(label, target_xy):
F14:0095         start = time.time()
F14:0096         target = t(target_xy)
F14:0097         delta = torch.zeros((len(active), 3), requires_grad=True)
F14:0098         gd = torch.zeros(3, requires_grad=True)
F14:0099         cam = cam0.clone().requires_grad_(True)
F14:0100         opt = torch.optim.Adam([{'params': [delta], 'lr': .0015}, {'params': [gd], 'lr': .0005}, {'p
F14:0100+ arams': [cam], 'lr': .0008}])
F14:0101         best_loss, best = float('inf'), None
F14:0102         history = []
F14:0103         for step in range(a.steps + 1):
F14:0104             opt.zero_grad()
F14:0105             body = body0.clone()
F14:0106             body[active_indices] = body[active_indices] + delta
F14:0107             mesh, local, joints, pixels = forward(body, glob0 + gd, cam)
F14:0108             rep = F.smooth_l1_loss((pixels[indices] - target) / t([w, h]), torch.zeros_like(target),
F14:0108+ beta=.003)
F14:0109             pp = (delta / caps[:, None]).square().mean()
F14:0110             cp = ((cam - cam0) / t([.10, .10, .18])).square().mean()
F14:0111             gp = (gd / np.deg2rad(8.)).square().mean()
F14:0112             wrist_dist = joints[wrists].norm(dim=1)
F14:0113             guard = F.relu((wrist_floor - wrist_dist) / .05).square().mean()
F14:0114             # Start the barrier before the hard near plane. The legacy penalty
F14:0115             # in raw meters was too weak relative to normalized pixel residuals.
F14:0116             depth = F.relu((.03 - mesh[:, 2].min()) / .02).square()
F14:0117             loss = rep + .0008*pp + .0002*cp + .0004*gp + .002*guard + .02*depth
F14:0118             if not torch.isfinite(loss):
F14:0119                 raise RuntimeError('Nonfinite objective')
F14:0120             feasible = bool((wrist_dist >= wrist_floor - 1e-6).all() and mesh[:, 2].min() >= .02 - 1
F14:0120+ e-6)
F14:0121             value = float(loss.detach())
F14:0122             if feasible and value < best_loss:
F14:0123                 best_loss = value
F14:0124                 best = (delta.detach().clone(), gd.detach().clone(), cam.detach().clone(), step)
F14:0125             if step % 100 == 0 or step == a.steps:
F14:0126                 item = dict(step=step, loss=value, reprojection=float(rep.detach()), feasible=feasib
F14:0126+ le,
F14:0127                             minimum_z_m=float(mesh[:, 2].min().detach()))
F14:0128                 history.append(item)
F14:0129                 print(label, item, flush=True)
F14:0130             if step == a.steps:
F14:0131                 break
F14:0132             loss.backward()
F14:0133             if not all(torch.isfinite(x.grad).all() for x in [delta, gd, cam]):
F14:0134                 raise RuntimeError('Nonfinite optimization gradients')
F14:0135             opt.step()
F14:0136             with torch.no_grad():
F14:0137                 delta.mul_(torch.minimum(torch.ones_like(caps), caps / delta.norm(dim=1).clamp_min(1

```

```

F14:0137+ e-8))[:, None])
F14:0138         gd.mul_(torch.clamp(t(np.deg2rad(8.)) / gd.norm().clamp_min(1e-8), max=1.))
F14:0139         cam[2].clamp_(.05, 3.)
F14:0140         if best is None:
F14:0141             raise RuntimeError('No feasible refinement candidate')
F14:0142         with torch.no_grad():
F14:0143             bd, gg, cc, best_step = best
F14:0144             final_body = body0.clone()
F14:0145             final_body[active_indices] += bd
F14:0146             final_global = glob0 + gg
F14:0147             mesh, local, joints, pixels = forward(final_body, final_global, cc)
F14:0148             dst = a.output_dir / label
F14:0149             dst.mkdir()
F14:0150             saved = {k: v.copy() for k, v in raw.items()}
F14:0151             saved.update(body_pose_rigid_input=raw['body_pose'], global_orient_rigid_input=raw['global_o
F14:0151+ rient'], transl_rigid_input=raw['transl'],
F14:0152                         body_pose=cpu(final_body).reshape(raw['body_pose'].shape), global_orient=cpu(fi
F14:0152+ nal_global).reshape(1, 3), transl=cpu(cc).reshape(1, 3),
F14:0153                         smplx_mesh_cam=cpu(mesh), smplx_body_joints_3d_local=cpu(local[:25]), smplx_bod
F14:0153+ y_joints_3d_render_cam=cpu(joints[:25]),
F14:0154                         smplx_body_joints_2d_render_cam=cpu(pixels[:25]), smplx_joint_cam_root_relative
F14:0154+ =cpu(local-local[:1]),
F14:0155                         stage2_target_joint_pixels=target_xy, stage2_selected_joint_indices=cpu(indices
F14:0155+ ))
F14:0156             assert all(np.array_equal(saved[k], raw[k]) for k in fixed_keys)
F14:0157             np.savez_compressed(dst / 'refined_params.npz', **saved)
F14:0158             # Independently reload the actual artifact and regenerate its geometry.
F14:0159             reloaded = np.load(dst / 'refined_params.npz')
F14:0160             assert all(np.isfinite(reloaded[k]).all() for k in reloaded.files if np.issubdtype(reloaded[
F14:0160+ k].dtype, np.number))
F14:0161             with torch.no_grad():
F14:0162                 rm, _, _, rp = forward(t(reloaded['body_pose']).reshape(21, 3), t(reloaded['global_orien
F14:0162+ t']).reshape(3), t(reloaded['transl']).reshape(3))
F14:0163                 reload_mesh_error = float(np.abs(cpu(rm)-reloaded['smplx_mesh_cam']).max())
F14:0164                 reload_pixel_error = float(np.abs(cpu(rp[:25])-reloaded['smplx_body_joints_2d_render_cam'])).
F14:0164+ max())
F14:0165                 assert reload_mesh_error < 1e-5 and reload_pixel_error < .01
F14:0166                 save_obj(cpu(mesh), smpl_x.face, dst / 'refined_mesh.obj')
F14:0167                 overlay = render_mesh_fisheye(image, cpu(mesh), smpl_x.face, {}, camera, mode='wireframe', m
F14:0167+ esh_color=(255, 255, 255), alpha=.78, proj_trans=None)
F14:0168                 cv2.imwrite(str(dst / 'refined_wireframe.jpg'), overlay)
F14:0169                 diag = image.copy()
F14:0170                 for n, target_pt, pred in zip(names, target_xy, cpu(pixels[indices])):
F14:0171                     tx, px = tuple(np rint(target_pt).astype(int)), tuple(np rint(pred).astype(int))
F14:0172                     cv2.line(diag, tx, px, (0, 220, 220), 1, cv2.LINE_AA)
F14:0173                     cv2.circle(diag, tx, 6, (0, 220, 0), -1, cv2.LINE_AA)
F14:0174                     cv2.circle(diag, px, 4, (0, 0, 255), -1, cv2.LINE_AA)
F14:0175                     cv2.putText(diag, n, (tx[0]+6, tx[1]-6), cv2.FONT_HERSHEY_SIMPLEX, .4, (0, 180, 0), 1, c
F14:0175+ v2.LINE_AA)
F14:0176                 cv2.imwrite(str(dst / 'keypoint_diagnostic.jpg'), diag)

```

```

F14:0177         increments = np.rad2deg(cpu(bd.norm(dim=1)))
F14:0178         actual_angles = {n: angle_between(cpu(final_body[i]), cpu(body0[i])) for n, i in zip(active,
F14:0178+ active_indices)}
F14:0179         report = dict(branch=label, target_source='Sapiens2',
F14:0180                         manual_labels_used_for_optimization=False,
F14:0181                         target_fit_before=error(cpu(initial[3][indices]), target_xy), target_fit_after
F14:0181+ =error(cpu(pixels[indices]), target_xy),
F14:0182                         best_step=best_step, best_objective=best_loss, elapsed_seconds=time.time()-sta
F14:0182+ rt,
F14:0183                         pose_increment_deg={n: float(v) for n, v in zip(active, increments)}, pose_geo
F14:0183+ desic_change_deg=actual_angles,
F14:0184                         joints_at_increment_limit=[n for n, v in zip(active, increments) if v >= MAX_D
F14:0184+ ELTA_DEG[n]-.05],
F14:0185                         global_geodesic_change_deg=angle_between(cpu(final_global), cpu(glob0)),
F14:0186                         cam_trans_m=cpu(cc).tolist(), cam_trans_change_m=cpu(cc-cam0).tolist(),
F14:0187                         wrist_camera_distance_m=cpu(joints[wrists].norm(dim=1)).tolist(),
F14:0188                         initial_wrist_camera_distance_m=cpu(initial_wrists).tolist(), minimum_vertex_z
F14:0188+ _m=float(mesh[:, 2].min()),
F14:0189                         fixed_fields_bitwise_equal=fixed_keys, reload_mesh_error_m=reload_mesh_error,
F14:0189+ reload_pixel_error_px=reload_pixel_error,
F14:0190                         history=history)
F14:0191         (dst / 'refinement_report.json').write_text(json.dumps(report, indent=2)+'\n')
F14:0192         return report, cpu(pixels[indices]), cpu(mesh), overlay, cpu(final_body)
F14:0194         automatic = fit('automatic', auto_xy)
F14:0195         if True:
F14:0196             rows = {
F14:0197                 'rigid_shared_input': {'against_sapiens2': error(cpu(initial[3][indices]), auto_xy)},
F14:0198                 'automatic_stage2': {'against_sapiens2': error(automatic[1], auto_xy)},
F14:0199             }
F14:0200             summary = dict(source_npz=str(a.source_npz), source_sha256=hashlib.sha256(a.source_npz.read_
F14:0200+ bytes()).hexdigest(),
F14:0201                             automatic_keypoints=str(a.automatic_keypoints), image=str(a.image), calibrati
F14:0201+ on=str(a.calibration),
F14:0202                             comparison='Automatic Sapiens2 stage-two refinement from the supplied rigid i
F14:0202+ nitialization',
F14:0203                             names=names, settings=settings, source_mesh_reload_error_m=source_error, resu
F14:0203+ lts=rows,
F14:0204                             automatic_diagnostics=automatic[0])
F14:0205             (a.output_dir / 'comparison_summary.json').write_text(json.dumps(summary, indent=2)+'\n')
F14:0206             rigid_overlay = render_mesh_fisheye(image, cpu(initial[0]), smpl_x.face, {}, camera, mode='w
F14:0206+ ireframe', mesh_color=(255, 255, 255), alpha=.78, proj_trans=None)
F14:0207             panels = []
F14:0208             for title, overlay, key in [('Rigid input', rigid_overlay, 'rigid_shared_input'), ('Stage 2:
F14:0208+ Sapiens2 targets', automatic[3], 'automatic_stage2')]:
F14:0209                 panel = cv2.resize(overlay, (640, round(h*640/w)))
F14:0210                 cv2.rectangle(panel, (0, 0), (640, 62), (25, 25, 25), -1)
F14:0211                 cv2.putText(panel, title, (12, 25), cv2.FONT_HERSHEY_SIMPLEX, .65, (255, 255, 255), 1, c
F14:0211+ v2.LINE_AA)
F14:0212                 cv2.putText(panel, f"Sapiens2 RMSE: {rows[key]['against_sapiens2']['rmse_px']:.2f} px",
F14:0212+ (12, 49), cv2.FONT_HERSHEY_SIMPLEX, .55, (210, 210, 210), 1, cv2.LINE_AA)

```

```

F14:0213         panels.append(panel)
F14:0214         cv2.imwrite(str(a.output_dir / 'stage2_comparison.jpg'), np.concatenate(panels, axis=1))
F14:0215         print('AUTOMATIC_ONLY', json.dumps(rows, indent=2), flush=True)
F14:0216         return
F14:0219 if __name__ == '__main__':
F14:0220     main()
F15:0001 """FP64 Horner evaluation of the calibrated fisheye polynomial with autograd."""
F15:0002 import torch
F15:0004 def stable_world2camera(self, points, normalize=False):
F15:0005     dtype = points.dtype
F15:0006     axes = torch.as_tensor(self.camera_axis_transform, dtype=torch.float64, device=points.device)
F15:0007     xyz = points.to(torch.float64) @ axes.t()
F15:0008     radius = torch.linalg.vector_norm(xyz[:, :2], dim=1)
F15:0009     if not bool((radius != 0).all()):
F15:0010         raise ValueError('norm is zero!')
F15:0011     theta = torch.atan(-xyz[:, 2] / radius)
F15:0012     coefficients = self.fisheye_inverse_polynomial
F15:0013     rho = torch.full_like(theta, float(coefficients[-1]))
F15:0014     for coefficient in reversed(coefficients[:-1]):
F15:0015         rho = rho * theta + float(coefficient)
F15:0016     x, y = xyz[:, 0] * rho / radius, xyz[:, 1] * rho / radius
F15:0017     u = float(self.c) * x + float(self.d) * y + float(self.img_center[0])
F15:0018     v = float(self.e) * x + y + float(self.img_center[1])
F15:0019     pixels = torch.stack((u, v), dim=1)
F15:0020     if normalize:
F15:0021         width, height = self.img_size
F15:0022         assert width > height
F15:0023         pixels = (pixels - pixels.new_tensor([(width-height)//2, 0])) / (height-1) * 2 - 1
F15:0024     return pixels.to(dtype)
F16:0001 """One-to-one automatic hand assignment, frozen before fusion."""
F16:0002 import itertools
F16:0003 import numpy as np
F16:0004 SIDES = ["left", "right"]
F16:0006 def assign(w,lookup,initial_uv,width=1280,height=720):
F16:0007     """Global one-to-one assignment, using only automatic wrist evidence."""
F16:0008     costs={}; details={}
F16:0009     for si,s in enumerate(SIDES):
F16:0010         a=lookup.get(s+'_wrist', dict(score=0,x=0,y=0))
F16:0011         use= a['score']>=.3 and np.isfinite([a['x'],a['y']]).all() and 0<=a['x']<width and 0<=a['y']
F16:0011+ <height
F16:0012         anchor=np.array([a['x'],a['y']]) if use else initial_uv[12+si]
F16:0013         for i,c in enumerate(w['detector_confidence']):
F16:0014             uv=w['joints_2d'][i];dist=float(np.linalg.norm(uv[0]-anchor))
F16:0015             agree=bool(w['is_right'][i]>.5)==(s=='right')
F16:0016             valid=np.isfinite(uv).all(axis=1)&(uv[:,0]>=0)&(uv[:,0]<width)&(uv[:,1]>=0)&(uv[:,1]<hei
F16:0016+ ght)
F16:0017             if c>=.25 and dist<=150 and valid.sum()>=12:
F16:0018                 costs[si,i]=dist+(0 if agree else 45)+20*(1-float(c))
F16:0019                 details[si,i]=dict(candidate=i,confidence=float(c),wrist_distance_px=dist,detector_h
F16:0019+ andedness_agrees=agree,anchor_source='Sapiens2' if use else 'automatic_SMPLX',valid_keypoints=int(va

```

```

F16:0019+ lid.sum()))
F16:0020     options=[[-1]+[i for a,i in costs if a==s] for s in range(2)]
F16:0021     combos=[c for c in itertools.product(*options) if c[0]<0 or c[1]<0 or c[0]!=c[1]]
F16:0022     best=min(combos,key=lambda c:sum(180 if i<0 else costs[s,i] for s,i in enumerate(c)))
F16:0023     return {s:details[k,i] for k,(s,i) in enumerate(zip(SIDES,best)) if i>=0}
F17:0001 """Preserve MANO projection through calibrated rays and radial distance."""
F17:0002 import numpy as np
F17:0004 def native_projection(points_camera, focal, center):
F17:0005     normalized = points_camera[:, :2] / points_camera[:, 2:3].clip(1e-8)
F17:0006     return focal * normalized + np.asarray(center, dtype=np.float64)
F17:0009 def lift_mano_to_fisheye(camera, wilor, candidate, body_wrist, body_wrist_uv):
F17:0010     vertices_camera_native = (
F17:0011         np.asarray(wilor["vertices_3d_local"][candidate], dtype=np.float64)
F17:0012         + np.asarray(wilor["cam_t"][candidate], dtype=np.float64)[None]
F17:0013     )
F17:0014     joints_camera_native = (
F17:0015         np.asarray(wilor["joints_3d_local"][candidate], dtype=np.float64)
F17:0016         + np.asarray(wilor["cam_t"][candidate], dtype=np.float64)[None]
F17:0017     )
F17:0018     focal = float(wilor['focal_length_px'])
F17:0019     center = np.asarray(wilor['image_size_wh'], dtype=np.float64) / 2
F17:0020     vertex_uv = native_projection(vertices_camera_native, focal, center)
F17:0021     joint_uv = native_projection(joints_camera_native, focal, center)
F17:0022     # Keep the body wrist fixed and translate the complete hand projection.
F17:0023     # This residual can be large when the body estimate is inaccurate.
F17:0024     uv_shift = np.asarray(body_wrist_uv, dtype=np.float64) - joint_uv[0]
F17:0025     vertex_uv += uv_shift[None]
F17:0026     joint_uv += uv_shift[None]
F17:0028     native_root_depth = float(np.linalg.norm(joints_camera_native[0]))
F17:0029     body_root_depth = float(np.linalg.norm(body_wrist))
F17:0030     vertex_relative_depth = np.linalg.norm(vertices_camera_native, axis=1) - native_root_depth
F17:0031     joint_relative_depth = np.linalg.norm(joints_camera_native, axis=1) - native_root_depth
F17:0032     vertex_depth = np.clip(body_root_depth + vertex_relative_depth, 0.05, None)
F17:0033     joint_depth = np.clip(body_root_depth + joint_relative_depth, 0.05, None)
F17:0034     vertices_fisheye = camera.camera2world(vertex_uv, vertex_depth)
F17:0035     joints_fisheye = camera.camera2world(joint_uv, joint_depth)
F17:0036     return {
F17:0037         "vertices": vertices_fisheye.astype(np.float32),
F17:0038         "joints": joints_fisheye.astype(np.float32),
F17:0039         "vertex_uv": vertex_uv.astype(np.float32),
F17:0040         "joint_uv": joint_uv.astype(np.float32),
F17:0041         "uv_shift": uv_shift.astype(np.float32),
F17:0042         "body_wrist_depth": body_root_depth,
F17:0043         "native_wilor_root_depth": native_root_depth,
F17:0044     }
F18:0001 """Mesh boundary and breadth-first ring distances."""
F18:0002 from collections import Counter, deque
F18:0003 import numpy as np
F18:0005 def boundary_edges(faces):
F18:0006     counts = Counter(

```

```

F18:0007         tuple(sorted((int(first), int(second))))
F18:0008         for face in np.asarray(faces, dtype=np.int64)
F18:0009             for first, second in ((face[0], face[1]), (face[1], face[2]), (face[2], face[0]))
F18:0010     )
F18:0011     return {edge for edge, count in counts.items() if count == 1}
F18:0014 def graph_distance_from_boundary(faces, boundary_vertices, vertex_count):
F18:0015     adjacency = [set() for _ in range(vertex_count)]
F18:0016     for face in np.asarray(faces, dtype=np.int64):
F18:0017         for first, second in ((face[0], face[1]), (face[1], face[2]), (face[2], face[0])):
F18:0018             adjacency[int(first)].add(int(second))
F18:0019             adjacency[int(second)].add(int(first))
F18:0020     sentinel = np.iinfo(np.int32).max
F18:0021     distance = np.full(vertex_count, sentinel, dtype=np.int32)
F18:0022     queue = deque()
F18:0023     for vertex in boundary_vertices:
F18:0024         distance[int(vertex)] = 0
F18:0025         queue.append(int(vertex))
F18:0026     while queue:
F18:0027         first = queue.popleft()
F18:0028         for second in adjacency[first]:
F18:0029             if distance[second] > distance[first] + 1:
F18:0030                 distance[second] = distance[first] + 1
F18:0031                 queue.append(second)
F18:0032     if np.any(distance == sentinel):
F18:0033         raise RuntimeError("MANO topology contains vertices disconnected from wrist boundary")
F18:0034     return distance
F19:0001 """Seven-ring baseline used before the archived wrist/contour/surface refinement."""
F19:0002 from collections import Counter
F19:0003 import numpy as np
F19:0004 import cv2
F19:0005 import torch
F19:0006 from egosmplx_pipeline.io import arr, rmse
F19:0007 from egosmplx_pipeline.topology import boundary_edges, graph_distance_from_boundary
F19:0008 from egosmplx_pipeline.lift import lift_mano_to_fisheye
F19:0009 from egosmplx_pipeline.render import render
F19:0010 SIDES = ['left', 'right']
F19:0011 PARAMS = ['betas', 'jaw_pose', 'leye_pose', 'reye_pose', 'expression', 'body_pose', 'global_orient',
F19:0011+ 'left_hand_pose', 'right_hand_pose']
F19:0013 def initialize(model):
F19:0014     global smpl_x, LAYER, FACES, JI
F19:0015     smpl_x = model
F19:0016     LAYER = model.layer['neutral'].cpu().eval()
F19:0017     LAYER.requires_grad_(False)
F19:0018     FACES = np.asarray(model.face, dtype=np.int32)
F19:0019     JI = {n: i for i, n in enumerate(model.joints_name)}
F19:0021 def geometry(p):
F19:0022     with torch.no_grad():
F19:0023         r=LAYER(**{k:torch.as_tensor(p[k],dtype=torch.float32).reshape(1,-1) for k in PARAMS})
F19:0024     return (arr(r.vertices[0])+p['trans1'].reshape(3),
F19:0025             arr(r.joints[0,smpl_x.joint_idx])+p['trans1'].reshape(3))

```



```

F19:0028 def edge_counts(f):
F19:0029     return Counter(tuple(sorted((int(a),int(b)))) for tri in f for a,b in zip(tri,np.roll(tri,-1)))
F19:0032 def oriented_edges(f):
F19:0033     return Counter((int(a),int(b)) for tri in f for a,b in zip(tri,np.roll(tri,-1)))
F19:0036 def stitch(p,w,matches,cam):
F19:0037     original,j=geometry(p); uv=cam.world2camera(j.astype(np.float64))
F19:0038     vertices=original.copy(); keep=np.ones(len(FACES),bool); additions=[]; extra={}; reports={}
F19:0039     mf=np.asarray(w['mano_faces'],dtype=np.int32)
F19:0040     boundary=boundary_edges(mf); bv=np.array(sorted({i for e in boundary for i in e}))
F19:0041     assert len(bv)==len(boundary)==16
F19:0042     rings=graph_distance_from_boundary(mf,bv,778)
F19:0043     t=np.clip(rings.astype(np.float32)/7,0,1); blend=t*t*(3-2*t)
F19:0044     for side,a in matches.items():
F19:0045         ids=np.asarray(smpl_x.hand_vertex_idx[side+' _hand'],dtype=np.int64)
F19:0046         internal=np.isin(FACES,ids).all(axis=1)
F19:0047         inverse=np.full(len(vertices),-1,dtype=int); inverse[ids]=np.arange(778)
F19:0048         assert boundary_edges(inverse[FACES[internal]])==boundary
F19:0049         keep &= ~internal
F19:0050         k=12+SIDES.index(side)
F19:0051         lifted=lift_mano_to_fisheye(cam,w,a['candidate'],j[k],uv[k])
F19:0052         target=lifted['vertices'].copy()
F19:0053         # Preserve pixel rays while enforcing the original positive near-plane guard.
F19:0054         assert np.all(target[:,2]>0), 'Target ray behind the camera: reject rather than silently dra
F19:0054+ w it'
F19:0055         adjusted=target[:,2]<.020001
F19:0056         target*=np.maximum(1,.020001/target[:,2])[:,None]
F19:0057         vertices[ids]=(1-blend[:,None])*original[ids]+blend[:,None]*target
F19:0058         assert np.array_equal(vertices[ids[bv]],original[ids[bv]])
F19:0059         faces=ids[mf]
F19:0060         # Native MANO winding may disagree on the reflected hand. Match the body seam.
F19:0061         original_edge=oriented_edges(FACES[internal]); new_edge=oriented_edges(faces)
F19:0062         a0,b0=ids[list(next(iter(boundary)))]
F19:0063         reversed_winding=original_edge[(int(a0),int(b0))]!=new_edge[(int(a0),int(b0))]
F19:0064         if reversed_winding: faces=faces[:,[0,2,1]]
F19:0065         additions.append(faces)
F19:0066         extra.update({side+' _local_to_mesh':ids,side+' _lifted_vertices':target,
F19:0067                     side+' _blend_weights':blend,side+' _target_vertex_uv':lifted['vertex_uv'],
F19:0068                     side+' _wrist_loop_local':bv,side+' _candidate':np.array(a['candidate'])})
F19:0069         projected=cam.world2camera(vertices[ids].astype(np.float64))
F19:0070         interior=blend==1
F19:0071         reports[side]=dict(candidate=a['candidate'],wrist_shift_px=lifted['uv_shift'].tolist(),
F19:0072                          wrist_shift_norm_px=float(np.linalg.norm(lifted['uv_shift'])),
F19:0073                          blend_rings=7,wrist_boundary_vertex_count=16,boundary_max_change_m=0.,
F19:0074                          reversed_winding=bool(reversed_winding),near_plane_adjusted_vertices=int(a
F19:0074+ djusted.sum()),
F19:0075                          unblended_vertex_projection_rmse_px=rmse(projected[interior],lifted['verte
F19:0075+ x_uv'][interior]))
F19:0076         faces=np.concatenate([FACES[keep],*additions]).astype(np.int32)
F19:0077         counts=edge_counts(faces); directions=oriented_edges(faces)
F19:0078         for side in matches:

```

```

F19:0079         ids=extra[side+' _local_to_mesh']
F19:0080         for a,b in boundary:
F19:0081             u,v=int(ids[a]),int(ids[b]); assert counts[tuple(sorted((u,v)))]==2
F19:0082             assert directions[u,v]==directions[v,u]==1
F19:0083         # Check all MANO interior edges too, including corrected winding.
F19:0084         for a,b in edge_counts(mf):
F19:0085             u,v=int(ids[a]),int(ids[b]); assert counts[tuple(sorted((u,v)))]==2
F19:0086             assert directions[u,v]==directions[v,u]==1
F19:0087         reports[side]['seam_and_hand_edges_two_opposite_faces']=True
F19:0088         hand_joints=np.stack([smp1_x.orig_hand_regressor[s]@vertices for s in SIDES]).astype(np.float32)
F19:0089         assert np.isfinite(vertices).all() and vertices[:,2].min()>=.02-1e-6
F19:0090         untouched=np.ones(len(vertices),bool)
F19:0091         for s in matches: untouched[extra[s+' _local_to_mesh']]=False
F19:0092         assert np.array_equal(vertices[untouched],original[untouched])
F19:0093         return dict(vertices_cam=vertices,faces=faces,hand_joints_cam=hand_joints,body_joints_cam=j,
F19:0094                     method=np.array('automatic_projection_preserving_MANO_wrist_seam'),**extra),reports
F19:0097 def draw_geometry(image,cam,mesh):
F19:0098     image=render(image,mesh['vertices_cam'].astype(np.float64),mesh['faces'],cam,.12)
F19:0099     body=cam.world2camera(mesh['body_joints_cam'].astype(np.float64))
F19:0100     def segment(points,a,b,color):
F19:0101         q=points[[a,b]]
F19:0102         if np.isfinite(q).all() and np.abs(q).max()<100000:
F19:0103             cv2.line(image,tuple(np rint(q[0])).astype(int),tuple(np rint(q[1])).astype(int)),color,2
F19:0103+ ,cv2.LINE_AA)
F19:0104         for a,b in [('Neck','L_Shoulder'),('Neck','R_Shoulder'),('L_Shoulder','L_Elbow'),('L_Elbow','L_W
F19:0104+ rist'),('R_Shoulder','R_Elbow'),('R_Elbow','R_Wrist'),('L_Shoulder','L_Hip'),('R_Shoulder','R_Hip'),
F19:0104+ ('L_Hip','R_Hip')]:
F19:0105             segment(body,JI[a],JI[b],(0,185,235))
F19:0106         for k,s in enumerate(SIDES):
F19:0107             points=cam.world2camera(mesh['hand_joints_cam'][k].astype(np.float64))
F19:0108             for start in [1,5,9,13,17]:
F19:0109                 chain=[0]+list(range(start,start+4))
F19:0110                 color=((225,85,30) if s=='left' else (30,45,225)) if s+' _local_to_mesh' in mesh else (15
F19:0110+ 0,150,150)
F19:0111                 for a,b in zip(chain[:-1],chain[1:]):segment(points,a,b,color)
F19:0112         return image
F20:0001 """Clipped fisheye mesh overlays and Wavefront OBJ export."""
F20:0002 import numpy as np
F20:0003 import cv2
F20:0005 def read_obj(path):
F20:0006     vertices, faces = [], []
F20:0007     with path.open("r", encoding="utf-8") as handle:
F20:0008         for line in handle:
F20:0009             if line.startswith("v "):
F20:0010                 vertices.append([float(v) for v in line.split()[1:4]])
F20:0011             elif line.startswith("f "):
F20:0012                 faces.append([int(v.split("/") [0]) - 1 for v in line.split()[1:4]])
F20:0013     return np.asarray(vertices, np.float64), np.asarray(faces, np.int32)
F20:0016 def render(image, vertices, faces, camera, max_edge_fraction):
F20:0017     uv = camera.world2camera(vertices)

```

```

F20:0018     axes = np.asarray(getattr(camera, "camera_axis_transform", np.eye(3)), np.float64)
F20:0019     camera_vertices = vertices @ axes.T
F20:0020     valid = (
F20:0021         np.isfinite(camera_vertices).all(axis=1)
F20:0022         & np.isfinite(uv).all(axis=1)
F20:0023         & (camera_vertices[:, 2] > 1e-5)
F20:0024     )
F20:0025     height, width = image.shape[:2]
F20:0026     max_edge = max(40.0, max_edge_fraction * np.hypot(width, height))
F20:0027     overlay = image.copy()
F20:0028     for tri in faces:
F20:0029         if not valid[tri].all():
F20:0030             continue
F20:0031         points = uv[tri]
F20:0032         if (
F20:0033             points[:, 0].max() < 0 or points[:, 0].min() >= width
F20:0034             or points[:, 1].max() < 0 or points[:, 1].min() >= height
F20:0035         ):
F20:0036             continue
F20:0037         edges = points[[1, 2, 0]] - points
F20:0038         if np.linalg.norm(edges, axis=1).max() > max_edge:
F20:0039             continue
F20:0040         cv2.polylines(
F20:0041             overlay, [np rint(points).astype(np.int32)], True,
F20:0042             (255, 255, 255), 1, cv2.LINE_AA,
F20:0043         )
F20:0044     return cv2.addWeighted(image, 0.22, overlay, 0.78, 0)
F20:0046 def write_obj(path, vertices, faces):
F20:0047     with path.open("w", encoding="utf-8") as handle:
F20:0048         for vertex in vertices:
F20:0049             handle.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
F20:0050         for face in faces:
F20:0051             handle.write(f"f {face[0] + 1} {face[1] + 1} {face[2] + 1}\n")
F21:0001 """Static, shareable three-stage comparison without remote asset dependencies."""
F21:0002 import html
F21:0003 from pathlib import Path
F21:0004 import cv2
F21:0005 import numpy as np
F21:0006 from egosmplx_pipeline.io import write
F21:0009 def build(output, records):
F21:0010     output = Path(output)
F21:0011     tiles, sections = [], []
F21:0012     for row in records:
F21:0013         identifier = row['id']
F21:0014         panels = []
F21:0015         for stage, title in [('raw', 'Raw EgoSMPLX'), ('body', 'Sapiens2 two-stage body fit'), ('fus
F21:0015+ ed', 'MANO + contour + harmonic surface')]:
F21:0016             image = cv2.imread(str(output / 'frames' / identifier / stage / 'wireframe.jpg'))
F21:0017             if image is None:
F21:0018                 raise ValueError('Missing stage overlay: ' + identifier + '/' + stage)

```

```

F21:0019         panel = cv2.resize(image, (640, 360))
F21:0020         cv2.rectangle(panel, (0, 0), (640, 32), (20, 20, 20), -1)
F21:0021         cv2.putText(panel, title, (12, 23), cv2.FONT_HERSHEY_SIMPLEX, .55, (255, 255, 255), 1, c
F21:0021+ v2.LINE_AA)
F21:0022         panels.append(panel)
F21:0023         tile = np.concatenate(panels, axis=1)
F21:0024         image_name = identifier + '_comparison.jpg'
F21:0025         cv2.imwrite(str(output / image_name), tile)
F21:0026         tiles.append(cv2.resize(tile, (1440, 270)))
F21:0027         sections.append('<section><h2>' + html.escape(identifier) + '</h2><p>Matched hands: ' + str(
F21:0027+ row['matched_hands']) +
F21:0028                 '</p></section>')
F21:0029         cv2.imwrite(str(output / 'contact_sheet.jpg'), np.concatenate(tiles, axis=0))
F21:0030         body = [x for r in records for x in r['body_error_px']]
F21:0031         hands = [x for r in records for x in r['hand_error_px']]
F21:0032         summary = dict(frame_count=len(records), matched_hands=sum(r['matched_hands'] for r in records),
F21:0033                 body_target_count=len(body), hand_target_count=len(hands),
F21:0034                 body_rmse_px=float(np.sqrt(np.mean(np.square(body)))),
F21:0035                 hand_rmse_px=float(np.sqrt(np.mean(np.square(hands)))) if hands else None,
F21:0036                 all_reload_checks_passed=all(r['passed'] for r in records),
F21:0037                 manual_annotations_used=False, local_surface_optimizer=any(r.get('local_surface_o
F21:0037+ ptimizer') for r in records),
F21:0038                 pipeline_profile='session_hand6_full_pipeline_20260926_v1',
F21:0039                 reference='Sapiens2 body points / WiLoR hand points; not human ground truth')
F21:0040         write(output / 'summary.json', summary)
F21:0041         text = '<!doctype html><html lang="zh-CN"><meta charset="utf-8"><meta name="viewport" content="w
F21:0041+ idth=device-width">' + \
F21:0042                 '<title>EgoSmplx 三阶段对比</title><style>body{background:#161b22;color:#eee;font:16px sa
F21:0042+ ns-serif;margin:24px}img{width:100%}section{margin:32px 0}</style>' + \
F21:0043                 '<h1>EgoSmplx V1.0 三阶段输出</h1><p>自动目标拟合误差，不是人工真值精度。对应指定结果包的
F21:0043+ 腕点校正、轮廓约束与局部谐波连接流程。</p>' + ''.join(sections) + '</html>'
F21:0044         (output / 'index.html').write_text(text, encoding='utf-8')
F22:0001 import json
F22:0002 import numpy as np
F22:0003 import torch
F22:0004 from copy import deepcopy
F22:0007 class FishEyeCameraCalibrated:
F22:0008     def __init__(self, calibration_file_path, use_gpu=False):
F22:0009         with open(calibration_file_path) as f:
F22:0010             calibration_data = json.load(f)
F22:0011             self.img_size = np.array(calibration_data['size']) # w, h
F22:0012             self.fisheye_polynomial = np.array(calibration_data['polynomialC2W'])
F22:0013             self.fisheye_inverse_polynomial = np.array(calibration_data['polynomialW2C'])
F22:0014             if 'intrinsic' in calibration_data:
F22:0015                 self.intrinsic = np.array(calibration_data['intrinsic'])
F22:0016                 self.img_center = np.array([
F22:0017                     self.intrinsic[0][2], self.intrinsic[1][2]
F22:0018                 ])
F22:0019             elif 'image_center' in calibration_data:
F22:0020                 self.img_center = np.asarray(

```

```

F22:0021         calibration_data['image_center'], dtype=np.float64
F22:0022     )
F22:0023     self.intrinsic = np.eye(4, dtype=np.float64)
F22:0024     self.intrinsic[0, 2] = self.img_center[0]
F22:0025     self.intrinsic[1, 2] = self.img_center[1]
F22:0026 else:
F22:0027     raise KeyError(
F22:0028         "Calibration must contain either 'intrinsic' or 'image_center'"
F22:0029     )
F22:0030 affine = calibration_data.get('affine')
F22:0031 if affine is None:
F22:0032     affine = [
F22:0033         calibration_data.get('c', 1.0),
F22:0034         calibration_data.get('d', 0.0),
F22:0035         calibration_data.get('e', 0.0),
F22:0036     ]
F22:0037 self.affine = np.asarray(affine, dtype=np.float64).reshape(3)
F22:0038 self.c, self.d, self.e = self.affine
F22:0039 self.affine_det = self.c - self.d * self.e
F22:0040 if not np.isfinite(self.affine).all() or abs(self.affine_det) < 1e-12:
F22:0041     raise ValueError('Calibration affine parameters are invalid')
F22:0042 self.camera_axis_transform = np.asarray(
F22:0043     calibration_data.get('camera_axis_transform', np.eye(3)),
F22:0044     dtype=np.float64,
F22:0045 ).reshape(3, 3)
F22:0046 if not np.isfinite(self.camera_axis_transform).all():
F22:0047     raise ValueError("camera_axis_transform must contain finite values")
F22:0048 if not np.allclose(
F22:0049     self.camera_axis_transform @ self.camera_axis_transform.T,
F22:0050     np.eye(3),
F22:0051     atol=1e-6,
F22:0052 ):
F22:0053     raise ValueError("camera_axis_transform must be an orthonormal rotation")
F22:0054 self.camera_axis_transform_inv = self.camera_axis_transform.T
F22:0055 self.use_gpu = use_gpu
F22:0056 # self.img_center = np.array([self.img_size[0] / 2, self.img_size[1] / 2])
F22:0058 def camera2world(self, point: np.ndarray, depth: np.ndarray):
F22:0059     """
F22:0060     point: np.ndarray of 2D points on image (n * 2)
F22:0061     depth: np.ndarray of depth of every 2D points (n)
F22:0062     """
F22:0063     depth = depth.astype(np.float32)
F22:0064     point_centered = point.astype(np.float32) - self.img_center
F22:0065     distorted_x = point_centered[:, 0]
F22:0066     distorted_y = point_centered[:, 1]
F22:0067     x = (distorted_x - self.d * distorted_y) / self.affine_det
F22:0068     y = (-self.e * distorted_x + self.c * distorted_y) / self.affine_det
F22:0069     distance_from_center = np.sqrt(np.square(x) + np.square(y))
F22:0071     z = np.polyval(p=self.fisheye_polynomial[:, :-1], x=distance_from_center)
F22:0072     point_3d = np.array([x, y, -z]) # 3, n

```

```

F22:0073         norm = np.linalg.norm(point_3d, axis=0)
F22:0074         point_3d = point_3d / norm * depth
F22:0075         point_3d = point_3d.transpose()
F22:0076         return point_3d @ self.camera_axis_transform_inv.T
F22:0078     def camera2world_ray(self, point: np.ndarray):
F22:0079         """
F22:0080         calculate the ray direction from the image points
F22:0081         point: np.ndarray of 2D points on image (n * 2)
F22:0082         depth: np.ndarray of depth of every 2D points (n)
F22:0083         """
F22:0084         point_centered = point.astype(np.float32) - self.img_center
F22:0085         distorted_x = point_centered[:, 0]
F22:0086         distorted_y = point_centered[:, 1]
F22:0087         x = (distorted_x - self.d * distorted_y) / self.affine_det
F22:0088         y = (-self.e * distorted_x + self.c * distorted_y) / self.affine_det
F22:0089         distance_from_center = np.sqrt(np.square(x) + np.square(y))
F22:0091         z = np.polyval(p=self.fisheye_polynomial[:, :-1], x=distance_from_center)
F22:0092         point_3d = np.array([x, y, -z]) # 3, n
F22:0093         norm = np.linalg.norm(point_3d, axis=0)
F22:0094         point_3d = point_3d / norm
F22:0095         point_3d = point_3d.transpose()
F22:0096         return point_3d @ self.camera_axis_transform_inv.T
F22:0098     def getPolyVal(self, p, x):
F22:0099         curVal = torch.zeros_like(x)
F22:0100         for curValIndex in range(len(p) - 1):
F22:0101             curVal = (curVal + p[curValIndex]) * x
F22:0102         return curVal + p[len(p) - 1]
F22:0104     def camera2world_pytorch(self, point: torch.Tensor, depth: torch.Tensor):
F22:0105         """
F22:0106         point: np.ndarray of 2D points on image (n * 2)
F22:0107         depth: np.ndarray of depth of every 2D points (n)
F22:0108         """
F22:0109         img_center = torch.from_numpy(self.img_center).float().to(point.device)
F22:0110         point_centered = point.float() - img_center
F22:0111         distorted_x = point_centered[:, 0]
F22:0112         distorted_y = point_centered[:, 1]
F22:0113         c = torch.as_tensor(self.c, dtype=point.dtype, device=point.device)
F22:0114         d = torch.as_tensor(self.d, dtype=point.dtype, device=point.device)
F22:0115         e = torch.as_tensor(self.e, dtype=point.dtype, device=point.device)
F22:0116         det = torch.as_tensor(self.affine_det, dtype=point.dtype, device=point.device)
F22:0117         x = (distorted_x - d * distorted_y) / det
F22:0118         y = (-e * distorted_x + c * distorted_y) / det
F22:0119         distance_from_center = torch.sqrt(torch.square(x) + torch.square(y))
F22:0121         z = self.getPolyVal(p=self.fisheye_polynomial[:, :-1], x=distance_from_center)
F22:0122         point_3d = torch.stack([x, y, -z]).float() # 3, n
F22:0123         norm = torch.norm(point_3d, dim=0)
F22:0124         point_3d = point_3d / norm * depth.float()
F22:0125         point_3d = point_3d.t()
F22:0126         axis_transform_inv = torch.as_tensor(
F22:0127             self.camera_axis_transform_inv,

```

```

F22:0128         dtype=point_3d.dtype,
F22:0129         device=point_3d.device,
F22:0130     )
F22:0131     return point_3d @ axis_transform_inv.t()
F22:0133 def camera2world_ray_pytorch(self, point: torch.Tensor):
F22:0134     """
F22:0135         point: np.ndarray of 2D points on image (n * 2)
F22:0136         depth: np.ndarray of depth of every 2D points (n)
F22:0137         """
F22:0138     img_center = torch.as_tensor(
F22:0139         self.img_center, dtype=point.dtype, device=point.device
F22:0140     )
F22:0141     point_centered = point.float() - img_center
F22:0142     distorted_x = point_centered[:, 0]
F22:0143     distorted_y = point_centered[:, 1]
F22:0144     c = torch.as_tensor(self.c, dtype=point.dtype, device=point.device)
F22:0145     d = torch.as_tensor(self.d, dtype=point.dtype, device=point.device)
F22:0146     e = torch.as_tensor(self.e, dtype=point.dtype, device=point.device)
F22:0147     det = torch.as_tensor(self.affine_det, dtype=point.dtype, device=point.device)
F22:0148     x = (distorted_x - d * distorted_y) / det
F22:0149     y = (-e * distorted_x + c * distorted_y) / det
F22:0150     distance_from_center = torch.sqrt(torch.square(x) + torch.square(y))
F22:0152     z = self.getPolyVal(p=self.fisheye_polynomial1[::-1], x=distance_from_center)
F22:0153     point_3d = torch.stack([x, y, -z]).float() # 3, n
F22:0154     norm = torch.norm(point_3d, dim=0)
F22:0155     point_3d = point_3d / norm
F22:0156     point_3d = point_3d.t()
F22:0157     axis_transform_inv = torch.as_tensor(
F22:0158         self.camera_axis_transform_inv,
F22:0159         dtype=point_3d.dtype,
F22:0160         device=point_3d.device,
F22:0161     )
F22:0162     return point_3d @ axis_transform_inv.t()
F22:0164 def world2camera(self, point3D):
F22:0165     point3D = np.asarray(deepcopy(point3D))
F22:0166     point3D = point3D @ self.camera_axis_transform.T
F22:0167     point3D[:, 2] = point3D[:, 2] * -1
F22:0168     point3D = point3D.T
F22:0169     xc, yc = self.img_center[0], self.img_center[1]
F22:0170     point2D = []
F22:0172     norm = np.linalg.norm(point3D[:2], axis=0)
F22:0174     if (norm != 0).all():
F22:0175         theta = np.arctan(point3D[2] / norm)
F22:0176         invnorm = 1.0 / norm
F22:0177         t = theta
F22:0178         rho = self.fisheye_inverse_polynomial1[0]
F22:0179         t_i = 1.0
F22:0181         for i in range(1, len(self.fisheye_inverse_polynomial1)):
F22:0182             t_i *= t
F22:0183             rho += t_i * self.fisheye_inverse_polynomial1[i]

```

```

F22:0185         x = point3D[0] * invnorm * rho
F22:0186         y = point3D[1] * invnorm * rho
F22:0188         point2D.append(self.c * x + self.d * y + xc)
F22:0189         point2D.append(self.e * x + y + yc)
F22:0190     else:
F22:0191         point2D.append(xc)
F22:0192         point2D.append(yc)
F22:0193         raise Exception("norm is zero!")
F22:0195     return np.asarray(point2D).T
F22:0197 def world2camera_with_depth(self, point3D):
F22:0198     point3D_cloned = deepcopy(point3D)
F22:0199     point2D = self.world2camera(point3D_cloned)
F22:0201     depth = np.linalg.norm(point3D, axis=-1)
F22:0202     return point2D, depth
F22:0204 def world2camera_pytorch_with_depth(self, point3D):
F22:0205     point2D = self.world2camera_pytorch(point3D)
F22:0207     depth = torch.norm(point3D, dim=-1)
F22:0208     return point2D, depth
F22:0210 def world2camera_pytorch(self, point3d_original: torch.Tensor, normalize=False):
F22:0211     """
F22:0213     Args:
F22:0214         point3d_original: point
F22:0215         normalize: normalize to -1, 1
F22:0217     Returns:
F22:0219     """
F22:0220     fisheye_inv_polynomial = self.fisheye_inverse_polynomial
F22:0221     axis_transform = torch.as_tensor(
F22:0222         self.camera_axis_transform,
F22:0223         dtype=point3d_original.dtype,
F22:0224         device=point3d_original.device,
F22:0225     )
F22:0226     point3d = point3d_original.clone() @ axis_transform.t()
F22:0227     point3d[:, 2] = point3d[:, 2] * -1
F22:0228     point3d = point3d.transpose(0, 1)
F22:0229     xc, yc = self.img_center[0], self.img_center[1]
F22:0230     xc = torch.Tensor([xc]).float().to(point3d.device)
F22:0231     yc = torch.Tensor([yc]).float().to(point3d.device)
F22:0232     point2d = torch.empty((2, point3d.shape[-1])).to(point3d.device)
F22:0234     norm = torch.norm(point3d[:2], dim=0)
F22:0236     if (norm != 0).all():
F22:0237         theta = torch.atan(point3d[2] / norm)
F22:0238         invnorm = 1.0 / norm
F22:0239         t = theta
F22:0240         rho = fisheye_inv_polynomial[0]
F22:0241         t_i = 1.0
F22:0243         for i in range(1, len(fisheye_inv_polynomial)):
F22:0244             t_i *= t
F22:0245             rho += t_i * fisheye_inv_polynomial[i]
F22:0247         x = point3d[0] * invnorm * rho
F22:0248         y = point3d[1] * invnorm * rho

```



```

F22:0250         c = torch.as_tensor(self.c, dtype=x.dtype, device=x.device)
F22:0251         d = torch.as_tensor(self.d, dtype=x.dtype, device=x.device)
F22:0252         e = torch.as_tensor(self.e, dtype=x.dtype, device=x.device)
F22:0253         point2d[0] = c * x + d * y + xc
F22:0254         point2d[1] = e * x + y + yc
F22:0255     else:
F22:0256         point2d[0] = xc
F22:0257         point2d[1] = yc
F22:0258         raise Exception("norm is zero!")
F22:0260     # if normalize, the point result will be -1 to 1 from
F22:0261     if normalize is True:
F22:0262         image_w, image_h = self.img_size[0], self.img_size[1]
F22:0263         assert image_w > image_h
F22:0264         point2d[0] = point2d[0] - (image_w - image_h) // 2 # to square image
F22:0265         point2d = point2d / (image_h - 1) * 2 # to [0, 2]
F22:0266         point2d -= 1
F22:0268     return point2d.transpose(0, 1)
F22:0270     def undistort(self, point_2Ds):
F22:0271         """
F22:0272         undistort the input 2d points in fisheye camera
F22:0273         """
F22:0274         point_length = point_2Ds.shape[0]
F22:0275         depths = np.ones(shape=point_length)
F22:0276         point_3Ds = self.camera2world(point_2Ds, depths)
F22:0277         point_3Ds = point_3Ds @ self.camera_axis_transform.T
F22:0279         # point_3Ds_homo = np.ones((point_length, 4))
F22:0280         # point_3Ds_homo[:, :3] = point_3Ds
F22:0282         projected_2d_points = (self.intrinsic[:3, :3] @ point_3Ds.T).T
F22:0283         projected_2d_points = projected_2d_points[:, :2] / projected_2d_points[:, 2:]
F22:0284         return projected_2d_points
F23:0001 #!/usr/bin/env python
F23:0002 """Run person detection, EgoSMPLX inference, and fisheye projection on DownwardStereo extracted from
F23:0002+ es."""
F23:0004 import argparse
F23:0005 import json
F23:0006 import os
F23:0007 from pathlib import Path
F23:0008 import sys
F23:0009 import time
F23:0011 import cv2
F23:0012 import numpy as np
F23:0013 import torch
F23:0014 import torchvision.transforms as transforms
F23:0015 from tqdm import tqdm
F23:0016 from mmdet.apis import init_detector, inference_detector
F23:0018 REPO_ROOT = Path(os.environ["EGOSMPLX_REPO_ROOT"]).resolve()
F23:0019 from egosmplx_pipeline import camera as calibrated_camera
F23:0020 sys.modules["FishEyeCalibrated"] = calibrated_camera
F23:0021 sys.path.insert(0, str(REPO_ROOT))
F23:0022 sys.path.insert(0, str(REPO_ROOT / "main"))

```

```

F23:0023 sys.path.insert(0, str(REPO_ROOT / "data"))
F23:0024 sys.path.insert(0, str(REPO_ROOT / "common"))
F23:0026 from config import cfg
F23:0027 from utils.inference_utils import process_mmdet_results, non_max_suppression
F23:0030 def parse_args():
F23:0031     parser = argparse.ArgumentParser(description=__doc__)
F23:0032     parser.add_argument("--image-dir", type=Path, default=REPO_ROOT / "DownwardStereo" / "left_frame
F23:0032+ s_stridel0")
F23:0033     parser.add_argument("--image-glob", default="*.jpg")
F23:0034     parser.add_argument("--fisheye-calibration", type=Path, default=REPO_ROOT / "DownwardStereo" / "
F23:0034+ fisheye.left.calibration.json")
F23:0035     parser.add_argument(
F23:0036         "--config-path",
F23:0037         type=Path,
F23:0038         default=REPO_ROOT / "main" / "config" / "config_ft_egopw_sceneego_egowholebody_unrealego_ego
F23:0038+ fishbody.py",
F23:0039     )
F23:0040     parser.add_argument(
F23:0041         "--checkpoint-path",
F23:0042         type=Path,
F23:0043         default=REPO_ROOT / "output" / "train_5datasets_4gpu_resume_s7_to_sl2_20260801_191616" / "mo
F23:0043+ del_dump" / "snapshot_11.pth.tar",
F23:0044     )
F23:0045     parser.add_argument(
F23:0046         "--det-config",
F23:0047         type=Path,
F23:0048         default=REPO_ROOT / "pretrained_models" / "mmdet" / "mmdet_faster_rcnn_r50_fpn_coco.py",
F23:0049     )
F23:0050     parser.add_argument(
F23:0051         "--det-checkpoint",
F23:0052         type=Path,
F23:0053         default=REPO_ROOT / "pretrained_models" / "mmdet" / "faster_rcnn_r50_fpn_lx_coco_20200130-04
F23:0053+ 7c8118.pth",
F23:0054     )
F23:0055     parser.add_argument("--output-dir", type=Path, default=None)
F23:0056     parser.add_argument("--max-images", type=int, default=None)
F23:0057     parser.add_argument("--score-thr", type=float, default=0.3)
F23:0058     parser.add_argument("--bbox-thr", type=float, default=50.0)
F23:0059     parser.add_argument("--iou-thr", type=float, default=0.5)
F23:0060     parser.add_argument("--multi-person", action="store_true")
F23:0061     parser.add_argument("--no-detector", action="store_true", help="Run one full-image SMPL-X predic
F23:0061+ tion per image without Faster R-CNN.")
F23:0062     parser.add_argument("--save-mesh", action="store_true")
F23:0063     parser.add_argument(
F23:0064         "--save-wireframe-overlay",
F23:0065         action="store_true",
F23:0066         help="Also save a wireframe overlay from the same prediction and camera projection.",
F23:0067     )
F23:0068     parser.add_argument("--save-video", action="store_true")
F23:0069     parser.add_argument("--show-verts", action="store_true")

```

```

F23:0070     parser.add_argument("--show-bbox", action="store_true", default=True)
F23:0071     parser.add_argument("--mesh-mode", choices=("solid", "wireframe", "points"), default="solid")
F23:0072     parser.add_argument(
F23:0073         "--smp1x-input-mode",
F23:0074         choices=(
F23:0075             "full_image",
F23:0076             "det_crop",
F23:0077             "det_tangent",
F23:0078             "det_undistort_contiguous",
F23:0079         ),
F23:0080         default="full_image",
F23:0081         help=(
F23:0082             "det_tangent uses the detected person box to place the 16x12 ViT "
F23:0083             "tangent patches in original fisheye-camera coordinates; "
F23:0084             "det_undistort_contiguous first undistorts the full frame, detects "
F23:0085             "there, and samples one continuous 12-column x 16-row ViT crop."
F23:0086         ),
F23:0087     )
F23:0088     parser.add_argument(
F23:0089         "--det-tangent-expand",
F23:0090         type=float,
F23:0091         default=1.2,
F23:0092         help="Isotropic expansion applied to the detector box in det_tangent mode.",
F23:0093     )
F23:0094     parser.add_argument(
F23:0095         "--det-tangent-patch-size",
F23:0096         type=float,
F23:0097         default=0.1,
F23:0098         help="Half-size of each tangent-plane patch in det_tangent mode.",
F23:0099     )
F23:0100     parser.add_argument(
F23:0101         "--global-undistort-alpha",
F23:0102         type=float,
F23:0103         default=1.0,
F23:0104         help=(
F23:0105             "OpenCV optimal-new-camera alpha for det_undistort_contiguous: "
F23:0106             "0 crops invalid borders, 1 retains the full calibrated field."
F23:0107         ),
F23:0108     )
F23:0109     parser.add_argument(
F23:0110         "--contiguous-roi-mode",
F23:0111         choices=("stretch", "aspect_pad"),
F23:0112         default="stretch",
F23:0113         help=(
F23:0114             "For det_undistort_contiguous, stretch the clipped expanded box "
F23:0115             "to the 12:16 token canvas, or preserve 12:16 geometry with "
F23:0116             "out-of-image padding."
F23:0117         ),
F23:0118     )
F23:0119     parser.add_argument(

```

```

F23:0120         "--input-crop-mode",
F23:0121         choices=("none", "center_square", "training_horizontal"),
F23:0122         default="none",
F23:0123         help="Crop the network input without changing the original image used for fisheye mesh overl
F23:0123+ ay.",
F23:0124     )
F23:0125     parser.add_argument(
F23:0126         "--horizontal-crop-pixels",
F23:0127         type=int,
F23:0128         default=128,
F23:0129         help="Pixels removed from both sides with --input-crop-mode training_horizontal.",
F23:0130     )
F23:0131     parser.add_argument("--contact-sheet-limit", type=int, default=40)
F23:0132     parser.add_argument(
F23:0133         "--refine-camera-from-positionnet",
F23:0134         action="store_true",
F23:0135         help=(
F23:0136             "Keep the predicted SMPL-X local pose/shape fixed and refine a "
F23:0137             "camera extrinsic rotation/translation against PositionNet upper-body joints."
F23:0138         ),
F23:0139     )
F23:0140     parser.add_argument("--camera-fit-iters", type=int, default=300)
F23:0141     parser.add_argument("--camera-fit-lr", type=float, default=0.03)
F23:0142     parser.add_argument(
F23:0143         "--camera-fit-mode",
F23:0144         choices=("translation", "translation_global_orient", "extrinsics"),
F23:0145         default="translation",
F23:0146         help=(
F23:0147             "With --refine-camera-from-positionnet, optimize tx,ty,tz only; "
F23:0148             "SMPL-X global_orient plus translation; or the legacy camera-frame "
F23:0149             "rotation plus translation."
F23:0150         ),
F23:0151     )
F23:0152     parser.add_argument("--num-gpus", type=int, default=1)
F23:0153     return parser.parse_args()
F23:0154 def make_output_dir(args):
F23:0155     if args.output_dir is not None:
F23:0156         return args.output_dir
F23:0157     stamp = time.strftime("%Y%m%d_%H%M%S")
F23:0158     suffix = "full" if args.max_images is None else f"{args.max_images}imgs"
F23:0159     return REPO_ROOT / "output" / f"downwardstereo_left_stride10_detect_s11_{suffix}_{stamp}"
F23:0160 def setup_model(args, output_dir):
F23:0161     os.environ.setdefault("EGOSMPLX_REPO_ROOT", str(REPO_ROOT))
F23:0162     os.environ["EGOSMPLX_FISHEYE_CALIBRATION"] = str(args.fisheye_calibration.resolve())
F23:0163     cfg.get_config_fromfile(str(args.config_path.resolve()))
F23:0164     cfg.update_test_config(
F23:0165         testset="EHF",
F23:0166         agora_benchmark="na",
F23:0167         shapy_eval_split=None,
F23:0168         pretrained_model_path=str(args.checkpoint_path.resolve()),

```

```

F23:0173         use_cache=False,
F23:0174     )
F23:0175     cfg.fisheye_camera_path = str(args.fisheye_calibration.resolve())
F23:0176     cfg.fisheye_camera_paths = [str(args.fisheye_calibration.resolve())]
F23:0177     cfg.update_config(args.num_gpus, str(output_dir))
F23:0179     from base import Demoer
F23:0181     demoer = Demoer()
F23:0182     demoer._make_model()
F23:0183     demoer.model.eval()
F23:0184     return demoer
F23:0187 def load_runtime_utils():
F23:0188     from FishEyeCalibrated import FishEyeCameraCalibrated
F23:0189     from utils.human_models import smpl_x
F23:0190     from utils.preprocessing import load_img, process_bbox, generate_patch_image
F23:0191     from utils.vis import render_mesh_fisheye, save_obj
F23:0193     return FishEyeCameraCalibrated, smpl_x, load_img, process_bbox, generate_patch_image, render_mes
F23:0193+ h_fisheye, save_obj
F23:0196 def filter_bboxes(raw_bboxes, score_thr, bbox_thr):
F23:0197     kept = []
F23:0198     for bbox in raw_bboxes:
F23:0199         x1, y1, x2, y2, score = [float(v) for v in bbox[:5]]
F23:0200         w = abs(x2 - x1)
F23:0201         h = abs(y2 - y1)
F23:0202         if score < score_thr:
F23:0203             continue
F23:0204         if w < bbox_thr or h < bbox_thr:
F23:0205             continue
F23:0206         kept.append(np.array([x1, y1, x2, y2, score], dtype=np.float32))
F23:0207     return kept
F23:0210 def save_contact_sheet(image_paths, out_path, max_images=40, thumb_w=360):
F23:0211     paths = image_paths[:max_images]
F23:0212     if not paths:
F23:0213         return
F23:0214     thumbs = []
F23:0215     for path in paths:
F23:0216         img = cv2.imread(str(path), cv2.IMREAD_COLOR)
F23:0217         if img is None:
F23:0218             continue
F23:0219         h, w = img.shape[:2]
F23:0220         thumb_h = max(1, int(h * thumb_w / max(1, w)))
F23:0221         thumbs.append(cv2.resize(img, (thumb_w, thumb_h), interpolation=cv2.INTER_AREA))
F23:0222     if not thumbs:
F23:0223         return
F23:0224     cols = min(4, len(thumbs))
F23:0225     rows = int(np.ceil(len(thumbs) / cols))
F23:0226     max_h = max(t.shape[0] for t in thumbs)
F23:0227     canvas = np.full((rows * max_h, cols * thumb_w, 3), 255, dtype=np.uint8)
F23:0228     for i, thumb in enumerate(thumbs):
F23:0229         r, c = divmod(i, cols)
F23:0230         canvas[r * max_h : r * max_h + thumb.shape[0], c * thumb_w : c * thumb_w + thumb.shape[1]] =

```

```

F23:0230+ thumb
F23:0231     cv2.imwrite(str(out_path), canvas)
F23:0234 def draw_no_detection(img_rgb, text="NO PERSON DETECTED"):
F23:0235     img = img_rgb.copy()
F23:0236     cv2.putText(img, text, (40, 70), cv2.FONT_HERSHEY_SIMPLEX, 1.4, (255, 40, 40), 3, cv2.LINE_AA)
F23:0237     return img
F23:0240 def rgb_to_bgr_u8(img_rgb):
F23:0241     img = np.clip(img_rgb, 0, 255).astype(np.uint8)
F23:0242     return img[:, :, ::-1]
F23:0245 def expand_bbox_xyxy(box_xyxy, scale, image_w, image_h):
F23:0246     """Expand a detector box about its center and clip it to the source image."""
F23:0247     x1, y1, x2, y2 = [float(value) for value in box_xyxy[:4]]
F23:0248     center_x = (x1 + x2) * 0.5
F23:0249     center_y = (y1 + y2) * 0.5
F23:0250     width = max(2.0, abs(x2 - x1) * float(scale))
F23:0251     height = max(2.0, abs(y2 - y1) * float(scale))
F23:0252     x1 = np.clip(center_x - width * 0.5, 0.0, float(image_w - 1))
F23:0253     x2 = np.clip(center_x + width * 0.5, 0.0, float(image_w - 1))
F23:0254     y1 = np.clip(center_y - height * 0.5, 0.0, float(image_h - 1))
F23:0255     y2 = np.clip(center_y + height * 0.5, 0.0, float(image_h - 1))
F23:0256     if x2 - x1 < 2.0 or y2 - y1 < 2.0:
F23:0257         raise ValueError(f"Expanded detector box is degenerate: {(x1, y1, x2, y2)}")
F23:0258     return np.asarray([x1, y1, x2, y2], dtype=np.float32)
F23:0261 def create_global_undistort_model(calibration_path, image_w, image_h, alpha):
F23:0262     """Build one continuous Brown-Conrady rectification for the full frame."""
F23:0263     calibration = json.loads(Path(calibration_path).read_text())
F23:0264     required = (
F23:0265         "sourceFocalLength",
F23:0266         "sourcePrincipalPoint",
F23:0267         "sourceRadialDistortion",
F23:0268     )
F23:0269     missing = [key for key in required if key not in calibration]
F23:0270     if missing:
F23:0271         raise KeyError(
F23:0272             "Continuous global undistortion requires Brown-Conrady source "
F23:0273             f"fields in {calibration_path}; missing {missing}"
F23:0274         )
F23:0275     calibration_size = tuple(int(value) for value in calibration.get("size", (image_w, image_h)))
F23:0276     if calibration_size != (int(image_w), int(image_h)):
F23:0277         raise ValueError(
F23:0278             "Calibration/image size mismatch for global undistortion: "
F23:0279             f"calibration={calibration_size}, image={(image_w, image_h)}"
F23:0280         )
F23:0281     focal = np.asarray(calibration["sourceFocalLength"], dtype=np.float64)
F23:0282     principal = np.asarray(calibration["sourcePrincipalPoint"], dtype=np.float64)
F23:0283     radial = list(calibration["sourceRadialDistortion"])
F23:0284     tangential = list(calibration.get("sourceTangentialDistortion", (0.0, 0.0)))
F23:0285     while len(radial) < 3:
F23:0286         radial.append(0.0)
F23:0287     source_k = np.asarray(

```

```

F23:0288         [
F23:0289             [focal[0], 0.0, principal[0]],
F23:0290             [0.0, focal[1], principal[1]],
F23:0291             [0.0, 0.0, 1.0],
F23:0292         ],
F23:0293         dtype=np.float64,
F23:0294     )
F23:0295     distortion = np.asarray(
F23:0296         [radial[0], radial[1], tangential[0], tangential[1], radial[2]],
F23:0297         dtype=np.float64,
F23:0298     )
F23:0299     new_k, valid_roi = cv2.getOptimalNewCameraMatrix(
F23:0300         source_k,
F23:0301         distortion,
F23:0302         (int(image_w), int(image_h)),
F23:0303         float(alpha),
F23:0304         (int(image_w), int(image_h)),
F23:0305         centerPrincipalPoint=False,
F23:0306     )
F23:0307     map_x, map_y = cv2.initUndistortRectifyMap(
F23:0308         source_k,
F23:0309         distortion,
F23:0310         None,
F23:0311         new_k,
F23:0312         (int(image_w), int(image_h)),
F23:0313         cv2.CV_32FC1,
F23:0314     )
F23:0315     return {
F23:0316         "source_k": source_k,
F23:0317         "distortion": distortion,
F23:0318         "new_k": new_k,
F23:0319         "valid_roi": tuple(int(value) for value in valid_roi),
F23:0320         "map_x": map_x,
F23:0321         "map_y": map_y,
F23:0322         "alpha": float(alpha),
F23:0323     }
F23:0326 def undistort_full_image(image_rgb, undistort_model):
F23:0327     """Remap a complete fisheye frame into one continuous pinhole image."""
F23:0328     return cv2.remap(
F23:0329         image_rgb,
F23:0330         undistort_model["map_x"],
F23:0331         undistort_model["map_y"],
F23:0332         interpolation=cv2.INTER_LINEAR,
F23:0333         borderMode=cv2.BORDER_CONSTANT,
F23:0334         borderValue=(0, 0, 0),
F23:0335     )
F23:0338 def undistorted_pixels_to_original(points_xy, undistort_model):
F23:0339     """Map pixels from the continuous undistorted view to the raw frame."""
F23:0340     points = np.asarray(points_xy, dtype=np.float64).reshape(-1, 2)
F23:0341     new_k = undistort_model["new_k"]

```

```

F23:0342     normalized = np.column_stack(
F23:0343         (
F23:0344             (points[:, 0] - new_k[0, 2]) / new_k[0, 0],
F23:0345             (points[:, 1] - new_k[1, 2]) / new_k[1, 1],
F23:0346             np.ones((len(points),), dtype=np.float64),
F23:0347         )
F23:0348     )
F23:0349     projected, _ = cv2.projectPoints(
F23:0350         normalized,
F23:0351         np.zeros((3, 1), dtype=np.float64),
F23:0352         np.zeros((3, 1), dtype=np.float64),
F23:0353         undistort_model["source_k"],
F23:0354         undistort_model["distortion"],
F23:0355     )
F23:0356     return projected.reshape(-1, 2).astype(np.float32)
F23:0359 def expand_bbox_to_aspect(box_xyxy, scale, target_width_over_height):
F23:0360     """Expand a box and pad it to one aspect ratio without clipping."""
F23:0361     x1, y1, x2, y2 = [float(value) for value in box_xyxy[:4]]
F23:0362     center_x = (x1 + x2) * 0.5
F23:0363     center_y = (y1 + y2) * 0.5
F23:0364     width = max(2.0, abs(x2 - x1) * float(scale))
F23:0365     height = max(2.0, abs(y2 - y1) * float(scale))
F23:0366     if width / height > float(target_width_over_height):
F23:0367         height = width / float(target_width_over_height)
F23:0368     else:
F23:0369         width = height * float(target_width_over_height)
F23:0370     return np.asarray(
F23:0371         [
F23:0372             center_x - width * 0.5,
F23:0373             center_y - height * 0.5,
F23:0374             center_x + width * 0.5,
F23:0375             center_y + height * 0.5,
F23:0376         ],
F23:0377         dtype=np.float32,
F23:0378     )
F23:0381 def create_contiguous_bbox_pixel_grid(
F23:0382     bbox_xyxy,
F23:0383     patch_rows,
F23:0384     patch_columns,
F23:0385     patch_h,
F23:0386     patch_w,
F23:0387 ):
F23:0388     """Split one continuous rectified ROI into row-major ViT patch grids."""
F23:0389     x1, y1, x2, y2 = [float(value) for value in bbox_xyxy]
F23:0390     output_h = int(patch_rows) * int(patch_h)
F23:0391     output_w = int(patch_columns) * int(patch_w)
F23:0392     xs = np.linspace(x1, x2, output_w, dtype=np.float32)
F23:0393     ys = np.linspace(y1, y2, output_h, dtype=np.float32)
F23:0394     grid_x, grid_y = np.meshgrid(xs, ys)
F23:0395     full_grid = np.stack((grid_x, grid_y), axis=-1)

```



```

F23:0396     return (
F23:0397         full_grid.reshape(
F23:0398             patch_rows,
F23:0399             patch_h,
F23:0400             patch_columns,
F23:0401             patch_w,
F23:0402             2,
F23:0403         )
F23:0404         .transpose(0, 2, 1, 3, 4)
F23:0405         .reshape(patch_rows * patch_columns, patch_h, patch_w, 2)
F23:0406         .astype(np.float32)
F23:0407     )
F23:0410 def draw_contiguous_sampling_grid(image_rgb, grid_pixels, bbox_xyxy):
F23:0411     """Draw the regular 12-column x 16-row crop on the undistorted image."""
F23:0412     canvas = image_rgb.copy()
F23:0413     for patch in grid_pixels:
F23:0414         corners = np.asarray(
F23:0415             [patch[0, 0], patch[0, -1], patch[-1, -1], patch[-1, 0]],
F23:0416             dtype=np.float32,
F23:0417         )
F23:0418         cv2.polylines(
F23:0419             canvas,
F23:0420             [np rint(corners).astype(np.int32)],
F23:0421             isClosed=True,
F23:0422             color=(255, 0, 0),
F23:0423             thickness=1,
F23:0424             lineType=cv2.LINE_AA,
F23:0425         )
F23:0426         x1, y1, x2, y2 = np rint(bbox_xyxy).astype(int)
F23:0427         cv2.rectangle(canvas, (x1, y1), (x2, y2), (255, 0, 0), 3)
F23:0428         cv2.putText(
F23:0429             canvas,
F23:0430             "continuous undistorted ROI -> 12 cols x 16 rows",
F23:0431             (max(10, x1), max(30, y1 - 12)),
F23:0432             cv2.FONT_HERSHEY_SIMPLEX,
F23:0433             0.68,
F23:0434             (255, 0, 0),
F23:0435             2,
F23:0436             cv2.LINE_AA,
F23:0437         )
F23:0438     return canvas
F23:0441 def create_bbox_tangent_pixel_grid(
F23:0442     sampler,
F23:0443     fisheye_camera,
F23:0444     bbox_xyxy,
F23:0445     patch_rows=16,
F23:0446     patch_columns=12,
F23:0447     patch_size=0.1,
F23:0448 ):
F23:0449     """Create row-major tangent-patch samples in original fisheye pixels."""

```

```

F23:0450     x1, y1, x2, y2 = [float(value) for value in bbox_xyxy]
F23:0451     horizontal = (np.arange(patch_columns, dtype=np.float32) + 0.5) / patch_columns
F23:0452     vertical = (np.arange(patch_rows, dtype=np.float32) + 0.5) / patch_rows
F23:0453     normalized_centers = np.dstack(np.meshgrid(horizontal, vertical)).reshape(-1, 2)
F23:0454     centers_2d = normalized_centers.copy()
F23:0455     centers_2d[:, 0] = x1 + centers_2d[:, 0] * (x2 - x1)
F23:0456     centers_2d[:, 1] = y1 + centers_2d[:, 1] * (y2 - y1)
F23:0457     centers_2d_up = centers_2d.copy()
F23:0458     centers_2d_up[:, 1] -= (y2 - y1) / (2.0 * patch_rows)
F23:0460     depth = np.ones((len(centers_2d),), dtype=np.float32)
F23:0461     centers_3d = fisheye_camera.camera2world(centers_2d, depth)
F23:0462     centers_3d_up = fisheye_camera.camera2world(centers_2d_up, depth)
F23:0463     patch_h, patch_w = [
F23:0464         int(value) for value in sampler.patches_2d_per_camera_pixels.shape[-3:-1]
F23:0465     ]
F23:0466     tangent_patches = [
F23:0467         sampler._generate_patch_coordinates(
F23:0468             center,
F23:0469             center_up,
F23:0470             (float(patch_size), float(patch_size)),
F23:0471             (patch_h, patch_w),
F23:0472             visualize=False,
F23:0473         )
F23:0474         for center, center_up in zip(centers_3d, centers_3d_up)
F23:0475     ]
F23:0476     grid = sampler._project_patches_to_original_image(
F23:0477         tangent_patches,
F23:0478         fisheye_camera=fisheye_camera,
F23:0479         image_h=int(fisheye_camera.img_size[1]),
F23:0480         image_w=int(fisheye_camera.img_size[0]),
F23:0481         normalize=False,
F23:0482     )
F23:0483     return np.asarray(grid, dtype=np.float32)
F23:0486 def draw_tangent_sampling_grid(image_rgb, grid_pixels, bbox_xyxy):
F23:0487     """Draw every actual tangent-patch footprint and its detected ROI."""
F23:0488     canvas = image_rgb.copy()
F23:0489     for patch in grid_pixels:
F23:0490         corners = np.asarray(
F23:0491             [patch[0, 0], patch[0, -1], patch[-1, -1], patch[-1, 0]],
F23:0492             dtype=np.float32,
F23:0493         )
F23:0494         if np.isfinite(corners).all():
F23:0495             cv2.polylines(
F23:0496                 canvas,
F23:0497                 [np rint(corners).astype(np.int32)],
F23:0498                 True,
F23:0499                 (0, 255, 0),
F23:0500                 1,
F23:0501                 cv2.LINE_AA,
F23:0502             )

```

```

F23:0503     x1, y1, x2, y2 = np.rint(bbox_xyxy).astype(int)
F23:0504     cv2.rectangle(canvas, (x1, y1), (x2, y2), (255, 0, 0), 3)
F23:0505     cv2.putText(
F23:0506         canvas,
F23:0507         "det ROI -> 16x12 tangent patches",
F23:0508         (max(10, x1), max(30, y1 - 12)),
F23:0509         cv2.FONT_HERSHEY_SIMPLEX,
F23:0510         0.75,
F23:0511         (255, 0, 0),
F23:0512         2,
F23:0513         cv2.LINE_AA,
F23:0514     )
F23:0515     return canvas
F23:0518 def save_vit_patch_atlas(
F23:0519     sampled_patches,
F23:0520     out_path,
F23:0521     rows=16,
F23:0522     columns=12,
F23:0523     scale=4,
F23:0524     gap=2,
F23:0525     grid_color_rgb=None,
F23:0526 ):
F23:0527     """Save the exact RGB patches returned by UndistortPatch before PatchEmbed."""
F23:0528     patches = sampled_patches.detach().cpu().numpy()[0]
F23:0529     if patches.shape[0] != rows * columns:
F23:0530         raise ValueError(
F23:0531             f"Expected {rows * columns} ViT patches, got {patches.shape[0]}"
F23:0532         )
F23:0533     patches = np.clip(patches.transpose(0, 2, 3, 1) * 255.0, 0, 255).astype(np.uint8)
F23:0534     patch_h, patch_w = patches.shape[1:3]
F23:0535     atlas_h = rows * patch_h + (rows - 1) * gap
F23:0536     atlas_w = columns * patch_w + (columns - 1) * gap
F23:0537     atlas = np.zeros((atlas_h, atlas_w, 3), dtype=np.uint8)
F23:0538     for patch_index, patch in enumerate(patches):
F23:0539         row, column = divmod(patch_index, columns)
F23:0540         top = row * (patch_h + gap)
F23:0541         left = column * (patch_w + gap)
F23:0542         atlas[top:top + patch_h, left:left + patch_w] = patch
F23:0543     atlas = cv2.resize(
F23:0544         atlas,
F23:0545         (atlas.shape[1] * scale, atlas.shape[0] * scale),
F23:0546         interpolation=cv2.INTER_NEAREST,
F23:0547     )
F23:0548     if grid_color_rgb is not None:
F23:0549         patch_h_scaled = patch_h * scale
F23:0550         patch_w_scaled = patch_w * scale
F23:0551         for row in range(1, rows):
F23:0552             y = row * patch_h_scaled
F23:0553             cv2.line(
F23:0554                 atlas,

```

```

F23:0555             (0, y),
F23:0556             (atlas.shape[1] - 1, y),
F23:0557             grid_color_rgb,
F23:0558             2,
F23:0559             cv2.LINE_AA,
F23:0560         )
F23:0561         for column in range(1, columns):
F23:0562             x = column * patch_w_scaled
F23:0563             cv2.line(
F23:0564                 atlas,
F23:0565                 (x, 0),
F23:0566                 (x, atlas.shape[0] - 1),
F23:0567                 grid_color_rgb,
F23:0568                 2,
F23:0569                 cv2.LINE_AA,
F23:0570             )
F23:0571         cv2.imwrite(str(out_path), rgb_to_bgr_u8(atlas))
F23:0574 POSITIONNET_BODY_EDGES = (
F23:0575     (0, 1), (1, 3), (3, 5),
F23:0576     (0, 2), (2, 4), (4, 6),
F23:0577     (0, 7),
F23:0578     (7, 8), (8, 10), (10, 12),
F23:0579     (7, 9), (9, 11), (11, 13),
F23:0580     (5, 14), (5, 15), (5, 16),
F23:0581     (6, 17), (6, 18), (6, 19),
F23:0582     (7, 24), (24, 22), (22, 20), (24, 23), (23, 21),
F23:0583 )
F23:0586 def positionnet_coords_to_sampling_grid(
F23:0587     body_joint_img,
F23:0588     sampling_grid_pixels,
F23:0589     patch_rows=16,
F23:0590     patch_columns=12,
F23:0591 ):
F23:0592     """Map PositionNet coordinates through the actual tangent-patch centers."""
F23:0593     coords = np.asarray(body_joint_img, dtype=np.float32)
F23:0594     grid = np.asarray(sampling_grid_pixels, dtype=np.float32)
F23:0595     if grid.shape[0] != patch_rows * patch_columns:
F23:0596         raise ValueError(
F23:0597             f"Expected {patch_rows * patch_columns} sampling patches, got {grid.shape[0]}"
F23:0598         )
F23:0599     patch_h, patch_w = grid.shape[1:3]
F23:0600     center_rows = slice((patch_h - 1) // 2, patch_h // 2 + 1)
F23:0601     center_columns = slice((patch_w - 1) // 2, patch_w // 2 + 1)
F23:0602     patch_centers = grid.reshape(
F23:0603         patch_rows, patch_columns, patch_h, patch_w, 2
F23:0604     )[:, :, center_rows, center_columns].mean(axis=(2, 3))
F23:0606     x = np.clip(coords[:, 0], 0.0, float(patch_columns - 1))
F23:0607     y = np.clip(coords[:, 1], 0.0, float(patch_rows - 1))
F23:0608     x0 = np.floor(x).astype(np.int64)
F23:0609     y0 = np.floor(y).astype(np.int64)

```

```

F23:0610     x1 = np.minimum(x0 + 1, patch_columns - 1)
F23:0611     y1 = np.minimum(y0 + 1, patch_rows - 1)
F23:0612     wx = (x - x0).reshape(-1, 1)
F23:0613     wy = (y - y0).reshape(-1, 1)
F23:0614     pixels = (
F23:0615         (1.0 - wx) * (1.0 - wy) * patch_centers[y0, x0]
F23:0616         + wx * (1.0 - wy) * patch_centers[y0, x1]
F23:0617         + (1.0 - wx) * wy * patch_centers[y1, x0]
F23:0618         + wx * wy * patch_centers[y1, x1]
F23:0619     )
F23:0620     return pixels.astype(np.float32), patch_centers.astype(np.float32)
F23:0623 def draw_positionnet_result(image_rgb, body_joint_hm, body_joint_pixels, sampling_bbox_xyxy):
F23:0624     """Overlay the depth-marginalized PositionNet heatmap and body skeleton."""
F23:0625     canvas = image_rgb.copy()
F23:0626     heatmap_2d = np.asarray(body_joint_hm, dtype=np.float32).sum(axis=(0, 1))
F23:0627     heatmap_2d -= heatmap_2d.min()
F23:0628     peak = float(heatmap_2d.max())
F23:0629     if peak > 0:
F23:0630         heatmap_2d /= peak
F23:0631     heatmap_color_bgr = cv2.applyColorMap(
F23:0632         np rint(heatmap_2d * 255.0).astype(np.uint8),
F23:0633         cv2.COLORMAP_JET,
F23:0634     )
F23:0635     heatmap_color = heatmap_color_bgr[:, :, ::-1]
F23:0637     image_h, image_w = canvas.shape[:2]
F23:0638     x1, y1, x2, y2 = [int(round(value)) for value in sampling_bbox_xyxy]
F23:0639     x1, x2 = np.clip([x1, x2], 0, image_w)
F23:0640     y1, y2 = np.clip([y1, y2], 0, image_h)
F23:0641     if x2 > x1 and y2 > y1:
F23:0642         heatmap_roi = cv2.resize(
F23:0643             heatmap_color,
F23:0644             (x2 - x1, y2 - y1),
F23:0645             interpolation=cv2.INTER_CUBIC,
F23:0646         ).astype(canvas.dtype, copy=False)
F23:0647         canvas[y1:y2, x1:x2] = cv2.addWeighted(
F23:0648             canvas[y1:y2, x1:x2],
F23:0649             0.68,
F23:0650             heatmap_roi,
F23:0651             0.32,
F23:0652             0.0,
F23:0653         )
F23:0655     for start, end in POSITIONNET_BODY_EDGES:
F23:0656         points = body_joint_pixels[[start, end]]
F23:0657         if np.isfinite(points).all():
F23:0658             cv2.line(
F23:0659                 canvas,
F23:0660                 tuple(np rint(points[0]).astype(int)),
F23:0661                 tuple(np rint(points[1]).astype(int)),
F23:0662                 (255, 255, 0),
F23:0663                 4,

```

```

F23:0664             cv2.LINE_AA,
F23:0665         )
F23:0666     for point in body_joint_pixels:
F23:0667         if np.isfinite(point).all():
F23:0668             cv2.circle(
F23:0669                 canvas,
F23:0670                 tuple(np rint(point).astype(int)),
F23:0671                 6,
F23:0672                 (255, 0, 0),
F23:0673                 -1,
F23:0674                 cv2.LINE_AA,
F23:0675             )
F23:0676     cv2.putText(
F23:0677         canvas,
F23:0678         "PositionNet: 25 body joints (yellow) + heatmap",
F23:0679         (24, 42),
F23:0680         cv2.FONT_HERSHEY_SIMPLEX,
F23:0681         0.8,
F23:0682         (255, 255, 0),
F23:0683         2,
F23:0684         cv2.LINE_AA,
F23:0685     )
F23:0686     return canvas
F23:0689 def draw_positionnet_smplx_2d_comparison(
F23:0690     image_rgb,
F23:0691     positionnet_pixels,
F23:0692     smplx_pixels,
F23:0693 ):
F23:0694     """Draw all 25 corresponding body joints from PositionNet and SMPL-X."""
F23:0695     canvas = image_rgb.copy()
F23:0696     positionnet_pixels = np.asarray(positionnet_pixels, dtype=np.float32)
F23:0697     smplx_pixels = np.asarray(smplx_pixels, dtype=np.float32)
F23:0698     if positionnet_pixels.shape != (25, 2) or smplx_pixels.shape != (25, 2):
F23:0699         raise ValueError(
F23:0700             "PositionNet and SMPL-X body points must both have shape (25, 2)"
F23:0701         )
F23:0703     valid = np.isfinite(positionnet_pixels).all(axis=1) & np.isfinite(smplx_pixels).all(axis=1)
F23:0704     valid &= (np.abs(positionnet_pixels) < 1e6).all(axis=1)
F23:0705     valid &= (np.abs(smplx_pixels) < 1e6).all(axis=1)
F23:0706     errors = np.linalg.norm(smplx_pixels - positionnet_pixels, axis=1)
F23:0708     # Draw per-joint correspondence first so the two skeletons remain visible.
F23:0709     for joint_index in np.flatnonzero(valid):
F23:0710         positionnet_point = tuple(np rint(positionnet_pixels[joint_index]).astype(int))
F23:0711         smplx_point = tuple(np rint(smplx_pixels[joint_index]).astype(int))
F23:0712         cv2.line(
F23:0713             canvas,
F23:0714             positionnet_point,
F23:0715             smplx_point,
F23:0716             (0, 255, 255),
F23:0717             1,

```

```

F23:0718         cv2.LINE_AA,
F23:0719     )
F23:0721     for start, end in POSITIONNET_BODY_EDGES:
F23:0722         if valid[start] and valid[end]:
F23:0723             cv2.line(
F23:0724                 canvas,
F23:0725                 tuple(np rint(positionnet_pixels[start]).astype(int)),
F23:0726                 tuple(np rint(positionnet_pixels[end]).astype(int)),
F23:0727                 (255, 0, 0),
F23:0728                 3,
F23:0729                 cv2.LINE_AA,
F23:0730             )
F23:0731             cv2.line(
F23:0732                 canvas,
F23:0733                 tuple(np rint(smplx_pixels[start]).astype(int)),
F23:0734                 tuple(np rint(smplx_pixels[end]).astype(int)),
F23:0735                 (0, 255, 0),
F23:0736                 3,
F23:0737                 cv2.LINE_AA,
F23:0738             )
F23:0740     for joint_index in np.flatnonzero(valid):
F23:0741         positionnet_point = tuple(np rint(positionnet_pixels[joint_index]).astype(int))
F23:0742         smplx_point = tuple(np rint(smplx_pixels[joint_index]).astype(int))
F23:0743         cv2.circle(canvas, positionnet_point, 6, (255, 0, 0), -1, cv2.LINE_AA)
F23:0744         cv2.circle(canvas, smplx_point, 7, (0, 255, 0), 2, cv2.LINE_AA)
F23:0745         midpoint = tuple(
F23:0746             np rint((positionnet_pixels[joint_index] + smplx_pixels[joint_index]) * 0.5)
F23:0747             .astype(int)
F23:0748         )
F23:0749         cv2.putText(
F23:0750             canvas,
F23:0751             str(joint_index),
F23:0752             midpoint,
F23:0753             cv2.FONT_HERSHEY_SIMPLEX,
F23:0754             0.42,
F23:0755             (255, 255, 255),
F23:0756             1,
F23:0757             cv2.LINE_AA,
F23:0758         )
F23:0760     selected_valid = valid[CAMERA_FIT_BODY_INDICES]
F23:0761     selected_errors = errors[CAMERA_FIT_BODY_INDICES][selected_valid]
F23:0762     all_mean_error = float(errors[valid].mean()) if valid.any() else float("nan")
F23:0763     fit_mean_error = (
F23:0764         float(selected_errors.mean()) if len(selected_errors) else float("nan")
F23:0765     )
F23:0766     labels = (
F23:0767         ((255, 0, 0), "red: PositionNet 2D (filled)"),
F23:0768         ((0, 255, 0), "green: SMPL-X projected 2D (ring)"),
F23:0769         ((0, 255, 255), "cyan: corresponding joint"),
F23:0770         ((255, 255, 255), f"mean all125={all_mean_error:.1f}px, fit12={fit_mean_error:.1f}px"),

```

```

F23:0771     )
F23:0772     for row, (color, label) in enumerate(labels):
F23:0773         cv2.putText(
F23:0774             canvas,
F23:0775             label,
F23:0776             (24, 38 + row * 31),
F23:0777             cv2.FONT_HERSHEY_SIMPLEX,
F23:0778             0.68,
F23:0779             color,
F23:0780             2,
F23:0781             cv2.LINE_AA,
F23:0782         )
F23:0783     return canvas
F23:0786 CAMERA_FIT_BODY_INDICES = np.asarray(
F23:0787     [7, 8, 9, 10, 11, 12, 13, 20, 21, 22, 23, 24], dtype=np.int64
F23:0788 )
F23:0789 CAMERA_FIT_BODY_EDGES = (
F23:0790     (7, 8), (7, 9), (8, 10), (10, 12), (9, 11), (11, 13),
F23:0791     (7, 20), (7, 21), (20, 22), (21, 23), (22, 24), (23, 24),
F23:0792 )
F23:0795 def heatmap_joints_to_original_pixels(joint_img, input_hw, input_offset):
F23:0796     """Invert the normalization used by Model.get_coord for full-image input."""
F23:0797     input_h, input_w = [float(v) for v in input_hw]
F23:0798     offset_x, offset_y = [float(v) for v in input_offset]
F23:0799     joint_px = np.asarray(joint_img, dtype=np.float64).copy()
F23:0800     joint_px[:, 0] = joint_px[:, 0] / float(cfg.output_hm_shape[2]) * input_w + offset_x
F23:0801     joint_px[:, 1] = joint_px[:, 1] / float(cfg.output_hm_shape[1]) * input_h + offset_y
F23:0802     return joint_px
F23:0805 def axis_angle_to_matrix(axis_angle):
F23:0806     """Differentiable Rodrigues transform for a single three-vector."""
F23:0807     theta = torch.linalg.vector_norm(axis_angle).clamp_min(1e-8)
F23:0808     axis = axis_angle / theta
F23:0809     x, y, z = axis.unbind()
F23:0810     zero = torch.zeros((), device=axis.device, dtype=axis.dtype)
F23:0811     skew = torch.stack(
F23:0812         (zero, -z, y, z, zero, -x, -y, x, zero)
F23:0813     ).reshape(3, 3)
F23:0814     identity = torch.eye(3, device=axis.device, dtype=axis.dtype)
F23:0815     return identity + torch.sin(theta) * skew + (1.0 - torch.cos(theta)) * (skew @ skew)
F23:0818 def compose_global_orient(delta_rotation, network_global_orient):
F23:0819     """Left-compose a camera-frame delta into the SMPL-X root axis-angle."""
F23:0820     delta_np = delta_rotation.detach().cpu().numpy().astype(np.float64)
F23:0821     network_np = (
F23:0822         network_global_orient.detach().cpu().numpy().reshape(3).astype(np.float64)
F23:0823     )
F23:0824     network_rotation, _ = cv2.Rodrigues(network_np)
F23:0825     refined_rotation = delta_np @ network_rotation
F23:0826     refined_axis_angle, _ = cv2.Rodrigues(refined_rotation)
F23:0827     return torch.as_tensor(
F23:0828         refined_axis_angle.reshape(3),

```



```

F23:0829         device=network_global_orient.device,
F23:0830         dtype=network_global_orient.dtype,
F23:0831     )
F23:0834 def refine_camera_extrinsics(
F23:0835     body_joints_cam,
F23:0836     body_targets_px,
F23:0837     initial_translation,
F23:0838     fisheye_cam,
F23:0839     iterations,
F23:0840     learning_rate,
F23:0841 ):
F23:0842     """Fit a camera-frame delta R,t while local pose, shape, and expression stay fixed."""
F23:0843     if iterations <= 0:
F23:0844         raise ValueError("--camera-fit-iters must be positive")
F23:0845     if learning_rate <= 0:
F23:0846         raise ValueError("--camera-fit-lr must be positive")
F23:0848     device = body_joints_cam.device
F23:0849     dtype = body_joints_cam.dtype
F23:0850     indices = torch.as_tensor(CAMERA_FIT_BODY_INDICES, device=device, dtype=torch.long)
F23:0851     joints = body_joints_cam.index_select(0, indices).detach()
F23:0852     targets = torch.as_tensor(
F23:0853         body_targets_px[CAMERA_FIT_BODY_INDICES], device=device, dtype=dtype
F23:0854     )
F23:0855     initial = initial_translation.detach().to(device=device, dtype=dtype)
F23:0857     identity = torch.eye(3, device=device, dtype=dtype)
F23:0859     def project(rotation, translation):
F23:0860         transformed = joints @ rotation.t() + translation[None, :]
F23:0861         return fisheye_cam.world2camera_pytorch(transformed)
F23:0863     with torch.no_grad():
F23:0864         initial_projection = project(identity, initial)
F23:0865         initial_error = torch.linalg.vector_norm(
F23:0866             initial_projection - targets, dim=1
F23:0867         ).mean()
F23:0869     initial_z = max(0.1, float(initial[2].item()))
F23:0870     depth_starts = sorted(set([initial_z, 0.5, 0.75, 1.0, 1.5, 2.0]))
F23:0871     best = None
F23:0872     for depth_start in depth_starts:
F23:0873         parameter = torch.tensor(
F23:0874             [
F23:0875                 0.0,
F23:0876                 0.0,
F23:0877                 0.0,
F23:0878                 float(initial[0]),
F23:0879                 float(initial[1]),
F23:0880                 float(np.log(depth_start)),
F23:0881             ],
F23:0882             device=device,
F23:0883             dtype=dtype,
F23:0884             requires_grad=True,
F23:0885         )

```

```

F23:0886         optimizer = torch.optim.Adam([parameter], lr=learning_rate)
F23:0887         for _ in range(iterations):
F23:0888             rotation = axis_angle_to_matrix(parameter[:3])
F23:0889             translation = torch.stack(
F23:0890                 (parameter[3], parameter[4], torch.exp(parameter[5]))
F23:0891             )
F23:0892             projection = project(rotation, translation)
F23:0893             # Charbonnier-style robust pixel loss. Dividing by 100 keeps the
F23:0894             # Adam step scale stable while preserving the minimizer.
F23:0895             residual = (projection - targets) / 100.0
F23:0896             loss = torch.sqrt(residual.square() + 1e-4).mean()
F23:0897             if not torch.isfinite(loss):
F23:0898                 break
F23:0899             optimizer.zero_grad()
F23:0900             loss.backward()
F23:0901             optimizer.step()
F23:0902             with torch.no_grad():
F23:0903                 rotation_norm = torch.linalg.vector_norm(parameter[:3])
F23:0904                 if rotation_norm > np.pi:
F23:0905                     parameter[:3].mul_(float(np.pi / rotation_norm))
F23:0906                     parameter[3].clamp_(-3.0, 3.0)
F23:0907                     parameter[4].clamp_(-3.0, 3.0)
F23:0908                     parameter[5].clamp_(float(np.log(0.1)), float(np.log(5.0)))
F23:0910             with torch.no_grad():
F23:0911                 rotation = axis_angle_to_matrix(parameter[:3])
F23:0912                 translation = torch.stack(
F23:0913                     (parameter[3], parameter[4], torch.exp(parameter[5]))
F23:0914                 )
F23:0915                 projection = project(rotation, translation)
F23:0916                 error = torch.linalg.vector_norm(projection - targets, dim=1).mean()
F23:0917                 candidate = (
F23:0918                     float(error),
F23:0919                     rotation.detach().clone(),
F23:0920                     translation.detach().clone(),
F23:0921                     projection.detach().clone(),
F23:0922                 )
F23:0923                 if best is None or candidate[0] < best[0]:
F23:0924                     best = candidate
F23:0926             if best is None:
F23:0927                 raise RuntimeError("Camera translation refinement produced no finite solution")
F23:0928             return {
F23:0929                 "rotation": best[1],
F23:0930                 "translation": best[2],
F23:0931                 "initial_projection": initial_projection.detach(),
F23:0932                 "refined_projection": best[3],
F23:0933                 "initial_mean_error_px": float(initial_error),
F23:0934                 "refined_mean_error_px": float(best[0]),
F23:0935                 "joint_indices": CAMERA_FIT_BODY_INDICES.copy(),
F23:0936             }
F23:0939 def refine_camera_translation(

```

```

F23:0940     body_joints_local,
F23:0941     body_targets_px,
F23:0942     initial_translation,
F23:0943     fisheye_cam,
F23:0944     iterations,
F23:0945     learning_rate,
F23:0946 ):
F23:0947     """Fit tx, ty, and positive tz while keeping pose and rotation fixed."""
F23:0948     if iterations <= 0:
F23:0949         raise ValueError("--camera-fit-iters must be positive")
F23:0950     if learning_rate <= 0:
F23:0951         raise ValueError("--camera-fit-lr must be positive")
F23:0953     device = body_joints_local.device
F23:0954     dtype = body_joints_local.dtype
F23:0955     indices = torch.as_tensor(CAMERA_FIT_BODY_INDICES, device=device, dtype=torch.long)
F23:0956     joints = body_joints_local.index_select(0, indices).detach()
F23:0957     targets = torch.as_tensor(
F23:0958         body_targets_px[CAMERA_FIT_BODY_INDICES], device=device, dtype=dtype
F23:0959     )
F23:0960     initial = initial_translation.detach().to(device=device, dtype=dtype)
F23:0961     identity = torch.eye(3, device=device, dtype=dtype)
F23:0963     def project(translation):
F23:0964         return fisheye_cam.world2camera_pytorch(joints + translation[None, :])
F23:0966     with torch.no_grad():
F23:0967         initial_projection = project(initial)
F23:0968         initial_error = torch.linalg.vector_norm(
F23:0969             initial_projection - targets, dim=1
F23:0970         ).mean()
F23:0972     initial_z = max(0.1, float(initial[2].item()))
F23:0973     depth_starts = sorted(set([initial_z, 0.5, 0.75, 1.0, 1.5, 2.0]))
F23:0974     best = None
F23:0975     for depth_start in depth_starts:
F23:0976         parameter = torch.tensor(
F23:0977             [
F23:0978                 float(initial[0]),
F23:0979                 float(initial[1]),
F23:0980                 float(np.log(depth_start)),
F23:0981             ],
F23:0982             device=device,
F23:0983             dtype=dtype,
F23:0984             requires_grad=True,
F23:0985         )
F23:0986         optimizer = torch.optim.Adam([parameter], lr=learning_rate)
F23:0987         for _ in range(iterations):
F23:0988             translation = torch.stack(
F23:0989                 (parameter[0], parameter[1], torch.exp(parameter[2]))
F23:0990             )
F23:0991             projection = project(translation)
F23:0992             residual = (projection - targets) / 100.0
F23:0993             loss = torch.sqrt(residual.square() + 1e-4).mean()

```

```

F23:0994         if not torch.isfinite(loss):
F23:0995             break
F23:0996         optimizer.zero_grad()
F23:0997         loss.backward()
F23:0998         optimizer.step()
F23:0999         with torch.no_grad():
F23:1000             parameter[0].clamp_(-3.0, 3.0)
F23:1001             parameter[1].clamp_(-3.0, 3.0)
F23:1002             parameter[2].clamp_(float(np.log(0.1)), float(np.log(5.0)))
F23:1004     with torch.no_grad():
F23:1005         translation = torch.stack(
F23:1006             (parameter[0], parameter[1], torch.exp(parameter[2])))
F23:1007     )
F23:1008     projection = project(translation)
F23:1009     error = torch.linalg.vector_norm(projection - targets, dim=1).mean()
F23:1010     candidate = (
F23:1011         float(error),
F23:1012         translation.detach().clone(),
F23:1013         projection.detach().clone(),
F23:1014     )
F23:1015     if best is None or candidate[0] < best[0]:
F23:1016         best = candidate
F23:1018     if best is None:
F23:1019         raise RuntimeError("Camera translation refinement produced no finite solution")
F23:1020     return {
F23:1021         "rotation": identity,
F23:1022         "translation": best[1],
F23:1023         "initial_projection": initial_projection.detach(),
F23:1024         "refined_projection": best[2],
F23:1025         "initial_mean_error_px": float(initial_error),
F23:1026         "refined_mean_error_px": float(best[0]),
F23:1027         "joint_indices": CAMERA_FIT_BODY_INDICES.copy(),
F23:1028     }
F23:1031 def draw_camera_fit_debug(
F23:1032     image_rgb,
F23:1033     target_body_px,
F23:1034     initial_body_px,
F23:1035     refined_body_px,
F23:1036     initial_error,
F23:1037     refined_error,
F23:1038 ):
F23:1039     canvas = image_rgb.copy()
F23:1040     series = (
F23:1041         (target_body_px, (255, 0, 0), "PositionNet"),
F23:1042         (initial_body_px, (0, 0, 255), "network cam"),
F23:1043         (refined_body_px, (0, 255, 0), "refined cam"),
F23:1044     )
F23:1045     for points, color, _ in series:
F23:1046         for start, end in CAMERA_FIT_BODY_EDGES:
F23:1047             if np.isfinite(points[[start, end]]).all():

```

```

F23:1048                cv2.line(
F23:1049                    canvas,
F23:1050                    tuple(np rint(points[start]).astype(int)),
F23:1051                    tuple(np rint(points[end]).astype(int)),
F23:1052                    color,
F23:1053                    4,
F23:1054                    cv2.LINE_AA,
F23:1055                )
F23:1056            for index in CAMERA_FIT_BODY_INDICES:
F23:1057                if np.isfinite(points[index]).all():
F23:1058                    cv2.circle(
F23:1059                        canvas,
F23:1060                        tuple(np rint(points[index]).astype(int)),
F23:1061                        8,
F23:1062                        color,
F23:1063                        -1,
F23:1064                        cv2.LINE_AA,
F23:1065                    )
F23:1066            labels = (
F23:1067                ((255, 0, 0), "red: PositionNet target"),
F23:1068                ((0, 0, 255), f"blue: network cam ({initial_error:.1f}px)"),
F23:1069                ((0, 255, 0), f"green: refined cam ({refined_error:.1f}px)"),
F23:1070            )
F23:1071            for row, (color, label) in enumerate(labels):
F23:1072                cv2.putText(
F23:1073                    canvas,
F23:1074                    label,
F23:1075                    (30, 45 + row * 38),
F23:1076                    cv2.FONT_HERSHEY_SIMPLEX,
F23:1077                    0.85,
F23:1078                    color,
F23:1079                    3,
F23:1080                    cv2.LINE_AA,
F23:1081                )
F23:1082            return canvas
F23:1085 def main():
F23:1086     args = parse_args()
F23:1087     paths_to_check = [args.image_dir, args.fisheye_calibration, args.config_path, args.checkpoint_pa
F23:1087+ th]
F23:1088     if not args.no_detector:
F23:1089         paths_to_check.extend([args.det_config, args.det_checkpoint])
F23:1090     for path in paths_to_check:
F23:1091         if not path.exists():
F23:1092             raise FileNotFoundError(path)
F23:1093     if args.no_detector and args.smplx_input_mode != "full_image":
F23:1094         raise ValueError("--no-detector requires --smplx-input-mode full_image")
F23:1095     if (
F23:1096         args.refine_camera_from_positionnet
F23:1097         and args.smplx_input_mode
F23:1098         not in ("full_image", "det_tangent", "det_undistort_contiguous")

```

```

F23:1099 ):
F23:1100     raise ValueError(
F23:1101         "Camera refinement requires --smlx-input-mode full_image, "
F23:1102         "det_tangent, or det_undistort_contiguous"
F23:1103     )
F23:1104     if args.horizontal_crop_pixels < 0:
F23:1105         raise ValueError("--horizontal-crop-pixels must be non-negative")
F23:1106     if args.det_tangent_expand <= 0:
F23:1107         raise ValueError("--det-tangent-expand must be positive")
F23:1108     if args.det_tangent_patch_size <= 0:
F23:1109         raise ValueError("--det-tangent-patch-size must be positive")
F23:1110     if not 0.0 <= args.global_undistort_alpha <= 1.0:
F23:1111         raise ValueError("--global-undistort-alpha must be in [0, 1]")
F23:1112     if (
F23:1113         args.smlx_input_mode in ("det_tangent", "det_undistort_contiguous")
F23:1114         and args.input_crop_mode != "none"
F23:1115     ):
F23:1116         raise ValueError(
F23:1117             f"{args.smlx_input_mode} requires --input-crop-mode none"
F23:1118         )
F23:1121     image_paths = sorted(args.image_dir.glob(args.image_glob))
F23:1122     if args.max_images is not None:
F23:1123         image_paths = image_paths[: args.max_images]
F23:1124     if not image_paths:
F23:1125         raise RuntimeError(f"No images found in {args.image_dir} with glob {args.image_glob}")
F23:1127     output_dir = make_output_dir(args)
F23:1128     overlay_dir = output_dir / "img"
F23:1129     wireframe_dir = output_dir / "wireframe"
F23:1130     smlx_dir = output_dir / "smlx"
F23:1131     meta_dir = output_dir / "meta"
F23:1132     det_dir = output_dir / "detections"
F23:1133     debug_dir = output_dir / "projection_debug"
F23:1134     mesh_dir = output_dir / "mesh"
F23:1135     vit_patch_dir = output_dir / "vit_patches"
F23:1136     positionnet_dir = output_dir / "positionnet"
F23:1137     undistorted_dir = output_dir / "undistorted"
F23:1138     undistorted_detection_dir = output_dir / "undistorted_detections"
F23:1139     joint_compare_dir = output_dir / "joint_compare"
F23:1140     mesh_joint_compare_dir = output_dir / "mesh_joint_compare"
F23:1141     for path in (overlay_dir, smlx_dir, meta_dir, det_dir, debug_dir):
F23:1142         path.mkdir(parents=True, exist_ok=True)
F23:1143     if args.smlx_input_mode in ("det_tangent", "det_undistort_contiguous"):
F23:1144         vit_patch_dir.mkdir(parents=True, exist_ok=True)
F23:1145         positionnet_dir.mkdir(parents=True, exist_ok=True)
F23:1146     if args.smlx_input_mode == "det_undistort_contiguous":
F23:1147         undistorted_dir.mkdir(parents=True, exist_ok=True)
F23:1148         undistorted_detection_dir.mkdir(parents=True, exist_ok=True)
F23:1149         joint_compare_dir.mkdir(parents=True, exist_ok=True)
F23:1150         mesh_joint_compare_dir.mkdir(parents=True, exist_ok=True)
F23:1151     if args.save_mesh:

```

```

F23:1152         mesh_dir.mkdir(parents=True, exist_ok=True)
F23:1153         if args.save_wireframe_overlay:
F23:1154             wireframe_dir.mkdir(parents=True, exist_ok=True)
F23:1156         demoer = setup_model(args, output_dir)
F23:1157         FishEyeCameraCalibrated, smpl_x, load_img, process_bbox, generate_patch_image, render_mesh_fishe
F23:1157+ ye, save_obj = load_runtime_utils()
F23:1158         fisheye_cam = FishEyeCameraCalibrated(str(args.fisheye_calibration.resolve()))
F23:1159         positionnet_capture = {}
F23:1160         if args.smplx_input_mode in ("det_tangent", "det_undistort_contiguous"):
F23:1161             model_core = demoer.model.module if hasattr(demoer.model, "module") else demoer.model
F23:1163             def capture_positionnet_output(_module, _inputs, outputs):
F23:1164                 positionnet_capture["body_joint_hm"] = outputs[0].detach().cpu()
F23:1165                 positionnet_capture["body_joint_img"] = outputs[1].detach().cpu()
F23:1167             model_core.body_position_net.register_forward_hook(capture_positionnet_output)
F23:1168         detector = None
F23:1169         if not args.no_detector:
F23:1170             detector = init_detector(str(args.det_config.resolve()), str(args.det_checkpoint.resolve()),
F23:1170+ device="cuda:0")
F23:1171         transform = transforms.ToTensor()
F23:1173         first = cv2.imread(str(image_paths[0]), cv2.IMREAD_COLOR)
F23:1174         if first is None:
F23:1175             raise RuntimeError(f"Failed to read {image_paths[0]}")
F23:1176         img_h0, img_w0 = first.shape[:2]
F23:1177         undistort_model = None
F23:1178         if args.smplx_input_mode == "det_undistort_contiguous":
F23:1179             undistort_model = create_global_undistort_model(
F23:1180                 args.fisheye_calibration,
F23:1181                 img_w0,
F23:1182                 img_h0,
F23:1183                 args.global_undistort_alpha,
F23:1184             )
F23:1185         video_writer = None
F23:1186         if args.save_video:
F23:1187             fourcc = cv2.VideoWriter_fourcc(*"mp4v")
F23:1188             video_writer = cv2.VideoWriter(str(output_dir / "detected_mesh_overlay_stride10.mp4"), fourc
F23:1188+ c, 10.0, (img_w0, img_h0))
F23:1190         saved_images = []
F23:1191         saved_wireframe_images = []
F23:1192         saved_sampling_grid_images = []
F23:1193         saved_patch_atlas_images = []
F23:1194         saved_positionnet_images = []
F23:1195         saved_camera_fit_images = []
F23:1196         saved_joint_compare_images = []
F23:1197         saved_mesh_joint_compare_images = []
F23:1198         saved_undistorted_images = []
F23:1199         saved_undistorted_detection_images = []
F23:1200         saved_contiguous_patch_grid_images = []
F23:1201         records = []
F23:1202         projection_consistency_errors = []
F23:1203         total_persons = 0

```

```

F23:1204     no_detection = 0
F23:1205     start_time = time.time()
F23:1207     progress_desc = "Full-image project" if args.no_detector else "Detect + project"
F23:1208     with tqdm(image_paths, desc=progress_desc, mininterval=10.0) as pbar:
F23:1209         for image_path in pbar:
F23:1210             original_img = load_img(str(image_path))
F23:1211             vis_img = original_img.copy()
F23:1212             wireframe_img = original_img.copy() if args.save_wireframe_overlay else None
F23:1213             img_h, img_w = original_img.shape[:2]
F23:1214             detector_img = original_img
F23:1215             if args.smplx_input_mode == "det_undistort_contiguous":
F23:1216                 if (img_w, img_h) != (img_w0, img_h0):
F23:1217                     raise ValueError(
F23:1218                         "All globally undistorted frames must share one size; "
F23:1219                         f"expected {(img_w0, img_h0)}, got {(img_w, img_h)}"
F23:1220                     )
F23:1221                 detector_img = undistort_full_image(original_img, undistort_model)
F23:1222                 undistorted_out = undistorted_dir / f"{image_path.stem}_undistorted.jpg"
F23:1223                 cv2.imwrite(str(undistorted_out), rgb_to_bgr_u8(detector_img))
F23:1224                 saved_undistorted_images.append(undistorted_out)
F23:1226             if args.no_detector:
F23:1227                 raw_boxes = []
F23:1228                 det_boxes = [np.array([0.0, 0.0, float(img_w - 1), float(img_h - 1), 1.0], dtype=np.
F23:1228+ float32)]
F23:1229             else:
F23:1230                 if args.smplx_input_mode == "det_undistort_contiguous":
F23:1231                     detector_input = np.ascontiguousarray(rgb_to_bgr_u8(detector_img))
F23:1232                 else:
F23:1233                     detector_input = str(image_path)
F23:1234                 mmdet_results = inference_detector(detector, detector_input)
F23:1235                 raw_boxes = process_mmdet_results(mmdet_results, cat_id=0, multi_person=True)[0]
F23:1236                 filtered = filter_bboxes(raw_boxes, args.score_thr, args.bbox_thr)
F23:1237                 if args.multi_person:
F23:1238                     det_boxes = non_max_suppression(filtered, args.iou_thr) if filtered else []
F23:1239                 else:
F23:1240                     det_boxes = filtered[:1]
F23:1242             if args.smplx_input_mode == "det_undistort_contiguous":
F23:1243                 detector_vis = detector_img.copy()
F23:1244                 for selected_box in det_boxes:
F23:1245                     dx1, dy1, dx2, dy2, dscore = [
F23:1246                         float(value) for value in selected_box[:5]
F23:1247                     ]
F23:1248                     cv2.rectangle(
F23:1249                         detector_vis,
F23:1250                         (int(round(dx1)), int(round(dy1))),
F23:1251                         (int(round(dx2)), int(round(dy2))),
F23:1252                         (255, 0, 0),
F23:1253                         3,
F23:1254                     )
F23:1255                 cv2.putText(

```



```

F23:1256             detector_vis,
F23:1257             f"person {dscore:.2f}",
F23:1258             (int(round(dx1)), max(25, int(round(dy1)) - 10)),
F23:1259             cv2.FONT_HERSHEY_SIMPLEX,
F23:1260             0.8,
F23:1261             (255, 0, 0),
F23:1262             2,
F23:1263             cv2.LINE_AA,
F23:1264         )
F23:1265         undistorted_detection_out = (
F23:1266             undistorted_detection_dir
F23:1267             / f"{image_path.stem}_undistorted_detection.jpg"
F23:1268         )
F23:1269         cv2.imwrite(
F23:1270             str(undistorted_detection_out),
F23:1271             rgb_to_bgr_u8(detector_vis),
F23:1272         )
F23:1273         saved_undistorted_detection_images.append(
F23:1274             undistorted_detection_out
F23:1275         )
F23:1277         det_record = {
F23:1278             "detector_enabled": not args.no_detector,
F23:1279             "image_path": str(image_path.resolve()),
F23:1280             "raw_person_detections": int(len(raw_boxes)),
F23:1281             "score_thr": float(args.score_thr),
F23:1282             "bbox_thr": float(args.bbox_thr),
F23:1283             "detector_coordinate_space": (
F23:1284                 "global_undistorted"
F23:1285                 if args.smplx_input_mode == "det_undistort_contiguous"
F23:1286                 else "original_fisheye"
F23:1287             ),
F23:1288             "selected_bboxes_xyxy_score": [np.asarray(b).astype(float).tolist() for b in det_box
F23:1288+ es],
F23:1289         }
F23:1290         (det_dir / f"{image_path.stem}_detections.json").write_text(json.dumps(det_record, inden
F23:1290+ t=2) + "\n")
F23:1292         if not det_boxes:
F23:1293             no_detection += 1
F23:1294             vis_img = draw_no_detection(vis_img)
F23:1295             out_img = overlay_dir / f"{image_path.stem}_det_mesh_overlay.jpg"
F23:1296             cv2.imwrite(str(out_img), rgb_to_bgr_u8(vis_img))
F23:1297             saved_images.append(out_img)
F23:1298             if wireframe_img is not None:
F23:1299                 wireframe_out = wireframe_dir / f"{image_path.stem}_det_mesh_wireframe.jpg"
F23:1300                 cv2.imwrite(str(wireframe_out), rgb_to_bgr_u8(wireframe_img))
F23:1301                 saved_wireframe_images.append(wireframe_out)
F23:1302             if video_writer is not None:
F23:1303                 video_writer.write(rgb_to_bgr_u8(vis_img))
F23:1304             records.append(**det_record, "persons_projected": 0)
F23:1305             continue

```

```

F23:1307         persons_projected = 0
F23:1308     for bbox_id, det_box in enumerate(det_boxes):
F23:1309         x1, y1, x2, y2, score = [float(v) for v in det_box[:5]]
F23:1310         mmdet_box_xywh = np.array([x1, y1, abs(x2 - x1), abs(y2 - y1)], dtype=np.float32)
F23:1311         sampling_bbox_xyxy = None
F23:1312         sampling_grid_pixels = None
F23:1313         if args.smplx_input_mode == "det_tangent":
F23:1314             sampling_bbox_xyxy = expand_bbox_xyxy(
F23:1315                 det_box,
F23:1316                 args.det_tangent_expand,
F23:1317                 img_w,
F23:1318                 img_h,
F23:1319             )
F23:1320             sx1, sy1, sx2, sy2 = sampling_bbox_xyxy
F23:1321             bbox = np.asarray([sx1, sy1, sx2 - sx1, sy2 - sy1], dtype=np.float32)
F23:1322         elif args.smplx_input_mode == "det_undistort_contiguous":
F23:1323             if args.contiguous_roi_mode == "stretch":
F23:1324                 sampling_bbox_xyxy = expand_bbox_xyxy(
F23:1325                     det_box,
F23:1326                     args.det_tangent_expand,
F23:1327                     img_w,
F23:1328                     img_h,
F23:1329                 )
F23:1330             else:
F23:1331                 sampling_bbox_xyxy = expand_bbox_to_aspect(
F23:1332                     det_box,
F23:1333                     args.det_tangent_expand,
F23:1334                     target_width_over_height=12.0 / 16.0,
F23:1335                 )
F23:1336             sx1, sy1, sx2, sy2 = sampling_bbox_xyxy
F23:1337             bbox = np.asarray(
F23:1338                 [sx1, sy1, sx2 - sx1, sy2 - sy1],
F23:1339                 dtype=np.float32,
F23:1340             )
F23:1341         else:
F23:1342             bbox = process_bbox(mmdet_box_xywh, img_w, img_h)
F23:1343         if bbox is None:
F23:1344             continue
F23:1346         if args.smplx_input_mode == "det_crop":
F23:1347             img_crop, img2bb_trans, bb2img_trans = generate_patch_image(
F23:1348                 original_img, bbox, 1.0, 0.0, False, cfg.input_img_shape
F23:1349             )
F23:1350             img = transform(img_crop.astype(np.float32)) / 255.0
F23:1351             img = img.cuda() [None, :, :, :]
F23:1352             meta_info = {}
F23:1353             # FishEyeCameraCalibrated.world2camera already returns
F23:1354             # original fisheye-image pixel coordinates. Applying
F23:1355             # bb2img_trans here double-applies the crop inverse and
F23:1356             # shifts/scales the mesh overlay.
F23:1357             render_proj_trans = None

```

```

F23:1358         else:
F23:1359             img2bb_trans = np.eye(2, 3, dtype=np.float32)
F23:1360             bb2img_trans = np.eye(2, 3, dtype=np.float32)
F23:1361             input_img = (
F23:1362                 detector_img
F23:1363                 if args.smplx_input_mode == "det_undistort_contiguous"
F23:1364                 else original_img
F23:1365             )
F23:1366             input_offset = np.array([0.0, 0.0], dtype=np.float32)
F23:1367             input_crop_xyxy = [0, 0, int(img_w), int(img_h)]
F23:1368             if args.input_crop_mode == "center_square":
F23:1369                 side = min(img_h, img_w)
F23:1370                 crop_left = int(round((img_w - side) / 2.0))
F23:1371                 crop_top = int(round((img_h - side) / 2.0))
F23:1372                 input_img = original_img[crop_top:crop_top + side, crop_left:crop_left + side, :]
F23:1373                 input_offset = np.array([crop_left, crop_top], dtype=np.float32)
F23:1374                 input_crop_xyxy = [crop_left, crop_top, crop_left + side, crop_top + side]
F23:1375             elif args.input_crop_mode == "training_horizontal":
F23:1376                 crop_left = int(args.horizontal_crop_pixels)
F23:1377                 crop_right = int(args.horizontal_crop_pixels)
F23:1378                 if crop_left + crop_right >= img_w:
F23:1379                     raise ValueError(
F23:1380                         "Horizontal crop removes the entire image: "
F23:1381                         f"width={img_w}, left={crop_left}, right={crop_right}"
F23:1382                     )
F23:1383                 input_img = original_img[:, crop_left:img_w - crop_right, :]
F23:1384                 input_offset = np.array([crop_left, 0.0], dtype=np.float32)
F23:1385                 input_crop_xyxy = [crop_left, 0, img_w - crop_right, img_h]
F23:1386                 input_h, input_w = input_img.shape[:2]
F23:1387                 img = transform(input_img.astype(np.float32)) / 255.0
F23:1388                 img = img.cuda()[None, :, :, :]
F23:1389                 meta_info = {
F23:1390                     "camera_id": torch.zeros((1,), dtype=torch.long, device=img.device),
F23:1391                     "input_img_shape": torch.tensor([[input_h, input_w]], dtype=torch.float32, device=img.device),
F23:1392                     "input_coord_offset": torch.from_numpy(input_offset[None]).to(device=img.device, dtype=torch.float32),
F23:1393                 }
F23:1394                 render_proj_trans = None
F23:1395                 if args.smplx_input_mode in (
F23:1396                     "det_tangent",
F23:1397                     "det_undistort_contiguous",
F23:1398                 ):
F23:1399                     model_core = demoer.model.module if hasattr(demoer.model, "module") else demoer.model
F23:1400                     sampler = model_core.encoder.fisheye2sphere
F23:1401                     patch_rows = int(sampler.patch_num_vertical)
F23:1402                     patch_columns = int(sampler.selected_patch_indices.numel() // patch_rows)
F23:1403                     if patch_rows * patch_columns != 192:

```

```
F23:1404         raise ValueError(
F23:1405             "Detection-aligned sampling expects the checkpoint's "
F23:1406             "192-token 16-row x 12-column ViT layout: "
F23:1407             f"got rows={patch_rows}, columns={patch_columns}"
F23:1408         )
F23:1409     if args.smplx_input_mode == "det_tangent":
F23:1410         sampling_grid_pixels = create_bbox_tangent_pixel_grid(
F23:1411             sampler,
F23:1412             fisheye_cam,
F23:1413             sampling_bbox_xyxy,
F23:1414             patch_rows=patch_rows,
F23:1415             patch_columns=patch_columns,
F23:1416             patch_size=args.det_tangent_patch_size,
F23:1417         )
F23:1418     else:
F23:1419         patch_h, patch_w = [
F23:1420             int(value)
F23:1421             for value in sampler.patches_2d_per_camera_pixels.shape[-3:-1]
F23:1422         ]
F23:1423         sampling_grid_pixels = create_contiguous_bbox_pixel_grid(
F23:1424             sampling_bbox_xyxy,
F23:1425             patch_rows=patch_rows,
F23:1426             patch_columns=patch_columns,
F23:1427             patch_h=patch_h,
F23:1428             patch_w=patch_w,
F23:1429         )
F23:1430         sampler.patches_2d_per_camera_pixels = torch.from_numpy(
F23:1431             sampling_grid_pixels[None]
F23:1432         ).to(device=img.device, dtype=img.dtype)
F23:1433         sampler.selected_patch_indices = torch.arange(
F23:1434             sampling_grid_pixels.shape[0],
F23:1435             device=img.device,
F23:1436             dtype=torch.long,
F23:1437         )
F23:1438         with torch.no_grad():
F23:1439             sampled_vit_patches = sampler(
F23:1440                 img,
F23:1441                 camera_ids=meta_info["camera_id"],
F23:1442                 input_img_shapes=meta_info["input_img_shape"],
F23:1443                 input_coord_offsets=meta_info["input_coord_offset"],
F23:1444             )
F23:1445         patch_prefix = f"{image_path.stem}_{bbox_id:02d}"
F23:1446         np.save(
F23:1447             vit_patch_dir / f"{patch_prefix}_sampling_grid_pixels.npy",
F23:1448             sampling_grid_pixels,
F23:1449         )
F23:1450     if args.smplx_input_mode == "det_tangent":
F23:1451         sampling_overlay = draw_tangent_sampling_grid(
F23:1452             original_img,
F23:1453             sampling_grid_pixels,
```

```

F23:1454             sampling_bbox_xyxy,
F23:1455         )
F23:1456     else:
F23:1457         sampling_overlay = draw_contiguous_sampling_grid(
F23:1458             detector_img,
F23:1459             sampling_grid_pixels,
F23:1460             sampling_bbox_xyxy,
F23:1461         )
F23:1462         sampling_grid_out = vit_patch_dir / f"{patch_prefix}_sampling_grid.jpg"
F23:1463         patch_atlas_out = vit_patch_dir / f"{patch_prefix}_patch_atlas.jpg"
F23:1464         cv2.imwrite(str(sampling_grid_out), rgb_to_bgr_u8(sampling_overlay))
F23:1465         if args.smplx_input_mode == "det_tangent":
F23:1466             save_vit_patch_atlas(
F23:1467                 sampled_vit_patches,
F23:1468                 patch_atlas_out,
F23:1469                 rows=patch_rows,
F23:1470                 columns=patch_columns,
F23:1471             )
F23:1472     else:
F23:1473         save_vit_patch_atlas(
F23:1474             sampled_vit_patches,
F23:1475             patch_atlas_out,
F23:1476             rows=patch_rows,
F23:1477             columns=patch_columns,
F23:1478             gap=0,
F23:1479         )
F23:1480         patch_grid_out = (
F23:1481             vit_patch_dir
F23:1482             / f"{patch_prefix}_patch_atlas_grid.jpg"
F23:1483         )
F23:1484         save_vit_patch_atlas(
F23:1485             sampled_vit_patches,
F23:1486             patch_grid_out,
F23:1487             rows=patch_rows,
F23:1488             columns=patch_columns,
F23:1489             gap=0,
F23:1490             grid_color_rgb=(255, 0, 0),
F23:1491         )
F23:1492         saved_contiguous_patch_grid_images.append(
F23:1493             patch_grid_out
F23:1494         )
F23:1495         saved_sampling_grid_images.append(sampling_grid_out)
F23:1496         saved_patch_atlas_images.append(patch_atlas_out)
F23:1497     positionnet_capture.clear()
F23:1498     with torch.no_grad():
F23:1499         out = demoer.model1({"img": img, "img_ori": img}, {}, meta_info, "test")
F23:1500     body_joint_pixels_undistorted = None
F23:1501     if args.smplx_input_mode in (
F23:1502         "det_tangent",
F23:1503         "det_undistort_contiguous",

```

```

F23:1504         ):
F23:1505             if not {"body_joint_hm", "body_joint_img"}.issubset(positionnet_capture):
F23:1506                 raise RuntimeError("PositionNet forward hook did not capture its outputs")
F23:1507             body_joint_hm = positionnet_capture["body_joint_hm"].numpy()[0]
F23:1508             body_joint_img = positionnet_capture["body_joint_img"].numpy()[0]
F23:1509             body_joint_pixels_sampling, patch_center_grid_pixels = (
F23:1510                 positionnet_coords_to_sampling_grid(
F23:1511                     body_joint_img,
F23:1512                     sampling_grid_pixels,
F23:1513                     patch_rows=patch_rows,
F23:1514                     patch_columns=patch_columns,
F23:1515                 )
F23:1516             )
F23:1517             if args.smplx_input_mode == "det_undistort_contiguous":
F23:1518                 body_joint_pixels_undistorted = body_joint_pixels_sampling
F23:1519                 body_joint_pixels = undistorted_pixels_to_original(
F23:1520                     body_joint_pixels_undistorted,
F23:1521                     undistort_model,
F23:1522                 )
F23:1523                 positionnet_canvas = detector_img
F23:1524                 positionnet_canvas_points = body_joint_pixels_undistorted
F23:1525             else:
F23:1526                 body_joint_pixels = body_joint_pixels_sampling
F23:1527                 positionnet_canvas = original_img
F23:1528                 positionnet_canvas_points = body_joint_pixels
F23:1529             positionnet_prefix = f"{image_path.stem}_{bbox_id:02d}"
F23:1530             positionnet_payload = {
F23:1531                 "body_joint_hm": body_joint_hm.astype(np.float32),
F23:1532                 "body_joint_img": body_joint_img.astype(np.float32),
F23:1533                 "body_joint_xy_original_from_grid": body_joint_pixels.astype(np.float32),
F23:1534                 "patch_center_grid_pixels": patch_center_grid_pixels.astype(np.float32),
F23:1535                 "body_joint_names": np.asarray(smpl_x.pos_joints_name[:25]),
F23:1536                 "sampling_bbox_xyxy": sampling_bbox_xyxy.astype(np.float32),
F23:1537                 "output_hm_shape": np.asarray(cfg.output_hm_shape, dtype=np.int32),
F23:1538             }
F23:1539             if body_joint_pixels_undistorted is not None:
F23:1540                 positionnet_payload[
F23:1541                     "body_joint_xy_global_undistorted"
F23:1542                 ] = body_joint_pixels_undistorted.astype(np.float32)
F23:1543                 positionnet_payload[
F23:1544                     "global_undistorted_new_k"
F23:1545                 ] = undistort_model["new_k"].astype(np.float64)
F23:1546             np.savez_compressed(
F23:1547                 positionnet_dir / f"{positionnet_prefix}_positionnet.npz",
F23:1548                 **positionnet_payload,
F23:1549             )
F23:1550             positionnet_vis = draw_positionnet_result(
F23:1551                 positionnet_canvas,
F23:1552                 body_joint_hm,
F23:1553                 positionnet_canvas_points,

```

```

F23:1554             sampling_bbox_xyxy,
F23:1555         )
F23:1556         positionnet_vis_out = positionnet_dir / f"{positionnet_prefix}_positionnet.jpg"
F23:1557         cv2.imwrite(str(positionnet_vis_out), rgb_to_bgr_u8(positionnet_vis))
F23:1558         saved_positionnet_images.append(positionnet_vis_out)
F23:1559         mesh = out["smplx_mesh_cam"].detach().cpu().numpy()[0]
F23:1560         network_translation = out["cam_trans"].detach()[0]
F23:1561         render_translation = network_translation
F23:1562         network_global_orient = out["smplx_root_pose"].detach()[0]
F23:1563         render_global_orient = network_global_orient.clone()
F23:1564         identity_rotation = torch.eye(
F23:1565             3,
F23:1566             device=network_global_orient.device,
F23:1567             dtype=network_global_orient.dtype,
F23:1568         )
F23:1569         global_orient_delta_matrix = identity_rotation.clone()
F23:1570         camera_rotation_delta = identity_rotation.clone()
F23:1571         global_orient_delta_degrees = 0.0
F23:1572         global_orient_mesh_consistency_max_error_m = None
F23:1573         global_orient_stage_mean_error_px = None
F23:1574         body_targets_px = None
F23:1575         body_projection_initial_px = None
F23:1576         body_projection_refined_px = None
F23:1577         body_projection_model_px = None
F23:1578         projection_consistency_max_error_px = None
F23:1579         camera_fit = None
F23:1581         if args.smplx_input_mode in (
F23:1582             "full_image",
F23:1583             "det_tangent",
F23:1584             "det_undistort_contiguous",
F23:1585         ):
F23:1586             # smplx_joint_cam_pred is root-relative and is intended for
F23:1587             # 3D evaluation. Rebuild the absolute local joints from the
F23:1588             # predicted SMPL-X parameters before adding cam_trans; these
F23:1589             # are the joints used by get_coord to create joint_proj.
F23:1590             zero_eye_pose = torch.zeros_like(out["smplx_jaw_pose"])
F23:1591             with torch.no_grad():
F23:1592                 model_core = demoer.model.module if hasattr(demoer.model, "module") else dem
F23:1592+ oer.model
F23:1593                 reconstructed_smplx = model_core.smplx_layer(
F23:1594                     betas=out["smplx_shape"],
F23:1595                     body_pose=out["smplx_body_pose"],
F23:1596                     global_orient=out["smplx_root_pose"],
F23:1597                     right_hand_pose=out["smplx_rhand_pose"],
F23:1598                     left_hand_pose=out["smplx_lhand_pose"],
F23:1599                     jaw_pose=out["smplx_jaw_pose"],
F23:1600                     leye_pose=zero_eye_pose,
F23:1601                     reye_pose=zero_eye_pose,
F23:1602                     expression=out["smplx_expr"],
F23:1603                 )

```

```

F23:1604         body_joints_local = reconstructed_smplx.joints[0, smplx.joint_idx[:25], :]
F23:1605         body_joints_local_render = body_joints_local
F23:1606         if args.smplx_input_mode in (
F23:1607             "det_tangent",
F23:1608             "det_undistort_contiguous",
F23:1609         ):
F23:1610             body_targets_px = body_joint_pixels
F23:1611         else:
F23:1612             joint_img = out["joint_img"].detach().cpu().numpy()[0]
F23:1613             body_targets_px = heatmap_joints_to_original_pixels(
F23:1614                 joint_img[:25, :2], (input_h, input_w), input_offset
F23:1615             )
F23:1616         with torch.no_grad():
F23:1617             body_projection_initial = fisheye_cam.world2camera_pytorch(
F23:1618                 body_joints_local + network_translation[None, :]
F23:1619             )
F23:1620             body_projection_initial_px = body_projection_initial.cpu().numpy()
F23:1621             body_projection_refined_px = body_projection_initial_px.copy()
F23:1622             body_projection_model_px = heatmap_joints_to_original_pixels(
F23:1623                 out["smplx_joint_proj"].detach().cpu().numpy()[0, :25, :2],
F23:1624                 (input_h, input_w),
F23:1625                 input_offset,
F23:1626             )
F23:1627             projection_consistency_max_error_px = float(
F23:1628                 np.linalg.norm(
F23:1629                     body_projection_model_px - body_projection_initial_px,
F23:1630                     axis=1,
F23:1631                 ).max()
F23:1632             )
F23:1633             projection_consistency_errors.append(projection_consistency_max_error_px)
F23:1635             if args.refine_camera_from_positionnet:
F23:1636                 camera_fit_fn = (
F23:1637                     refine_camera_translation
F23:1638                     if args.camera_fit_mode == "translation"
F23:1639                     else refine_camera_extrinsics
F23:1640                 )
F23:1641                 camera_fit = camera_fit_fn(
F23:1642                     body_joints_local,
F23:1643                     body_targets_px,
F23:1644                     network_translation,
F23:1645                     fisheye_cam,
F23:1646                     args.camera_fit_iters,
F23:1647                     args.camera_fit_lr,
F23:1648                 )
F23:1649                 render_rotation = camera_fit["rotation"]
F23:1650                 render_translation = camera_fit["translation"]
F23:1651                 if args.camera_fit_mode == "translation_global_orient":
F23:1652                     global_orient_stage_mean_error_px = camera_fit[
F23:1653                         "refined_mean_error_px"
F23:1654                     ]

```



```

F23:1655         global_orient_delta_matrix = render_rotation
F23:1656         render_global_orient = compose_global_orient(
F23:1657             render_rotation,
F23:1658             network_global_orient,
F23:1659         )
F23:1660         cosine = ((torch.trace(render_rotation) - 1.0) * 0.5).clamp(-1.0, 1.0)
F23:1661         global_orient_delta_degrees = float(
F23:1662             torch.rad2deg(torch.acos(cosine)).item()
F23:1663         )
F23:1664         with torch.no_grad():
F23:1665             refined_smplx = model_core.smplx_layer(
F23:1666                 betas=out["smplx_shape"],
F23:1667                 body_pose=out["smplx_body_pose"],
F23:1668                 global_orient=render_global_orient[None, :],
F23:1669                 right_hand_pose=out["smplx_rhand_pose"],
F23:1670                 left_hand_pose=out["smplx_lhand_pose"],
F23:1671                 jaw_pose=out["smplx_jaw_pose"],
F23:1672                 leye_pose=zero_eye_pose,
F23:1673                 reye_pose=zero_eye_pose,
F23:1674                 expression=out["smplx_expr"],
F23:1675             )
F23:1676         body_joints_local_render = refined_smplx.joints[
F23:1677             0, smplx.joint_idx[:25], :
F23:1678         ]
F23:1679         translation_refit = refine_camera_translation(
F23:1680             body_joints_local_render,
F23:1681             body_targets_px,
F23:1682             render_translation,
F23:1683             fisheye_cam,
F23:1684             args.camera_fit_iters,
F23:1685             args.camera_fit_lr,
F23:1686         )
F23:1687         render_translation = translation_refit["translation"]
F23:1688         camera_fit["translation"] = render_translation
F23:1689         camera_fit["refined_projection"] = translation_refit[
F23:1690             "refined_projection"
F23:1691         ]
F23:1692         camera_fit["refined_mean_error_px"] = translation_refit[
F23:1693             "refined_mean_error_px"
F23:1694         ]
F23:1695         mesh = (
F23:1696             refined_smplx.vertices[0]
F23:1697             + render_translation[None, :]
F23:1698         ).detach().cpu().numpy()
F23:1699         mesh_from_refined_params = mesh.copy()
F23:1700         global_orient_mesh_consistency_max_error_m = float(
F23:1701             np.linalg.norm(
F23:1702                 mesh - mesh_from_refined_params,
F23:1703                 axis=1,
F23:1704             ).max()

```

```

F23:1705         )
F23:1706         elif args.camera_fit_mode == "extrinsics":
F23:1707             camera_rotation_delta = render_rotation
F23:1708             body_joints_local_render = (
F23:1709                 body_joints_local @ render_rotation.t()
F23:1710             )
F23:1711         else:
F23:1712             body_joints_local_render = body_joints_local
F23:1713         with torch.no_grad():
F23:1714             body_projection_refined = fisheye_cam.world2camera_pytorch(
F23:1715                 body_joints_local_render
F23:1716                 + render_translation[None, :])
F23:1717         )
F23:1718         body_projection_refined_px = body_projection_refined.cpu().numpy()
F23:1719         if args.camera_fit_mode != "translation_global_orient":
F23:1720             network_translation_np = network_translation.cpu().numpy()
F23:1721             render_translation_np = render_translation.cpu().numpy()
F23:1722             mesh = (
F23:1723                 (mesh - network_translation_np[None, :])
F23:1724                 @ render_rotation.cpu().numpy().T
F23:1725                 + render_translation_np[None, :])
F23:1726             )
F23:1727         debug_image = draw_camera_fit_debug(
F23:1728             original_img,
F23:1729             body_targets_px,
F23:1730             body_projection_initial_px,
F23:1731             body_projection_refined_px,
F23:1732             camera_fit["initial_mean_error_px"],
F23:1733             camera_fit["refined_mean_error_px"],
F23:1734         )
F23:1735         camera_fit_out = debug_dir / f"{image_path.stem}_{bbox_id:02d}_camera_fit.jp
F23:1735+ g"
F23:1736         cv2.imwrite(str(camera_fit_out), rgb_to_bgr_u8(debug_image))
F23:1737         saved_camera_fit_images.append(camera_fit_out)
F23:1739         if body_targets_px is not None:
F23:1740             joint_compare_image = draw_positionnet_smplx_2d_comparison(
F23:1741                 original_img,
F23:1742                 body_targets_px,
F23:1743                 body_projection_refined_px,
F23:1744             )
F23:1745             joint_compare_out = (
F23:1746                 joint_compare_dir
F23:1747                 / f"{image_path.stem}_{bbox_id:02d}_positionnet_vs_smplx.jpg"
F23:1748             )
F23:1749             cv2.imwrite(
F23:1750                 str(joint_compare_out),
F23:1751                 rgb_to_bgr_u8(joint_compare_image),
F23:1752             )
F23:1753             saved_joint_compare_images.append(joint_compare_out)
F23:1755         smplx_pred = {

```

```

F23:1756         "global_orient": render_global_orient.reshape(-1, 3).detach().cpu().numpy(),
F23:1757         "global_orient_network": network_global_orient.reshape(-1, 3).detach().cpu().num
F23:1757+ py(),
F23:1758         "global_orient_delta_matrix": global_orient_delta_matrix.reshape(1, 3, 3).detach
F23:1758+ ().cpu().numpy(),
F23:1759         "body_pose": out["smplx_body_pose"].reshape(-1, 3).detach().cpu().numpy(),
F23:1760         "left_hand_pose": out["smplx_lhand_pose"].reshape(-1, 3).detach().cpu().numpy(),
F23:1761         "right_hand_pose": out["smplx_rhand_pose"].reshape(-1, 3).detach().cpu().numpy()
F23:1761+ ,
F23:1762         "jaw_pose": out["smplx_jaw_pose"].reshape(-1, 3).detach().cpu().numpy(),
F23:1763         "leye_pose": np.zeros((1, 3), dtype=np.float32),
F23:1764         "reye_pose": np.zeros((1, 3), dtype=np.float32),
F23:1765         "betas": out["smplx_shape"].reshape(-1, 10).detach().cpu().numpy(),
F23:1766         "expression": out["smplx_expr"].reshape(-1, 10).detach().cpu().numpy(),
F23:1767         "transl": render_translation.reshape(-1, 3).detach().cpu().numpy(),
F23:1768         "transl_network": network_translation.reshape(-1, 3).detach().cpu().numpy(),
F23:1769         "camera_rotation_delta": camera_rotation_delta.reshape(1, 3, 3).detach().cpu().n
F23:1769+ ump(),
F23:1770         "det_bbox_xyxy_score": np.asarray(det_box).reshape(1, 5).astype(np.float32),
F23:1771         "smplx_joint_cam_root_relative": out["smplx_joint_cam_pred"][0].detach().cpu().n
F23:1771+ ump().astype(np.float32),
F23:1772         "smplx_mesh_cam": mesh.astype(np.float32),
F23:1773     }
F23:1774     if body_targets_px is not None:
F23:1775         body_joints_camera_network = body_joints_local + network_translation[None, :]
F23:1776         if camera_fit is not None:
F23:1777             body_joints_camera_render = (
F23:1778                 body_joints_local_render + render_translation[None, :]
F23:1779             )
F23:1780         else:
F23:1781             body_joints_camera_render = body_joints_camera_network
F23:1782         smplx_pred["smplx_body_joints_3d_local"] = body_joints_local.detach().cpu().nump
F23:1782+ y().astype(np.float32)
F23:1783         if args.camera_fit_mode == "translation_global_orient" and camera_fit is not Non
F23:1783+ e:
F23:1784             smplx_pred["smplx_body_joints_3d_local_refined_global_orient"] = (
F23:1785                 body_joints_local_render
F23:1786             ).detach().cpu().numpy().astype(np.float32)
F23:1787             smplx_pred["smplx_body_joints_3d_network_cam"] = body_joints_camera_network.deta
F23:1787+ ch().cpu().numpy().astype(np.float32)
F23:1788             smplx_pred["smplx_body_joints_3d_render_cam"] = body_joints_camera_render.detach
F23:1788+ ().cpu().numpy().astype(np.float32)
F23:1789             smplx_pred["positionnet_body_joints_2d"] = body_targets_px.astype(np.float32)
F23:1790             if body_joint_pixels_undistorted is not None:
F23:1791                 smplx_pred[
F23:1792                     "positionnet_body_joints_2d_global_undistorted"
F23:1793                 ] = body_joint_pixels_undistorted.astype(np.float32)
F23:1794             smplx_pred["smplx_body_joints_2d_network_cam"] = body_projection_initial_px.astyp
F23:1794+ e(np.float32)
F23:1795             smplx_pred["smplx_body_joints_2d_render_cam"] = body_projection_refined_px.astyp

```

```

F23:1795+ e(np.float32)
F23:1796             smplx_pred["smplx_body_joints_2d_model_output"] = body_projection_model_px.astyp
F23:1796+ e(np.float32)
F23:1797             smplx_pred["projection_consistency_max_error_px"] = np.asarray(
F23:1798                 projection_consistency_max_error_px, dtype=np.float32
F23:1799             )
F23:1800 np.savez(smplx_dir / f"{image_path.stem}_{bbox_id:02d}_smplx.npz", **smplx_pred)
F23:1801 if args.save_mesh:
F23:1802     save_obj(mesh, smpl_x.face, str(mesh_dir / f"{image_path.stem}_{bbox_id:02d}.obj
F23:1802+ "))
F23:1804     vis_img = render_mesh_fisheye(
F23:1805         vis_img,
F23:1806         mesh,
F23:1807         smpl_x.face,
F23:1808         {"focal": [1.0, 1.0], "princpt": [0.0, 0.0]},
F23:1809         fisheye_cam,
F23:1810         mesh_as_vertices=args.show_verts,
F23:1811         mode=args.mesh_mode,
F23:1812         proj_trans=render_proj_trans,
F23:1813     )
F23:1814 if wireframe_img is not None:
F23:1815     wireframe_img = render_mesh_fisheye(
F23:1816         wireframe_img,
F23:1817         mesh,
F23:1818         smpl_x.face,
F23:1819         {"focal": [1.0, 1.0], "princpt": [0.0, 0.0]},
F23:1820         fisheye_cam,
F23:1821         mesh_as_vertices=False,
F23:1822         mode="wireframe",
F23:1823         proj_trans=render_proj_trans,
F23:1824     )
F23:1825 if body_targets_px is not None:
F23:1826     mesh_joint_compare_image = draw_positionnet_smplx_2d_comparison(
F23:1827         wireframe_img,
F23:1828         body_targets_px,
F23:1829         body_projection_refined_px,
F23:1830     )
F23:1831     mesh_joint_compare_out = (
F23:1832         mesh_joint_compare_dir
F23:1833         / f"{image_path.stem}_{bbox_id:02d}_mesh_positionnet_smplx.jpg"
F23:1834     )
F23:1835     cv2.imwrite(
F23:1836         str(mesh_joint_compare_out),
F23:1837         rgb_to_bgr_u8(mesh_joint_compare_image),
F23:1838     )
F23:1839     saved_mesh_joint_compare_images.append(mesh_joint_compare_out)
F23:1840 if args.show_bbox and not args.no_detector:
F23:1841     if args.smplx_input_mode == "det_undistort_contiguous":
F23:1842         det_corners_undistorted = np.asarray(
F23:1843             [[x1, y1], [x2, y1], [x2, y2], [x1, y2]],

```

```

F23:1844         dtype=np.float32,
F23:1845     )
F23:1846     det_corners_original = undistorted_pixels_to_original(
F23:1847         det_corners_undistorted,
F23:1848         undistort_model,
F23:1849     )
F23:1850     det_corners_int = np rint(
F23:1851         det_corners_original
F23:1852     ).astype(np.int32)
F23:1853     cv2.polylines(
F23:1854         vis_img,
F23:1855         [det_corners_int],
F23:1856         isClosed=True,
F23:1857         color=(255, 0, 0),
F23:1858         thickness=3,
F23:1859         lineType=cv2.LINE_AA,
F23:1860     )
F23:1861     label_point = tuple(det_corners_int[0])
F23:1862 else:
F23:1863     pt1 = (int(round(x1)), int(round(y1)))
F23:1864     pt2 = (int(round(x2)), int(round(y2)))
F23:1865     cv2.rectangle(vis_img, pt1, pt2, (255, 0, 0), 3)
F23:1866     label_point = pt1
F23:1867     cv2.putText(
F23:1868         vis_img,
F23:1869         f"person {score:.2f}",
F23:1870         (label_point[0], max(25, label_point[1] - 10)),
F23:1871         cv2.FONT_HERSHEY_SIMPLEX,
F23:1872         0.8,
F23:1873         (255, 0, 0),
F23:1874         2,
F23:1875         cv2.LINE_AA,
F23:1876     )
F23:1877 meta = {
F23:1878     "image_path": str(image_path.resolve()),
F23:1879     "bbox_id": int(bbox_id),
F23:1880     "det_bbox_xyxy_score": np.asarray(det_box).astype(float).tolist(),
F23:1881     "bbox_processed_xywh": bbox.tolist(),
F23:1882     "img2bb_trans": img2bb_trans.tolist(),
F23:1883     "bb2img_trans": bb2img_trans.tolist(),
F23:1884     "fisheye_calibration": str(args.fisheye_calibration.resolve()),
F23:1885     "checkpoint_path": str(args.checkpoint_path.resolve()),
F23:1886     "config_path": str(args.config_path.resolve()),
F23:1887     "smplx_input_mode": args.smplx_input_mode,
F23:1888     "detector_coordinate_space": (
F23:1889         "global_undistorted"
F23:1890         if args.smplx_input_mode == "det_undistort_contiguous"
F23:1891         else "original_fisheye"
F23:1892     ),
F23:1893     "input_crop_mode": args.input_crop_mode,
F23:1894

```

```

F23:1895             "input_crop_xyxy": input_crop_xyxy if args.smplx_input_mode != "det_crop" else N
F23:1895+ one,
F23:1896             "input_coord_offset": input_offset.astype(float).tolist() if args.smplx_input_mo
F23:1896+ de != "det_crop" else [0.0, 0.0],
F23:1897             "det_tangent_sampling_bbox_xyxy": (
F23:1898                 sampling_bbox_xyxy.astype(float).tolist()
F23:1899                 if sampling_bbox_xyxy is not None
F23:1900                 else None
F23:1901             ),
F23:1902             "sampling_bbox_coordinate_space": (
F23:1903                 "global_undistorted"
F23:1904                 if args.smplx_input_mode == "det_undistort_contiguous"
F23:1905                 else (
F23:1906                     "original_fisheye"
F23:1907                     if sampling_bbox_xyxy is not None
F23:1908                     else None
F23:1909                 )
F23:1910             ),
F23:1911             "det_tangent_patch_grid_shape": (
F23:1912                 list(sampling_grid_pixels.shape)
F23:1913                 if sampling_grid_pixels is not None
F23:1914                 else None
F23:1915             ),
F23:1916             "det_tangent_patch_size": (
F23:1917                 float(args.det_tangent_patch_size)
F23:1918                 if args.smplx_input_mode == "det_tangent"
F23:1919                 else None
F23:1920             ),
F23:1921             "positionnet_2d_mapping": (
F23:1922                 "bilinear_tangent_patch_centers"
F23:1923                 if args.smplx_input_mode == "det_tangent"
F23:1924                 else (
F23:1925                     "continuous_undistorted_grid_then_brown_conrady_to_original"
F23:1926                     if args.smplx_input_mode == "det_undistort_contiguous"
F23:1927                     else "heatmap_to_full_image"
F23:1928                 )
F23:1929             ),
F23:1930             "global_undistort_alpha": (
F23:1931                 float(args.global_undistort_alpha)
F23:1932                 if args.smplx_input_mode == "det_undistort_contiguous"
F23:1933                 else None
F23:1934             ),
F23:1935             "contiguous_roi_mode": (
F23:1936                 args.contiguous_roi_mode
F23:1937                 if args.smplx_input_mode == "det_undistort_contiguous"
F23:1938                 else None
F23:1939             ),
F23:1940             "global_undistorted_new_k": (
F23:1941                 undistort_model["new_k"].astype(float).tolist()
F23:1942                 if args.smplx_input_mode == "det_undistort_contiguous"

```

```

F23:1943                 else None
F23:1944                 ),
F23:1945                 "camera_translation_network": network_translation.detach().cpu().numpy().astype(
F23:1945+ float).tolist(),
F23:1946                 "camera_translation_render": render_translation.detach().cpu().numpy().astype(float
F23:1946+ oat).tolist(),
F23:1947                 "global_orient_network": network_global_orient.detach().cpu().numpy().astype(float
F23:1947+ at).tolist(),
F23:1948                 "global_orient_render": render_global_orient.detach().cpu().numpy().astype(float
F23:1948+ ).tolist(),
F23:1949                 "global_orient_delta_matrix": global_orient_delta_matrix.detach().cpu().numpy().
F23:1949+ astype(float).tolist(),
F23:1950                 "global_orient_delta_degrees": float(global_orient_delta_degrees),
F23:1951                 "global_orient_mesh_consistency_max_error_m": (
F23:1952                     global_orient_mesh_consistency_max_error_m
F23:1953                 ),
F23:1954                 "global_orient_stage_mean_error_px": (
F23:1955                     global_orient_stage_mean_error_px
F23:1956                 ),
F23:1957                 "camera_rotation_delta": camera_rotation_delta.detach().cpu().numpy().astype(float
F23:1957+ at).tolist(),
F23:1958                 "camera_refined_from_positionnet": bool(camera_fit is not None),
F23:1959                 "camera_fit_mode": args.camera_fit_mode if camera_fit is not None else None,
F23:1960                 "projection_consistency_max_error_px": projection_consistency_max_error_px,
F23:1961             }
F23:1962         if camera_fit is not None:
F23:1963             meta["camera_fit_joint_indices"] = camera_fit["joint_indices"].astype(int).tolist()
F23:1963+ t()
F23:1964             meta["camera_fit_initial_mean_error_px"] = camera_fit["initial_mean_error_px"]
F23:1965             meta["camera_fit_refined_mean_error_px"] = camera_fit["refined_mean_error_px"]
F23:1966             (meta_dir / f"{image_path.stem}_{bbox_id:02d}.json").write_text(json.dumps(meta, indent=2) + "\n")
F23:1966+ ent=2) + "\n")
F23:1967             persons_projected += 1
F23:1968             total_persons += 1
F23:1969             torch.cuda.empty_cache()
F23:1971             out_img = overlay_dir / f"{image_path.stem}_det_mesh_overlay.jpg"
F23:1972             cv2.imwrite(str(out_img), rgb_to_bgr_u8(vis_img))
F23:1973             saved_images.append(out_img)
F23:1974             if wireframe_img is not None:
F23:1975                 wireframe_out = wireframe_dir / f"{image_path.stem}_det_mesh_wireframe.jpg"
F23:1976                 cv2.imwrite(str(wireframe_out), rgb_to_bgr_u8(wireframe_img))
F23:1977                 saved_wireframe_images.append(wireframe_out)
F23:1978             if video_writer is not None:
F23:1979                 video_writer.write(rgb_to_bgr_u8(vis_img))
F23:1980             records.append(**{"det_record", "persons_projected": int(persons_projected)})
F23:1982         if video_writer is not None:
F23:1983             video_writer.release()
F23:1984         save_contact_sheet(saved_images, output_dir / "contact_sheet.jpg", max_images=args.contact_sheet
F23:1984+ _limit)
F23:1985         if saved_wireframe_images:

```

```
F23:1986         save_contact_sheet(
F23:1987             saved_wireframe_images,
F23:1988             output_dir / "contact_sheet_wireframe.jpg",
F23:1989             max_images=args.contact_sheet_limit,
F23:1990         )
F23:1991     if saved_sampling_grid_images:
F23:1992         save_contact_sheet(
F23:1993             saved_sampling_grid_images,
F23:1994             output_dir / "contact_sheet_vit_sampling_grid.jpg",
F23:1995             max_images=args.contact_sheet_limit,
F23:1996         )
F23:1997     if saved_patch_atlas_images:
F23:1998         save_contact_sheet(
F23:1999             saved_patch_atlas_images,
F23:2000             output_dir / "contact_sheet_vit_patch_atlas.jpg",
F23:2001             max_images=args.contact_sheet_limit,
F23:2002         )
F23:2003     if saved_contiguous_patch_grid_images:
F23:2004         save_contact_sheet(
F23:2005             saved_contiguous_patch_grid_images,
F23:2006             output_dir / "contact_sheet_vit_patch_atlas_grid.jpg",
F23:2007             max_images=args.contact_sheet_limit,
F23:2008         )
F23:2009     if saved_undistorted_images:
F23:2010         save_contact_sheet(
F23:2011             saved_undistorted_images,
F23:2012             output_dir / "contact_sheet_global_undistorted.jpg",
F23:2013             max_images=args.contact_sheet_limit,
F23:2014         )
F23:2015     if saved_undistorted_detection_images:
F23:2016         save_contact_sheet(
F23:2017             saved_undistorted_detection_images,
F23:2018             output_dir / "contact_sheet_global_undistorted_detection.jpg",
F23:2019             max_images=args.contact_sheet_limit,
F23:2020         )
F23:2021     if saved_positionnet_images:
F23:2022         save_contact_sheet(
F23:2023             saved_positionnet_images,
F23:2024             output_dir / "contact_sheet_positionnet.jpg",
F23:2025             max_images=args.contact_sheet_limit,
F23:2026         )
F23:2027     if saved_camera_fit_images:
F23:2028         save_contact_sheet(
F23:2029             saved_camera_fit_images,
F23:2030             output_dir / "contact_sheet_camera_fit.jpg",
F23:2031             max_images=args.contact_sheet_limit,
F23:2032         )
F23:2033     if saved_joint_compare_images:
F23:2034         save_contact_sheet(
F23:2035             saved_joint_compare_images,
```



```

F23:2036         output_dir / "contact_sheet_positionnet_vs_smplx_2d.jpg",
F23:2037         max_images=args.contact_sheet_limit,
F23:2038     )
F23:2039     if saved_mesh_joint_compare_images:
F23:2040         save_contact_sheet(
F23:2041             saved_mesh_joint_compare_images,
F23:2042             output_dir / "contact_sheet_mesh_positionnet_smplx_2d.jpg",
F23:2043             max_images=args.contact_sheet_limit,
F23:2044         )
F23:2046     summary = {
F23:2047         "image_dir": str(args.image_dir.resolve()),
F23:2048         "image_glob": args.image_glob,
F23:2049         "processed_images": int(len(image_paths)),
F23:2050         "images_without_detection": int(no_detection),
F23:2051         "projected_persons": int(total_persons),
F23:2052         "multi_person": bool(args.multi_person),
F23:2053         "detector_enabled": not args.no_detector,
F23:2054         "score_thr": float(args.score_thr),
F23:2055         "bbox_thr": float(args.bbox_thr),
F23:2056         "smplx_input_mode": args.smplx_input_mode,
F23:2057         "det_tangent_expand": float(args.det_tangent_expand),
F23:2058         "det_tangent_patch_size": float(args.det_tangent_patch_size),
F23:2059         "global_undistort_alpha": float(args.global_undistort_alpha),
F23:2060         "contiguous_roi_mode": (
F23:2061             args.contiguous_roi_mode
F23:2062             if args.smplx_input_mode == "det_undistort_contiguous"
F23:2063             else None
F23:2064         ),
F23:2065         "global_undistorted_source_k": (
F23:2066             undistort_model["source_k"].astype(float).tolist()
F23:2067             if undistort_model is not None
F23:2068             else None
F23:2069         ),
F23:2070         "global_undistorted_new_k": (
F23:2071             undistort_model["new_k"].astype(float).tolist()
F23:2072             if undistort_model is not None
F23:2073             else None
F23:2074         ),
F23:2075         "global_undistorted_distortion": (
F23:2076             undistort_model["distortion"].astype(float).tolist()
F23:2077             if undistort_model is not None
F23:2078             else None
F23:2079         ),
F23:2080         "global_undistorted_valid_roi": (
F23:2081             list(undistort_model["valid_roi"])
F23:2082             if undistort_model is not None
F23:2083             else None
F23:2084         ),
F23:2085         "input_crop_mode": args.input_crop_mode,
F23:2086         "horizontal_crop_pixels": int(args.horizontal_crop_pixels),

```

```

F23:2087         "camera_refined_from_positionnet": bool(args.refine_camera_from_positionnet),
F23:2088         "camera_fit_mode": args.camera_fit_mode,
F23:2089         "camera_fit_iters": int(args.camera_fit_iters),
F23:2090         "camera_fit_lr": float(args.camera_fit_lr),
F23:2091         "save_wireframe_overlay": bool(args.save_wireframe_overlay),
F23:2092         "projection_consistency_max_error_px": (
F23:2093             float(max(projection_consistency_errors))
F23:2094             if projection_consistency_errors
F23:2095             else None
F23:2096     ),
F23:2097     "output_dir": str(output_dir.resolve()),
F23:2098     "fisheye_calibration": str(args.fisheye_calibration.resolve()),
F23:2099     "checkpoint_path": str(args.checkpoint_path.resolve()),
F23:2100     "det_config": str(args.det_config.resolve()),
F23:2101     "det_checkpoint": str(args.det_checkpoint.resolve()),
F23:2102     "elapsed_sec": float(time.time() - start_time),
F23:2103 }
F23:2104 (output_dir / "manifest.json").write_text(json.dumps(summary, indent=2) + "\n")
F23:2105 (output_dir / "records.json").write_text(json.dumps(records, indent=2) + "\n")
F23:2106 print(json.dumps(summary, indent=2))
F23:2107 sys.stdout.flush()
F23:2108 sys.stderr.flush()
F23:2109 os._exit(0)
F23:2112 if __name__ == "__main__":
F23:2113     main()
F24:0001 """Package a verified run, its three-stage results and reconstruction inputs.
F24:0003 This exports a new bundle; it does not overwrite or impersonate the historical
F24:0004 session_hand6 ZIP. Network weights and segmentation probability caches stay out.
F24:0005 """
F24:0006 import argparse
F24:0007 import hashlib
F24:0008 import json
F24:0009 from pathlib import Path
F24:0010 import zipfile
F24:0011 from egosmplx_pipeline.io import read, sha
F24:0012 from egosmplx_pipeline.configuration import PACKAGE_ROOT
F24:0015 def package(run, destination):
F24:0016     run, destination = Path(run).resolve(), Path(destination).resolve()
F24:0017     if read(run / 'run_state.json')['state'] != 'complete':
F24:0018         raise ValueError('Only complete, verified runs can be packaged')
F24:0019     if destination.exists():
F24:0020         raise FileExistsError(destination)
F24:0021     sources = {}
F24:0022     for path in sorted((run / 'frames').glob('*/*validation.json')):
F24:0023         report = read(path)
F24:0024         if not report.get('passed') or report.get('pipeline_profile') != 'session_hand6_full_pipeline':
F24:0024+ e_20260926_v1':
F24:0025             raise ValueError('Frame is not a verified reference-profile result')
F24:0026         frame = path.parent
F24:0027         for name, digest in report['artifact_sha256'].items():

```

```

F24:0028         candidate = (frame / name).resolve()
F24:0029         if frame not in candidate.parents or sha(candidate) != digest:
F24:0030             raise ValueError('Frame artifact changed: ' + name)
F24:0031         identifier = report['id']
F24:0032         if frame.name != identifier:
F24:0033             raise ValueError('Frame ID mismatch')
F24:0034         row = read(run / 'jobs' / (identifier + '.json'))['row']
F24:0035         if sha(row['image']) != report['input_image_sha256'] or sha(row['calibration']) != report['c
F24:0035+ alibration_sha256']:
F24:0036             raise ValueError('Original image or calibration changed')
F24:0037         prefix = 'frames/' + identifier + '/'
F24:0038         sources[prefix + 'input' + Path(row['image']).suffix.lower()] = Path(row['image'])
F24:0039         sources[prefix + 'calibration.json'] = Path(row['calibration'])
F24:0040         sources[prefix + 'automatic_observations/sapiens2_keypoints.json'] = Path(row['predictions']
F24:0040+ ['sapiens'])
F24:0041         for stage in ['raw', 'body', 'fused']:
F24:0042             for asset in sorted((frame / stage).iterdir()):
F24:0043                 if asset.is_file():
F24:0044                     sources[prefix + stage + '/' + asset.name] = asset
F24:0045                 sources[prefix + 'validation.json'] = path
F24:0046         if len(list((run / 'frames').glob('*/*validation.json')) != read(run / 'summary.json')['frame_co
F24:0046+ unt']):
F24:0047             raise ValueError('Run frame count differs from summary')
F24:0048         for name in ['index.html', 'contact_sheet.jpg']:
F24:0049             sources[prefix + name] = run / name
F24:0050         for path in run.glob('*_comparison.jpg'):
F24:0051             sources[prefix + path.name] = path
F24:0052         sources['summary.json'] = run / 'summary.json'
F24:0053         sources['REFERENCE_PIPELINE_zh.md'] = PACKAGE_ROOT / 'docs/REFERENCE_PIPELINE_zh.md'
F24:0054         for path in sorted((PACKAGE_ROOT / 'egosmplx_pipeline').rglob '*.py')):
F24:0055             sources[prefix + str(path.relative_to(PACKAGE_ROOT))] = path
F24:0056         destination.parent.mkdir(parents=True, exist_ok=True)
F24:0057         manifest = {}
F24:0058         # Exclusive creation prevents an existing historical bundle from being replaced.
F24:0059         with zipfile.ZipFile(destination, 'x', zipfile.ZIP_DEFLATED, compresslevel=6) as archive:
F24:0060             for name, path in sorted(sources.items()):
F24:0061                 data = path.read_bytes()
F24:0062                 archive.writestr(name, data)
F24:0063                 manifest[name] = dict(bytes=len(data), sha256=hashlib.sha256(data).hexdigest())
F24:0064             archive.writestr('ARCHIVE_MANIFEST.json', json.dumps(manifest, indent=2))
F24:0065         with zipfile.ZipFile(destination) as archive:
F24:0066             if archive.testzip() is not None:
F24:0067                 raise RuntimeError('Exported ZIP failed CRC validation')
F24:0068         return dict(path=str(destination), sha256=sha(destination), manifest_members=len(manifest),
F24:0069             contains_weights=False, contains_full_segmentation_probabilities=False)
F24:0072 def main():
F24:0073     parser = argparse.ArgumentParser(description=__doc__)
F24:0074     parser.add_argument('--run', type=Path, required=True)
F24:0075     parser.add_argument('--archive', type=Path, required=True)
F24:0076     args = parser.parse_args()

```

```

F24:0077     print(json.dumps(package(args.run, args.archive), indent=2))
F24:0080 if __name__ == '__main__':
F24:0081     main()
F25:0001 """Exact algorithm profile of the session_hand6 result bundle dated 2026-09-26."""
F26:0001 """Fixed-shape SMPL-X wrist and projected seam compatibility refinement."""
F26:0002 from pathlib import Path
F26:0003 import os
F26:0004 import sys,copy,argparse,time
F26:0005 import numpy as np
F26:0006 import torch
F26:0007 import torch.nn.functional as F
F26:0008 OUT=Path(os.environ['EGOSMPLX_REFERENCE_ROOT'])/'contour'
F26:0009 BASE=OUT.parent/'baseline'
F26:0011 from egosmplx_pipeline.reference.support import (read,write,load,arr,geometry,constraints,PARAMS,FIX
F26:0011+ ED,SIDES,JI,MAX_DELTA_DEG,BODY_POSE_NAMES,JOINT_MAPPING,smpl_x,FishEyeCameraCalibrated)
F26:0012 from egosmplx_pipeline.reference.support import native_projection
F26:0013 from egosmplx_pipeline.reference.silhouette import targets,prepare,loss as silhouette_loss,metrics a
F26:0013+ s silhouette_metrics,PREV
F26:0015 def main():
F26:0016     ap=argparse.ArgumentParser():ap.add_argument('--steps',type=int,default=500):ap.add_argument('--ind
F26:0016+ ices',type=int,nargs='*'):ap.add_argument('--active-projection',action='store_true'):a=ap.parse_args
F26:0016+ ()
F26:0017     torch.backends.cuda.matmul.allow_tf32=False;torch.backends.cudnn.allow_tf32=False
F26:0018     torch.set_num_threads(2);dev='cuda' if torch.cuda.is_available() else 'cpu';layer=copy.deepcopy(smp
F26:0018+ l_x.layer['neutral']).to(dev).eval();layer.requires_grad_(False)
F26:0019     t=lambda x:torch.as_tensor(x, dtype=torch.float32, device=dev)
F26:0020     rows=read(BASE/'automatic_input_manifest.json')['frames']
F26:0021     for ii,row in enumerate(rows):
F26:0022         if a.indices and ii not in a.indices:continue
F26:0023         dst=OUT/('active_frames' if a.active_projection else 'frames')/row['condition']/row['camera'];dst.
F26:0023+ mkdir(parents=True,exist_ok=True)
F26:0024         if (dst/'refinement.json').exists():continue
F26:0025         bd=BASE/'frames'/row['condition']/row['camera'];p=load(PREV/'final_frames'/row['condition']/row['c
F26:0025+ amera']/'body_params.npz');source=load(row['source_npz']);rigid=load(row['rigid_npz']);w=load(row['w
F26:0025+ ilor_npz']);oldmesh=load(bd/'hybrid_mesh.npz');matches=read(bd/'recipe.json')['matches'];cam=FishEye
F26:0025+ CameraCalibrated(row['calibration'])
F26:0026         lookup={q['name']:q for q in read(row['sapiens_json'])['keypoints']}
F26:0027         active=list(MAX_DELTA_DEG);ai=torch.tensor([BODY_POSE_NAMES.index(n) for n in active],device=dev);
F26:0027+ caps=t(np.deg2rad([MAX_DELTA_DEG[n] for n in active]))
F26:0028         b0=t(rigid['body_pose']).reshape(21,3);g0=t(rigid['global_orient']).reshape(3);tr0=t(rigid['transl
F26:0028+ ']).reshape(3)
F26:0029         init_path=OUT/'frames'/row['condition']/row['camera']/'body_params.npz'
F26:0030         initp=p
F26:0031         delta=(t(initp['body_pose']).reshape(21,3)-b0)[ai].clone().requires_grad_();gd=(t(initp['global_or
F26:0031+ ient']).reshape(3)-g0).clone().requires_grad_();tr=t(initp['transl']).reshape(3).clone().requires_gr
F26:0031+ ad_()
F26:0032         names=[n for n in JOINT_MAPPING if lookup[n]['score']>=.3 and np.isfinite([lookup[n]['x'],lookup[n]
F26:0032+ ]['y']]).all() and 0<=lookup[n]['x']<1280 and 0<=lookup[n]['y']<720]
F26:0033         bi=torch.tensor([JI[JOINT_MAPPING[n]] for n in names],device=dev);bt=t([lookup[n]['x'],lookup[n]
F26:0033+ 'y']] for n in names);bw=t([min(1,lookup[n]['score'])**2 for n in names]);bw/=bw.mean()

```

```

F26:0034 ix=torch.tensor([12+SIDES.index(s) for s in matches],device=dev);goals=t([w['joints_2d'][m['candid
F26:0034+ ate'],0] for s,m in matches.items()))
F26:0035 _,srcj=geometry(source);_,rigj=geometry(rigid);floor=t(np.maximum(np.linalg.norm(srcj[[12,13]],axi
F26:0035+ s=1),np.linalg.norm(rigj[[12,13]],axis=1))*.85)
F26:0036 bodylimit=float(np.sqrt(np.mean(np.sum((cam.world2camera(srcj.astype(np.float64))[bi.cpu().numpy()
F26:0036+ ]-arr(bt))*2,axis=1))))+5
F26:0037 sv,sj=geometry(p);initial_werr=np.linalg.norm(cam.world2camera(sj.astype(np.float64))[ix.cpu().num
F26:0037+ py()-arr(goals),axis=1)
F26:0038 loopids=[];loopgoals=[]
F26:0039 for s,m in matches.items():
F26:0040 ids=oldmesh[s+'_local_to_mesh'];bv=oldmesh[s+'_wrist_loop_local'];loopids.append(torch.tensor(ids
F26:0040+ [bv],device=dev))
F26:0041 uv= native_projection(w['vertices_3d_local'][m['candidate']] + w['cam_t'][m['candidate']]);uv=uv[bv]
F26:0041+ ;loopgoals.append(t(uv-uv.mean(0)))
F26:0042 sil_data=prepare(targets(row,p,matches,w,cam,dst),dev)
F26:0043 sil_before=silhouette_metrics(sv,cam,sil_data)
F26:0044 def params():
F26:0045 pp={k:t(p[k]).reshape(1,-1) for k in PARAMS};body=b0.clone();body[ai]+=delta;pp.update(body_pose=
F26:0045+ body.reshape(1,-1),global_orient=(g0+gd).reshape(1,3));return pp
F26:0046 def forward():
F26:0047 o=layer(**params());v=o.vertices[0]+tr;j=o.joints[0,smpl_x.joint_idx]+tr;uv=cam.world2camera_pyto
F26:0047+ rch(j);return v,j,uv
F26:0048 def feasible(v,j,uv):
F26:0049 err=(uv[ix]-goals).norm(dim=1)
F26:0050 return bool(v[:,2].min()>=.020001 and (j[[12,13]].norm(dim=1)>=floor+1e-6).all() and torch.sqrt((
F26:0050+ uv[bi]-bt).square().sum(1).mean())<=bodylimit-1e-4 and (err<=t(initial_werr)+3).all())
F26:0051 def objective(v,j,uv):
F26:0052 body=(F.smooth_l1_loss((uv[bi]-bt)/t([1280,720]),torch.zeros_like(bt),beta=.003,reduction='none')
F26:0052+ .mean(1)*bw).mean()
F26:0053 wrists=F.smooth_l1_loss((uv[ix]-goals)/t([1280,720]),torch.zeros_like(goals),beta=.003)
F26:0054 seam=[]
F26:0055 for ids,target in zip(loopids,loopgoals):
F26:0056 q=cam.world2camera_pytorch(v[ids]);q=q-q.mean(0);radius=q.square().sum(1).mean().sqrt();rr=targe
F26:0056+ t.square().sum(1).mean().sqrt();seam.append(torch.log(radius/rr.clamp_min(1)).square())
F26:0057 shape=torch.stack(seam).mean()
F26:0058 priors=.0008*(delta/caps[:,None]).square().mean()+.0002*((tr-tr0)/t([.10,.10,.18])).square().mean
F26:0058+ ()+.0004*(gd/np.deg2rad(8)).square().mean()
F26:0059 depth=.02*F.relu((.03-v[:,2].min())/(.02)).square();guard=.002*F.relu((floor-j[[12,13]].norm(dim=1)
F26:0059+ )/.05).square().mean()
F26:0060 return body+4*wrists+.0008*shape+priors+depth+guard+.08*silhouette_loss(v,cam,sil_data)
F26:0061 opt=torch.optim.Adam([{'params':[delta],'lr':.0015},{'params':[gd],'lr':.0005},{'params':[tr],'lr'
F26:0061+ :.0008}])
F26:0062 with torch.no_grad():v,j,uv=forward();bestloss=float(objective(v,j,uv));best=[x.detach().clone() f
F26:0062+ or x in [delta,gd,tr]]
F26:0063 rejected=0;start=time.time()
F26:0064 for step in range(a.steps):
F26:0065 opt.zero_grad();v,j,uv=forward();loss=objective(v,j,uv);loss.backward()
F26:0066 previous=[x.detach().clone() for x in [delta,gd,tr]];opt.step()
F26:0067 if a.active_projection:
F26:0068 # Project infeasible Adam proposals back onto nonlinear constraint surfaces.

```

```

F26:0069     for project_step in range(12):
F26:0070         with torch.no_grad():
F26:0071             delta.mul_(torch.minimum(torch.ones_like(caps),caps/delta.norm(dim=1).clamp_min(1e-8))[:,None])
F26:0071+ ) ;gd.mul_(torch.clamp(t(np.deg2rad(8))/gd.norm().clamp_min(1e-8),max=1));tr[2].clamp_(.05,3)
F26:0072         pv,pj,pu=forward()
F26:0073         br=(pu[bi]-bt).square().sum(1).mean().sqrt()
F26:0074         violations=torch.cat([(br-bodylimit+.02).reshape(1)/5,(.02001-pv[:,2].min()).reshape(1)/.02,(f1
F26:0074+ oor+1e-5-pj[[12,13]].norm(dim=1))/.05,((pu[ix]-goals).norm(dim=1)-t(initial_werr)-2.99)/5])
F26:0075         g=violations.max()
F26:0076         if float(g)<=0:break
F26:0077         grads=torch.autograd.grad(g,[delta,gd,tr]);scales=[.0015,.0005,.0008]
F26:0078         denom=sum((grad.square().sum()*scale**2) for grad,scale in zip(grads,scales)).clamp_min(1e-12)
F26:0079         with torch.no_grad():
F26:0080             for x,grad,scale in zip([delta,gd,tr],grads,scales):x.sub_(1.05*g.detach()*grad*scale**2/denom
F26:0080+ )
F26:0081         with torch.no_grad():
F26:0082             delta.mul_(torch.minimum(torch.ones_like(caps),caps/delta.norm(dim=1).clamp_min(1e-8))[:,None]);
F26:0082+ gd.mul_(torch.clamp(t(np.deg2rad(8))/gd.norm().clamp_min(1e-8),max=1));tr[2].clamp_(.05,3)
F26:0083         ok=False
F26:0084         for trial in range(10):
F26:0085             v,j,uv=forward()
F26:0086             if feasible(v,j,uv):ok=True;break
F26:0087             for x,prev in zip([delta,gd,tr],previous):x.copy_((x+prev)*.5)
F26:0088             if not ok:
F26:0089                 rejected+=1
F26:0090             for x,prev in zip([delta,gd,tr],previous):x.copy_(prev)
F26:0091             v,j,uv=forward()
F26:0092             value=float(objective(v,j,uv))
F26:0093             if ok and value<bestloss:bestloss=value;best=[x.detach().clone() for x in [delta,gd,tr]]
F26:0094             if step%100==0:print(row['id'],step,'wrist',arr((uv[ix]-goals).norm(dim=1)).round(2).tolist(),'re
F26:0094+ jects',rejected,flush=True)
F26:0095         with torch.no_grad():
F26:0096             for x,b in zip([delta,gd,tr],best):x.copy_(b)
F26:0097             pp=params();saved={k:x.copy() for k,x in p.items()}
F26:0098             for k,x in pp.items():saved[k]=arr(x).reshape(p[k].shape)
F26:0099             saved['transl']=arr(tr).reshape(p['transl'].shape)
F26:0100             candidate={k:x.copy() for k,x in saved.items()};cpu_alpha=1.
F26:0101             for trial in range(22):
F26:0102                 for k in ['body_pose','global_orient','transl']:saved[k]=p[k]+cpu_alpha*(candidate[k]-p[k])
F26:0103                 try:acceptance=constraints(saved,source,rigid,cam,lookup);break
F26:0104                 except AssertionError:cpu_alpha*=.9
F26:0105             else:
F26:0106                 saved={k:x.copy() for k,x in p.items()};cpu_alpha=0.;acceptance=constraints(saved,source,rigid,ca
F26:0106+ m,lookup)
F26:0107                 vv,jj=geometry(saved);local=jj-saved['transl'].reshape(3);pixels=cam.world2camera(jj.astype(np.flo
F26:0107+ at64)).astype(np.float32)
F26:0108                 saved.update(smplx_mesh_cam=vv,smplx_joint_cam=jj,smplx_joint_img=pixels,smplx_body_joints_3d_loca
F26:0108+ l=local[:25],smplx_body_joints_3d_render_cam=jj[:25],smplx_body_joints_2d_render_cam=pixels[:25],smp
F26:0108+ lx_joint_cam_root_relative=jj-jj[:1])
F26:0109                 if 'smplx_body_joints_3d_cam' in saved:saved['smplx_body_joints_3d_cam']=jj[:25]

```

```

F26:0110 if 'smp1x_body_joints_2d' in saved:saved['smp1x_body_joints_2d']=pixels[:25]
F26:0111 for k in FIXED+['left_hand_pose','right_hand_pose']:assert np.array_equal(saved[k],p[k])
F26:0112 # Fixed beta kinematic limb lengths (selected joints) are conserved under pose.
F26:0113 lengths=lambda j:np.array([np.linalg.norm(j[a]-j[b]) for a,b in [(8,10),(10,12),(9,11),(11,13)])])
F26:0114 boneerr=float(np.max(np.abs(lengths(jj)-lengths(sj)))):assert boneerr<1e-5
F26:0115 np.savez_compressed(dst/'body_params.npz',**saved)
F26:0116 report=dict(id=row['id'],active_constraint_projection=a.active_projection,steps=a.steps,cpu_feasib
F26:0116+ ility_step_fraction=cpu_alpha,elapsed_seconds=time.time()-start>manual_inputs=False,original_hard_bo
F26:0116+ unds_preserved=True,body_shape_fixed=True,bone_length_max_change_m=boneerr,wrist_before_px=initial_w
F26:0116+ err.tolist(),wrist_after_px=np.linalg.norm(cam.world2camera(jj.astype(np.float64))[ix.cpu().numpy()])
F26:0116+ -arr(goals),axis=1).tolist(),matches=matches,loss=bestloss,rejected_steps=rejected,acceptance=accept
F26:0116+ ance)
F26:0117 report.update(silhouette_before=sil_before,silhouette_after=silhouette_metrics(vv,cam,sil_data),si
F26:0117+ lhouette_weight=.08,previous_identity_preserved=True)
F26:0118 write(dst/'refinement.json',report):print('DONE',row['id'],report['wrist_after_px'],flush=True)
F26:0119 if __name__=='__main__':main()
F27:0001 """Fixed-shape SMPL-X wrist and projected seam compatibility refinement."""
F27:0002 from pathlib import Path
F27:0003 import os
F27:0004 import sys,copy,argparse,time
F27:0005 import numpy as np
F27:0006 import torch
F27:0007 import torch.nn.functional as F
F27:0008 OUT=Path(os.environ['EGOSMPLX_REFERENCE_ROOT'])/'contour'
F27:0009 BASE=OUT.parent/'baseline'
F27:0011 from egosmplx_pipeline.reference.support import (read,write,load,arr,geometry,constraints,PARAMS,FIX
F27:0011+ ED,SIDES,JI,MAX_DELTA_DEG,BODY_POSE_NAMES,JOINT_MAPPING,smp1_x,FishEyeCameraCalibrated)
F27:0012 from egosmplx_pipeline.reference.support import native_projection
F27:0013 from egosmplx_pipeline.reference.silhouette import targets,prepare,loss as silhouette_loss,metrics a
F27:0013+ s silhouette_metrics,PREV
F27:0015 def main():
F27:0016 ap=argparse.ArgumentParser():ap.add_argument('--steps',type=int,default=500):ap.add_argument('--ind
F27:0016+ ices',type=int,nargs='*'):ap.add_argument('--active-projection',action='store_true'):a=ap.parse_args
F27:0016+ ()
F27:0017 torch.backends.cuda.matmul.allow_tf32=False:torch.backends.cudnn.allow_tf32=False
F27:0018 torch.set_num_threads(2):dev='cuda' if torch.cuda.is_available() else 'cpu':layer=copy.deepcopy(smp
F27:0018+ l_x.layer['neutral']).to(dev).eval():layer.requires_grad_(False)
F27:0019 t=lambda x:torch.as_tensor(x,device=torch.float32,device=dev)
F27:0020 rows=read(BASE/'automatic_input_manifest.json')['frames']
F27:0021 for ii,row in enumerate(rows):
F27:0022 if a.indices and ii not in a.indices:continue
F27:0023 dst=OUT/'strong_frames'/row['condition']/row['camera']:dst.mkdir(parents=True,exist_ok=True)
F27:0024 if (dst/'refinement.json').exists():continue
F27:0025 bd=BASE/'frames'/row['condition']/row['camera']:p=load(PREV/'final_frames'/row['condition']/row['c
F27:0025+ amera']/body_params.npz):source=load(row['source_npz']):rigid=load(row['rigid_npz']):w=load(row['w
F27:0025+ ilor_npz']):oldmesh=load(bd/'hybrid_mesh.npz'):matches=read(bd/'recipe.json')['matches']:cam=FishEye
F27:0025+ CameraCalibrated(row['calibration'])
F27:0026 lookup={q['name']:q for q in read(row['sapiens_json'])['keypoints']}
F27:0027 active=list(MAX_DELTA_DEG):ai=torch.tensor([BODY_POSE_NAMES.index(n) for n in active],device=dev):
F27:0027+ caps=t(np.deg2rad([MAX_DELTA_DEG[n] for n in active]))

```

```

F27:0028     b0=t(rigid['body_pose']).reshape(21,3);g0=t(rigid['global_orient']).reshape(3);tr0=t(rigid['transl
F27:0028+   ']).reshape(3)
F27:0029     init_path=OUT/'frames'/row['condition']/row['camera']/'body_params.npz'
F27:0030     initp=p
F27:0031     delta=(t(initp['body_pose']).reshape(21,3)-b0)[ai].clone().requires_grad_();gd=(t(initp['global_or
F27:0031+   ient']).reshape(3)-g0).clone().requires_grad_();tr=t(initp['transl']).reshape(3).clone().requires_gr
F27:0031+   ad_()
F27:0032     names=[n for n in JOINT_MAPPING if lookup[n]['score']>=.3 and np.isfinite([lookup[n]['x'],lookup[n]
F27:0032+   ]['y'])].all() and 0<=lookup[n]['x']<1280 and 0<=lookup[n]['y']<720]
F27:0033     bi=torch.tensor([JI[JOINT_MAPPING[n]] for n in names],device=dev);bt=t([lookup[n]['x'],lookup[n]
F27:0033+   'y']) for n in names];bw=t([min(1,lookup[n]['score'])**2 for n in names]);bw/=bw.mean()
F27:0034     ix=torch.tensor([12+SIDES.index(s) for s in matches],device=dev);goals=t([w['joints_2d'][m['candid
F27:0034+   ate'],0] for s,m in matches.items()])
F27:0035     _,srcj=geometry(source);_,rigj=geometry(rigid);floor=t(np.maximum(np.linalg.norm(srcj[[12,13]],axi
F27:0035+   s=1),np.linalg.norm(rigj[[12,13]],axis=1))*.85)
F27:0036     bodylimit=float(np.sqrt(np.mean(np.sum((cam.world2camera(srcj.astype(np.float64))[bi.cpu().numpy()
F27:0036+   ]-arr(bt))**2,axis=1))))+5
F27:0037     sv,sj=geometry(p);initial_werr=np.linalg.norm(cam.world2camera(sj.astype(np.float64))[ix.cpu().num
F27:0037+   py()-arr(goals),axis=1)
F27:0038     loopids=[];loopgoals=[]
F27:0039     for s,m in matches.items():
F27:0040         ids=oldmesh[s+'_local_to_mesh'];bv=oldmesh[s+'_wrist_loop_local'];loopids.append(torch.tensor(ids
F27:0040+   [bv],device=dev))
F27:0041         uv= native_projection(w['vertices_3d_local'][m['candidate']] + w['cam_t'][m['candidate']]);uv=uv[bv]
F27:0041+   ;loopgoals.append(t(uv-uv.mean(0)))
F27:0042     sil_data=targets(row,p,matches,w,cam,dst)
F27:0043     for sd in sil_data:
F27:0044         side=sd["side"];sd["ids"]=np.unique(np.r_[sd["ids"],oldmesh[side+"_local_to_mesh"][oldmesh[side+
F27:0044+   "_wrist_loop_local"]]))
F27:0045     sil_data=prepare(sil_data,dev)
F27:0046     sil_before=silhouette_metrics(sv,cam,sil_data)
F27:0047     def params():
F27:0048         pp={k:t(p[k]).reshape(1,-1) for k in PARAMS};body=b0.clone();body[ai]+=delta;pp.update(body_pose=
F27:0048+   body.reshape(1,-1),global_orient=(g0+gd).reshape(1,3));return pp
F27:0049     def forward():
F27:0050         o=layer(**params());v=o.vertices[0]+tr;j=o.joints[0,smpl_x.joint_idx]+tr;uv=cam.world2camera_pyto
F27:0050+   rch(j);return v,j,uv
F27:0051     def feasible(v,j,uv):
F27:0052         err=(uv[ix]-goals).norm(dim=1)
F27:0053         return bool(v[:,2].min())>=.020001 and (j[[12,13]].norm(dim=1)>=floor+1e-6).all() and torch.sqrt((
F27:0053+   uv[bi]-bt).square().sum(1).mean())<=bodylimit-1e-4 and (err<=t(initial_werr)+3).all())
F27:0054     def objective(v,j,uv):
F27:0055         body=(F.smooth_l1_loss((uv[bi]-bt)/t([1280,720]),torch.zeros_like(bt),beta=.003,reduction='none')
F27:0055+   .mean(1)*bw).mean()
F27:0056         wrists=F.smooth_l1_loss((uv[ix]-goals)/t([1280,720]),torch.zeros_like(goals),beta=.003)
F27:0057         seam=[]
F27:0058         for ids,target in zip(loopids,loopgoals):
F27:0059             q=cam.world2camera_pytorch(v[ids]);q=q-q.mean(0);radius=q.square().sum(1).mean().sqrt();rr=targe
F27:0059+   t.square().sum(1).mean().sqrt();seam.append(torch.log(radius/rr.clamp_min(1)).square())
F27:0060         shape=torch.stack(seam).mean()

```



```

F27:0061     priors=.0008*(delta/caps[:,None]).square().mean()+.0002*((tr-tr0)/t([.10,.10,.18])).square().mean
F27:0061+   )+.0004*(gd/np.deg2rad(8)).square().mean()
F27:0062     depth=.02*F.relu((.03-v[:,2].min())/0.02).square();guard=.002*F.relu((f1oor-j[[12,13]].norm(dim=1)
F27:0062+   )/.05).square().mean()
F27:0063     return body+4*wrists+.0008*shape+priors+depth+guard+.4*silhouette_loss(v,cam,sil_data)
F27:0064     opt=torch.optim.Adam([{'params':[delta],'lr':.0015},{'params':[gd],'lr':.0005},{'params':[tr],'lr'
F27:0064+   :.0008}])
F27:0065     with torch.no_grad():v,j,uv=forward();bestloss=float(objective(v,j,uv));best=[x.detach().clone() f
F27:0065+   or x in [delta,gd,tr]]
F27:0066     rejected=0;start=time.time()
F27:0067     for step in range(a.steps):
F27:0068         opt.zero_grad();v,j,uv=forward();loss=objective(v,j,uv);loss.backward()
F27:0069         previous=[x.detach().clone() for x in [delta,gd,tr]];opt.step()
F27:0070         if a.active_projection:
F27:0071             # Project infeasible Adam proposals back onto nonlinear constraint surfaces.
F27:0072             for project_step in range(12):
F27:0073                 with torch.no_grad():
F27:0074                     delta.mul_(torch.minimum(torch.ones_like(caps),caps/delta.norm(dim=1).clamp_min(1e-8))[:,None]
F27:0074+   );gd.mul_(torch.clamp(t(np.deg2rad(8))/gd.norm().clamp_min(1e-8),max=1));tr[2].clamp_(.05,3)
F27:0075                     pv,pj,pu=forward()
F27:0076                     br=(pu[bi]-bt).square().sum(1).mean().sqrt()
F27:0077                     violations=torch.cat([(br-bodylimit+.02).reshape(1)/5,(.02001-pv[:,2].min()).reshape(1)/.02,(f1
F27:0077+   oor+1e-5-pj[[12,13]].norm(dim=1))/0.05,((pu[ix]-goals).norm(dim=1)-t(initial_werr)-2.99)/5])
F27:0078                     g=violations.max()
F27:0079                     if float(g)<=0:break
F27:0080                     grads=torch.autograd.grad(g,[delta,gd,tr]);scales=[.0015,.0005,.0008]
F27:0081                     denom=sum((grad.square().sum()*scale**2) for grad,scale in zip(grads,scales)).clamp_min(1e-12)
F27:0082                     with torch.no_grad():
F27:0083                         for x,grad,scale in zip([delta,gd,tr],grads,scales):x.sub_(1.05*g.detach()*grad*scale**2/denom
F27:0083+   )
F27:0084                 with torch.no_grad():
F27:0085                     delta.mul_(torch.minimum(torch.ones_like(caps),caps/delta.norm(dim=1).clamp_min(1e-8))[:,None]);
F27:0085+   gd.mul_(torch.clamp(t(np.deg2rad(8))/gd.norm().clamp_min(1e-8),max=1));tr[2].clamp_(.05,3)
F27:0086                     ok=False
F27:0087                     for trial in range(10):
F27:0088                         v,j,uv=forward()
F27:0089                         if feasible(v,j,uv):ok=True;break
F27:0090                         for x,prev in zip([delta,gd,tr],previous):x.copy_((x+prev)*.5)
F27:0091                     if not ok:
F27:0092                         rejected+=1
F27:0093                         for x,prev in zip([delta,gd,tr],previous):x.copy_(prev)
F27:0094                         v,j,uv=forward()
F27:0095                         value=float(objective(v,j,uv))
F27:0096                         if ok and value<bestloss:bestloss=value;best=[x.detach().clone() for x in [delta,gd,tr]]
F27:0097                     if step%100==0:print(row['id'],step,'wrist',arr((uv[ix]-goals).norm(dim=1)).round(2).tolist(),'re
F27:0097+   jects',rejected,flush=True)
F27:0098                 with torch.no_grad():
F27:0099                     for x,b in zip([delta,gd,tr],best):x.copy_(b)
F27:0100                     pp=params();saved={k:x.copy() for k,x in p.items()}
F27:0101                     for k,x in pp.items():saved[k]=arr(x).reshape(p[k].shape)

```

```

F27:0102     saved['transl']=arr(tr).reshape(p['transl'].shape)
F27:0103     candidate={k:x.copy() for k,x in saved.items()};cpu_alpha=1.
F27:0104     for trial in range(22):
F27:0105         for k in ['body_pose','global_orient','transl']:saved[k]=p[k]+cpu_alpha*(candidate[k]-p[k])
F27:0106         try:acceptance=constraints(saved,source,rigid,cam,lookup);break
F27:0107     except AssertionError:cpu_alpha*=.9
F27:0108     else:
F27:0109         saved={k:x.copy() for k,x in p.items()};cpu_alpha=0.;acceptance=constraints(saved,source,rigid,ca
F27:0109+ m,lookup)
F27:0110         vv,jj=geometry(saved);local=jj-saved['transl'].reshape(3);pixels=cam.world2camera(jj.astype(np.flo
F27:0110+ at64)).astype(np.float32)
F27:0111         saved.update(smplx_mesh_cam=vv,smplx_joint_cam=jj,smplx_joint_img=pixels,smplx_body_joints_3d_loca
F27:0111+ l=local[:25],smplx_body_joints_3d_render_cam=jj[:25],smplx_body_joints_2d_render_cam=pixels[:25],smp
F27:0111+ lx_joint_cam_root_relative=jj-jj[:1])
F27:0112         if 'smplx_body_joints_3d_cam' in saved:saved['smplx_body_joints_3d_cam']=jj[:25]
F27:0113         if 'smplx_body_joints_2d' in saved:saved['smplx_body_joints_2d']=pixels[:25]
F27:0114         for k in FIXED+['left_hand_pose','right_hand_pose']:assert np.array_equal(saved[k],p[k])
F27:0115         # Fixed beta kinematic limb lengths (selected joints) are conserved under pose.
F27:0116         lengths=lambda j:np.array([np.linalg.norm(j[a]-j[b]) for a,b in [(8,10),(10,12),(9,11),(11,13)])])
F27:0117         boneerr=float(np.max(np.abs(lengths(jj)-lengths(sj)))):assert boneerr<1e-5
F27:0118         np.savez_compressed(dst/'body_params.npz',**saved)
F27:0119         report=dict(id=row['id'],active_constraint_projection=a.active_projection,steps=a.steps,cpu_feasib
F27:0119+ ility_step_fraction=cpu_alpha,elapsed_seconds=time.time()-start>manual_inputs=False,original_hard_bo
F27:0119+ unds_preserved=True,body_shape_fixed=True,bone_length_max_change_m=boneerr,wrist_before_px=initial_w
F27:0119+ err.tolist(),wrist_after_px=np.linalg.norm(cam.world2camera(jj.astype(np.float64))[ix.cpu().numpy()])
F27:0119+ -arr(goals),axis=1).tolist(),matches=matches,loss=bestloss,rejected_steps=rejected,acceptance=accept
F27:0119+ ance)
F27:0120         report.update(silhouette_before=sil_before,silhouette_after=silhouette_metrics(vv,cam,sil_data),si
F27:0120+ lhouette_weight=.4,previous_identity_preserved=True)
F27:0121         write(dst/'refinement.json',report);print('DONE',row['id'],report['wrist_after_px'],flush=True)
F27:0122 if __name__=='__main__':main()
F28:0001 """Standard-shape bounded wrist loop plus harmonic displacement on both sides."""
F28:0002 from pathlib import Path
F28:0003 import os
F28:0004 import sys,argparse
F28:0005 from collections import deque
F28:0006 import numpy as np,cv2
F28:0007 from scipy.sparse import coo_matrix
F28:0008 from scipy.sparse.linalg import spsolve
F28:0009 from scipy.spatial.transform import Rotation
F28:0010 OUT=Path(os.environ['EGOSMPLX_REFERENCE_ROOT'])/'contour';BASE=OUT.parent/'baseline'
F28:0011 from egosmplx_pipeline.reference.support import (read,write,load,geometry,stitch,draw_geometry,FACES
F28:0011+ ,SIDES,smplx_x,FishEyeCameraCalibrated,write_obj,read_obj,constraints)
F28:0013 def adjacency(faces,n):
F28:0014     a=[set() for _ in range(n)]
F28:0015     for face in faces:
F28:0016         for x,y in zip(face,np.roll(face,-1)):a[int(x)].add(int(y));a[int(y)].add(int(x))
F28:0017     return a
F28:0019 def distance(a,seeds):
F28:0020     d=np.full(len(a),9999, dtype=int);q=deque()

```

```

F28:0021 for v in seeds:d[v]=0;q.append(int(v))
F28:0022 while q:
F28:0023     v=q.popleft()
F28:0024     for u in a[v]:
F28:0025         if d[u]>d[v]+1:d[u]=d[v]+1;q.append(u)
F28:0026 return d
F28:0028 def harmonic(a,known,values,free):
F28:0029     """Dirichlet boundary displacements; everything outside free/known is fixed."""
F28:0030     n=len(a);disp=np.zeros((n,3));disp[known]=values;idx=np.full(n,-1,int);idx[free]=np.arange(len(free)
F28:0030+ )) ;rr=[];cc=[];vv=[];rhs=np.zeros((len(free),3))
F28:0031     for r,v in enumerate(free):
F28:0032         rr.append(r);cc.append(r);vv.append(len(a[v]))
F28:0033         for u in a[v]:
F28:0034             if idx[u]>=0:rr.append(r);cc.append(idx[u]);vv.append(-1)
F28:0035             else:rhs[r]+=disp[u]
F28:0036     if len(free):disp[free]=spsolve(coo_matrix((vv,(rr,cc)),shape=(len(free),len(free))).tocsr(),rhs)
F28:0037     return disp
F28:0039 def normals(v,f):return np.cross(v[f[:,1]]-v[f[:,0]],v[f[:,2]]-v[f[:,0]])
F28:0040 def edges(f):return np.unique(np.sort(np.concatenate([f[:,[0,1]],f[:,[1,2]],f[:,[2,0]]]),axis=1),axis=
F28:0040+ 1)
F28:0041 def hand_metrics(v,target,f):
F28:0042     e=edges(f);rat=np.linalg.norm(v[e[:,0]]-v[e[:,1]],axis=1)/np.maximum(1e-10,np.linalg.norm(target[e[
F28:0042+ :,0]]-target[e[:,1]],axis=1))
F28:0043     n=normals(v,f);nt=normals(target,f)
F28:0044     return dict(edge_log_rms=float(np.sqrt(np.mean(np.log(np.maximum(rat,1e-8))*2))),edge_ratio_p01=float
F28:0044+ (np.percentile(rat,1)),edge_ratio_p99=float(np.percentile(rat,99)),normal_reversals_vs_target=int
F28:0044+ ((np.sum(n*nt,axis=1)<0).sum()),minimum_double_area=float(np.linalg.norm(n,axis=1).min()))
F28:0046 def build(p,w,matches,cam,sil_data=None):
F28:0047     from egosmplx_pipeline.reference.silhouette import metrics
F28:0048     sil_data=sil_data or []
F28:0049     base,reports=stitch(p,w,matches,cam);original,j=geometry(p);v=base['vertices_cam'].copy();mf=np.asa
F28:0049+ rray(w['mano_faces']);ha=adjacency(mf,778);report={}
F28:0050     neutral={k:x.copy() for k,x in p.items()}
F28:0051     for k in ['body_pose','global_orient','left_hand_pose','right_hand_pose']:neutral[k]=np.zeros_like(
F28:0051+ neutral[k])
F28:0052     nv,_=geometry(neutral)
F28:0053     for s,a in matches.items():
F28:0054         ids=base[s+'_local_to_mesh'];bv=base[s+'_wrist_loop_local'];global_b=ids[bv];target=base[s+'_liffe
F28:0054+ d_vertices'];posed_loop=original[global_b]
F28:0055         dx=posed_loop-posed_loop.mean(0);dy=target[bv]-target[bv].mean(0)
F28:0056         u,_,vt=np.linalg.svd(dx.T@dy);rotation=u@vt
F28:0057         if np.linalg.det(rotation)<0:u[:,-1]*=-1;rotation=u@vt
F28:0058         rotvec=Rotation.from_matrix(rotation.T).as_rotvec();rn=np.linalg.norm(rotvec);rotvec*=min(1,np.deg
F28:0058+ 2rad(30)/max(rn,1e-8));rotation=Rotation.from_rotvec(rotvec).as_matrix().T
F28:0059         standard_radius=float(np.linalg.norm(nv[global_b]-nv[global_b].mean(0),axis=1).mean());posed_radiu
F28:0059+ s=float(np.linalg.norm(dx,axis=1).mean());target_radius=float(np.linalg.norm(dy,axis=1).mean())
F28:0060         scale=np.clip(target_radius/standard_radius,.9,1.1)*standard_radius/posed_radius
F28:0061         center_move=target[bv].mean(0)-posed_loop.mean(0);center_move*=min(1,.015/max(np.linalg.norm(cente
F28:0061+ r_move),1e-10))
F28:0062         goal_loop=dx@rotation*scale+posed_loop.mean(0)+center_move

```

```

F28:0063     internal=np.isin(FACES,ids).all(axis=1);bodyfaces=FACES[~internal];ba=adjacency(bodyfaces,len(v));
F28:0063+ bd=distance(ba,global_b);hd=distance(ha,bv)
F28:0064     freebody=np.where((bd>0)&(bd<10))[0];freehand=np.where((hd>0)&(hd<7))[0]
F28:0065     touched_faces=bodyfaces[np.isin(bodyfaces,np.concatenate([global_b,freebody]))].any(axis=1)];be=edge
F28:0065+ es(touched_faces);bn=normals(original,touched_faces);b1=np.linalg.norm(original[be[:,0]]-original[be
F28:0065+[:,1]],axis=1)
F28:0066     before=hand_metrics(v[ids],target,mf);candidates=[]
F28:0067     loop_candidates=[('toward_mano_'+str(alpha),posed_loop+alpha*(goal_loop-posed_loop)) for alpha in
F28:0067+ [0.,.25,.5,.75,1.]]
F28:0068     for axisidx in range(3):
F28:0069         for deg in [-30.,-15.,15.,30.]:
F28:0070             rv=np.zeros(3);rv[axisidx]=np.deg2rad(deg);rot=Rotation.from_rotvec(rv).as_matrix().T
F28:0071             for radius_scale in [.9,1.]:
F28:0072                 loop_candidates.append(('axis%d_g_scale%g'%(axisidx,deg,radius_scale),dx@rot*(radius_scale*sta
F28:0072+ ndard_radius/posed_radius)+posed_loop.mean(0)+.5*center_move))
F28:0073     sidedata=[d for d in sil_data if d['side']==s]
F28:0074     def loop_penalty(vertices):
F28:0075         if not sidedata:return 0.
F28:0076         uv=cam.world2camera(vertices[global_b].astype(np.float64)).astype(np.float32)
F28:0077         dd=cv2.remap(sidedata[0]['outside'],uv[:,0,None],uv[:,1,None],cv2.INTER_LINEAR,borderMode=cv2.BOR
F28:0077+ DER_REPLICATE).ravel()
F28:0078         return float(np.mean(dd**2)/10000)
F28:0079     original_sil=metrics(v,cam,sidedata).get(s,dict(score=0.))
F28:0080     original_sil['wrist_loop_outside_mse_normalized']=loop_penalty(v)
F28:0081     original_sil['score']+=original_sil['wrist_loop_outside_mse_normalized']
F28:0082     for candidate_name,newloop in loop_candidates:
F28:0083         alpha=candidate_name
F28:0084         bdsp=harmonic(ba,global_b,newloop-posed_loop,freebody)
F28:0085         hdsp=harmonic(ha,bv,newloop-target[bv],freehand)
F28:0086         proposal=v.copy();proposal[freebody]=original[freebody]+bdsp[freebody];proposal[ids]=target+hdsp
F28:0087         ratio=np.linalg.norm(proposal[be[:,0]]-proposal[be[:,1]],axis=1)/b1
F28:0088         nn=normals(proposal,touched_faces);areas=np.linalg.norm(nn,axis=1)/np.maximum(1e-12,np.linalg.nor
F28:0088+ m(bn,axis=1))
F28:0089         lr=float(np.linalg.norm(proposal[global_b]-proposal[global_b].mean(0),axis=1).mean()/standard_rad
F28:0089+ ius)
F28:0090         ok=bool(np.isfinite(proposal).all() and proposal[:,2].min()>=.02 and ratio.min()>=.70 and ratio.m
F28:0090+ ax()<=1.40 and areas.min()>=.4 and areas.max()<=2.2 and (np.sum(nn*bn,axis=1)>0).all() and .85<=lr<=
F28:0090+ 1.15)
F28:0091         hm=hand_metrics(proposal[ids],target,mf)
F28:0092         if hm['normal_reversals_vs_target']>before['normal_reversals_vs_target']:ok=False
F28:0093         # Separate geometric acceptance from 2D visual accuracy; no manual input.
F28:0094         px=cam.world2camera((smpl_x.orig_hand_regressor[s]@proposal).astype(np.float64));goal=w['joints_2
F28:0094+ d']][a['candidate']];err=float(np.sqrt(np.mean(np.sum((px-goal)**2,axis=1))))
F28:0095         oldpx=cam.world2camera((smpl_x.orig_hand_regressor[s]@v).astype(np.float64));olderr=float(np.sqrt
F28:0095+ (np.mean(np.sum((oldpx-goal)**2,axis=1))))
F28:0096         if err>olderr+3:ok=False
F28:0097         sm=metrics(proposal,cam,sidedata).get(s,dict(score=0.))
F28:0098         sm['wrist_loop_outside_mse_normalized']=loop_penalty(proposal)
F28:0099         sm['score']+=sm['wrist_loop_outside_mse_normalized']
F28:0100         score=hm['edge_log_rms']+.002*err+.01*hm['normal_reversals_vs_target']+.05*sm['score']

```

```

F28:0101     diag=dict(alpha=alpha,accepted=ok,hand_metrics=hm,hand_rmse_px=err,forearm_edge_ratio_min=float(r
F28:0101+ atio.min()),forearm_edge_ratio_max=float(ratio.max()),forearm_area_ratio_min=float(areas.min()),fore
F28:0101+ arm_area_ratio_max=float(areas.max()),loop_radius_over_standard=1r,score=score,silhouette=sm)
F28:0102     candidates.append((diag,proposal))
F28:0103     originalpx=cam.world2camera((smpl_x.orig_hand_regressor[s]@v).astype(np.float64));originalerr=floa
F28:0103+ t(np.sqrt(np.mean(np.sum((originalpx-w['joints_2d'])[a['candidate']])**2,axis=1))))
F28:0104     oldscore=before['edge_log_rms']+.002*originalerr+.01*before['normal_reversals_vs_target']+5*origin
F28:0104+ al_sil['score']
F28:0105     valid=[c for c in candidates if c[0]['accepted'] and c[0]['score']<oldscore]
F28:0106     if valid:
F28:0107         chosen,v=min(valid,key=lambda c:c[0]['score']);used=True
F28:0108     else:chosen=dict(alpha=None,hand_metrics=before,hand_rmse_px=originalerr,score=oldscore,silhouette
F28:0108+ =original_sil);used=False
F28:0109     report[s]=dict(changed=used,standard_wrist_radius_m=standard_radius,posed_wrist_radius_m=posed_rad
F28:0109+ ius,lifted_mano_wrist_radius_m=target_radius,baseline_hand_metrics=before,baseline_hand_rmse_px=orig
F28:0109+ inalerr,selected=chosen,candidates=[d for d,_ in candidates],wrist_displacement_max_m=float(np.linalg
F28:0109+ g.norm(v[global_b]-original[global_b],axis=1).max()),forearm_support_vertices=int(len(freebody)),han
F28:0109+ d_transition_vertices=int(len(freehand)))
F28:0110     base['vertices_cam']=v.astype(np.float32);base['hand_joints_cam']=np.stack([smpl_x.orig_hand_regres
F28:0110+ sor[s]@v for s in SIDES]).astype(np.float32)
F28:0111     for s in matches:
F28:0112         base[s+'_legacy_blend_weights']=base.pop(s+'_blend_weights')
F28:0113     base['method']=np.array('fixed_shape_body_and_bounded_standard_wrist_harmonic_seam')
F28:0114     return base,report
F29:0001 from pathlib import Path
F29:0002 import os
F29:0003 import sys,argparse,shutil
F29:0004 import numpy as np,cv2
F29:0005 from egosmplx_pipeline.reference.silhouette import OUT,PREV,target,metrics,read,write,load,geometry
F29:0005+ ,SIDES,FishEyeCameraCalibrated,smpl_x
F29:0006 from egosmplx_pipeline.reference.contour_seam import build,write_obj,read_obj,constraints
F29:0007 from egosmplx_pipeline.reference.wrist_seam import build as previous_build,draw_geometry
F29:0008 BASE=OUT.parent/'baseline'
F29:0009 def main():
F29:0010     ap=argparse.ArgumentParser();ap.add_argument('--indices',type=int,nargs='*');ap.add_argument('--ver
F29:0010+ ify',action='store_true');a=ap.parse_args()
F29:0011     for i,row in enumerate(read(BASE/'automatic_input_manifest.json')['frames']):
F29:0012         if a.indices and i not in a.indices:continue
F29:0013         src=OUT/'selected_frames'/row['condition']/row['camera'];dst=OUT/'final_frames'/row['condition']/r
F29:0013+ ow['camera'];dst.mkdir(parents=True,exist_ok=True)
F29:0014         if not (src/'refinement.json').exists():continue
F29:0015         if (dst/'acceptance.json').exists() and not a.verify:continue
F29:0016         p=load(src/'body_params.npz');w=load(row['wilor_npz']);matches=read(BASE/'frames'/row['condition']
F29:0016+ /row['camera']/'recipe.json')['matches'];cam=FishEyeCameraCalibrated(row['calibration']);lookup={q['
F29:0016+ name']:q for q in read(row['sapiens_json'])['keypoints']}
F29:0017         # Freeze exact target strips from pre-optimization geometry for fair before/after comparison.
F29:0018         oldp=load(PREV/'final_frames'/row['condition']/row['camera']/'body_params.npz');data=target,oldp,
F29:0018+ oldp,matches,w,cam,dst)
F29:0019         mesh,report=build(p,w,matches,cam,data);oldmesh=load(PREV/'final_frames'/row['condition']/row['cam
F29:0019+ era']/'hybrid_mesh.npz');bodyonly,=previous_build(p,w,matches,cam)

```

```

F29:0020     check=constraints(p,load(row['source_npz']),load(row['rigid_npz']),cam,lookup);check.update(identi
F29:0020+ ties_unchanged=True,manual_annotations_used=False,topology_preserved=bool(np.array_equal(mesh['faces
F29:0020+ '],oldmesh['faces'])),minimum_mesh_depth_m=float(mesh['vertices_cam'][:2].min()))
F29:0021     assert check['topology_preserved'] and check['minimum_mesh_depth_m']>=.02
F29:0022     rec=dict(id=row['id'],condition=row['condition'],camera=row['camera'],hands=report,silhouette_base
F29:0022+ line=metrics(oldmesh['vertices_cam'],cam,data),silhouette_body_only=metrics(bodyonly['vertices_cam']
F29:0022+ ,cam,data),silhouette_final=metrics(mesh['vertices_cam'],cam,data))
F29:0023     if a.verify:
F29:0024         saved=load(dst/'hybrid_mesh.npz');assert set(saved)==set(mesh)
F29:0025         for k in mesh:assert np.array_equal(saved[k],mesh[k]),k
F29:0026         v,f=read_obj(dst/'hybrid_mesh.obj');assert np.array_equal(f,mesh['faces']) and np.max(np.abs(v-me
F29:0026+ sh['vertices_cam']))<1e-7
F29:0027         check.update(independent_reload=True,vertex_reload_max_abs_m=0.)
F29:0028     else:
F29:0029         np.savez_compressed(dst/'hybrid_mesh.npz',**mesh);np.savez_compressed(dst/'body_only_mesh.npz',**
F29:0029+ bodyonly);shutil.copyfile(src/'body_params.npz',dst/'body_params.npz');shutil.copyfile(src/'refineme
F29:0029+ nt.json',dst/'refinement.json');write_obj(dst/'hybrid_mesh.obj',mesh['vertices_cam'],mesh['faces']);
F29:0029+ write(dst/'geometry_report.json',rec)
F29:0030     im=cv2.imread(row['image']);cv2.imwrite(str(dst/'wireframe_skeleton.jpg'),draw_geometry(im,cam,me
F29:0030+ sh));cv2.imwrite(str(dst/'body_only_wireframe.jpg'),draw_geometry(im,cam,bodyonly))
F29:0031     write(dst/'acceptance.json',check);print('VERIFIED' if a.verify else 'FINAL',row['id'],rec['silhou
F29:0031+ ette_final'],flush=True)
F29:0032 if __name__=='__main__':main()
F30:0001 """Run the archived wrist/contour stages after the two-stage body fit.
F30:0003 The numerical worker functions are migrated from the reference ZIP. This module
F30:0004 only adapts paths, invokes them in the recorded order, and exports verified data.
F30:0005 """
F30:0006 import argparse
F30:0007 import os
F30:0008 from pathlib import Path
F30:0009 import shutil
F30:0010 import numpy as np
F30:0011 import cv2
F30:0012 from egosmplx_pipeline.io import read, write, load, sha
F30:0013 from egosmplx_pipeline.configuration import PACKAGE_ROOT
F30:0014 from egosmplx_pipeline.pipeline import command, environment
F30:0016 PROFILE = 'session_hand6_full_pipeline_20260926_v1'
F30:0019 def prepare(rows, output):
F30:0020     root = output / 'reference'
F30:0021     prepared = []
F30:0022     for row in rows:
F30:0023         frame = output / 'frames' / row['id']
F30:0024         baseline = frame / 'baseline_fused'
F30:0025         if not baseline.exists():
F30:0026             shutil.copytree(frame / 'fused', baseline)
F30:0027         matches = read(baseline / 'recipe.json')['matches']
F30:0028         if not matches:
F30:0029             continue
F30:0030         if cv2.imread(row['image']).shape[1::-1] != (1280, 720):
F30:0031             raise ValueError('The archived reference profile requires 1280 x 720 images')

```

```

F30:0032         camera = row.get('camera', 'camera')
F30:0033         if camera not in ('cam3', 'cam4', 'camera'):
F30:0034             raise ValueError('Expected cam3/cam4 for the reference profile')
F30:0035         condition = row['id']
F30:0036         destination = root / 'baseline/frames' / condition / camera
F30:0037         destination.mkdir(parents=True, exist_ok=True)
F30:0038         for source, name in [(frame / 'body/params.npz', 'body_params.npz'),
F30:0039                             (baseline / 'mesh.npz', 'hybrid_mesh.npz'),
F30:0040                             (baseline / 'recipe.json', 'recipe.json')]:
F30:0041             shutil.copyfile(source, destination / name)
F30:0042         segmentation = root / 'segmentation/frames' / condition / camera
F30:0043         segmentation.mkdir(parents=True, exist_ok=True)
F30:0044         for name in ['labels.npy', 'probabilities.npz']:
F30:0045             target = segmentation / name
F30:0046             source = Path(row['predictions'] ['segmentation']) / name
F30:0047             if target.exists() and sha(target) != sha(source):
F30:0048                 raise ValueError('Changed segmentation input: ' + row['id'])
F30:0049             if not target.exists():
F30:0050                 target.symlink_to(source.resolve())
F30:0051         prepared.append(dict(id=row['id'], condition=condition, camera=camera,
F30:0052                             image=row['image'], calibration=row['calibration'],
F30:0053                             source_npz=str(frame / 'body/params.npz'),
F30:0054                             rigid_npz=str(frame / 'rigid/optimized_params.npz'),
F30:0055                             sapiens_json=row['predictions'] ['sapiens'],
F30:0056                             wilor_npz=str(baseline / 'wilor_predictions.npz'))))
F30:0057     write(root / 'baseline/automatic_input_manifest.json', dict(frames=prepared))
F30:0058     return prepared
F30:0061 def run_workers(config, rows, output):
F30:0062     root = output / 'reference'
F30:0063     env = environment(config)
F30:0064     env['EGOSMPLX_REFERENCE_ROOT'] = str(root)
F30:0065     def execute(module, arguments, log_name):
F30:0066         command([config['paths'] ['body_python'], '-m',
F30:0067                     'egosmplx_pipeline.reference.' + module] + arguments,
F30:0068                     root / (log_name + '.log'), PACKAGE_ROOT, env)
F30:0069     execute('wrist_refine', ['—steps', '500'], '01_wrist')
F30:0070     execute('wrist_refine', ['—steps', '400', '—active-projection'], '02_wrist_active')
F30:0071     for row in rows:
F30:0072         relative = Path(row['condition']) / row['camera']
F30:0073         source, destination = root / 'wrist/active_frames' / relative, root / 'wrist/final_frames' /
F30:0073+ relative
F30:0074         destination.mkdir(parents=True, exist_ok=True)
F30:0075         for name in ['body_params.npz', 'refinement.json']:
F30:0076             shutil.copyfile(source / name, destination / name)
F30:0077         execute('wrist_seam', ['—frame-dir', 'final_frames'], '03_wrist_surface')
F30:0078         execute('contour_refine', ['—steps', '400', '—active-projection'], '04_contour_standard')
F30:0079         execute('contour_refine_strong', ['—steps', '400', '—active-projection'], '05_contour_strong')
F30:0080         execute('select_candidates', [], '06_select')
F30:0081         execute('finalize', [], '07_surface')
F30:0082         execute('finalize', ['—verify'], '08_independent_reload')

```

```

F30:0085 def export(config, rows, prepared, output):
F30:0086     os.environ['EGOSMPLX_REPO_ROOT'] = config['paths']['egosmplx_repo']
F30:0087     os.environ['EGOSMPLX_CONFIG'] = config['paths']['egosmplx_config']
F30:0088     os.environ['EGOSMPLX_REFERENCE_ROOT'] = str(output / 'reference')
F30:0089     from egosmplx_pipeline.reference.support import FishEyeCameraCalibrated, geometry, render, draw_
F30:0089+ geometry
F30:0090     from egosmplx_pipeline.render import read_obj
F30:0091     mapping = {row['id']: row for row in prepared}
F30:0092     reports = []
F30:0093     for row in rows:
F30:0094         frame = output / 'frames' / row['id']
F30:0095         report = read(frame / 'validation.json')
F30:0096         report['pipeline_profile'] = PROFILE
F30:0097         if row['id'] in mapping:
F30:0098             reference_row = mapping[row['id']]
F30:0099             source = output / 'reference/contour/final_frames' / reference_row['condition'] / refere
F30:0099+ nce_row['camera']
F30:0100             fused = frame / 'fused'
F30:0101             acceptance = read(source / 'acceptance.json')
F30:0102             if not acceptance.get('independent_reload'):
F30:0103                 raise ValueError('Final surface did not pass independent reconstruction')
F30:0104             for name, renamed in [(('body_params.npz', 'body_params.npz'), ('hybrid_mesh.npz', 'mesh.
F30:0104+ npz')),
F30:0105                                     (('hybrid_mesh.obj', 'mesh.obj'), ('refinement.json', 'refinement.j
F30:0105+ son')),
F30:0106                                     (('geometry_report.json', 'geometry_report.json'), ('acceptance.jso
F30:0106+ n', 'acceptance.json')),
F30:0107                                     (('silhouette_targets.json', 'silhouette_targets.json'))]:
F30:0108                 shutil.copyfile(source / name, fused / renamed)
F30:0109                 shutil.copyfile(Path(row['predictions']['segmentation']) / 'labels.npy', fused / 'segmen
F30:0109+ tation_labels.npy')
F30:0110                 parameters, mesh = load(fused / 'body_params.npz'), load(fused / 'mesh.npz')
F30:0111                 v, joints = geometry(parameters)
F30:0112                 camera = FishEyeCameraCalibrated(row['calibration'])
F30:0113                 image = cv2.imread(row['image'])
F30:0114                 cv2.imwrite(str(fused / 'wireframe.jpg'), render(image, mesh['vertices_cam'].astype(np.f
F30:0114+ loat64), mesh['faces'], camera, .12))
F30:0115                 cv2.imwrite(str(fused / 'skeleton.jpg'), draw_geometry(image, camera, mesh))
F30:0116                 targets = load(frame / 'body/params.npz')
F30:0117                 body_errors = np.linalg.norm(camera.world2camera(joints.astype(np.float64)) [targets['sta
F30:0117+ ge2_selected_joint_indices']].astype(int))
F30:0118                     - targets['stage2_target_joint_pixels'], axis=1)
F30:0119                 hands = load(fused / 'wilor_predictions.npz')
F30:0120                 matches = read(frame / 'baseline_fused/recipe.json')['matches']
F30:0121                 hand_errors = []
F30:0122                 for k, side in enumerate(['left', 'right']):
F30:0123                     if side in matches:
F30:0124                         pixels = camera.world2camera(mesh['hand_joints_cam'][k].astype(np.float64))
F30:0125                         hand_errors.extend(np.linalg.norm(pixels-hands['joints_2d'] [matches[side] ['candi
F30:0125+ date'] ]], axis=1).tolist())

```



```

F30:0126         obj_v, obj_f = read_obj(fused / 'mesh.obj')
F30:0127         obj_error = float(np.abs(obj_v-mesh['vertices_cam']).max())
F30:0128         body_error = float(np.abs(v-parameters['smplx_mesh_cam']).max())
F30:0129         if obj_error >= 1e-7 or body_error > 1e-5 or not np.array_equal(obj_f, mesh['faces']):
F30:0130             raise RuntimeError('Exported final geometry differs from verified mesh')
F30:0131         if 'stage2_body_error_px' not in report:
F30:0132             report['stage2_body_error_px'] = report['body_error_px']
F30:0133             report['stage2_body_rmse_px'] = report['body_rmse_px']
F30:0134         report.update(body_error_px=body_errors.tolist(), hand_error_px=hand_errors,
F30:0135                        body_rmse_px=float(np.sqrt(np.mean(body_errors**2))),
F30:0136                        hand_rmse_px=float(np.sqrt(np.mean(np.square(hand_errors)))),
F30:0137                        body_reload_max_abs_m=body_error, fusion_reload_max_abs_m=0., obj_reload_m
F30:0137+ ax_abs_m=obj_error,
F30:0138                        minimum_fused_vertex_z_m=float(mesh['vertices_cam'][:, 2].min()),
F30:0139                        hand_reports=read(fused / 'geometry_report.json')['hands'], local_surface_
F30:0139+ optimizer=True,
F30:0140                        selected_body_candidate=read(fused / 'refinement.json')['selected_candidat
F30:0140+ e'])
F30:0141         write(fused / 'recipe.json', dict(matches=matches, pipeline_profile=PROFILE,
F30:0142                method='bounded_standard_wrist_harmonic_seam_with_visible_contour',
F30:0143                body_parameter_path='body_params.npz', frozen_targets='silhouette_targets.json',
F30:0144                segmentation_labels='segmentation_labels.npy', local_surface_optimizer=True,
F30:0145                baseline_smoothstep_rings=7, manual_annotations_used=False,
F30:0146                standard_smplx_parameter_only_reconstruction=False))
F30:0147         else:
F30:0148             report.update(local_surface_optimizer=False, reference_refinement_skipped='no_matched_ha
F30:0148+ nd')
F30:0149             report['artifact_sha256'] = {str(p.relative_to(frame)): sha(p) for p in frame.rglob('*')
F30:0150                if p.is_file() and p != frame / 'validation.json'}
F30:0151             write(frame / 'validation.json', report)
F30:0152             reports.append(report)
F30:0153         write(output / 'reference/validation.json', dict(pipeline_profile=PROFILE, frames=reports))
F30:0154         return reports
F30:0157 def run(config, rows, output):
F30:0158     output = Path(output).resolve()
F30:0159     prepared = prepare(rows, output)
F30:0160     if prepared:
F30:0161         run_workers(config, prepared, output)
F30:0162     return export(config, rows, prepared, output)
F30:0165 def main():
F30:0166     parser = argparse.ArgumentParser(description=__doc__)
F30:0167     parser.add_argument('--job', type=Path, required=True)
F30:0168     args = parser.parse_args()
F30:0169     job = read(args.job)
F30:0170     run(job['config'], job['rows'], job['output'])
F30:0173 if __name__ == '__main__':
F30:0174     main()
F31:0001 """Reconstruct the specified result bundle from saved parameters and constraints.
F31:0003 This verifies the final mesh-producing algorithm. It does not rerun the networks
F31:0004 or the body/wrist optimizers, and does not need the omitted probability caches.

```

```

F31:0005 """
F31:0006 import argparse
F31:0007 import hashlib
F31:0008 import io
F31:0009 import json
F31:0010 import os
F31:0011 from pathlib import Path
F31:0012 import zipfile
F31:0013 import cv2
F31:0014 import numpy as np
F31:0015 from egosmplx_pipeline.io import write, sha
F31:0017 REFERENCE_SHA256 = '19b9baf80998dbd3f6f0da8f620295251d97a2c9aeeb7c2d282d5c31835e7fd6'
F31:0020 def frozen_targets(labels, report):
F31:0021     """Recover exactly the saved visible strips, without re-estimating targets."""
F31:0022     skin = np.isin(labels, [6,7,11,15,16,20])
F31:0023     outside = cv2.distanceTransform((~skin).astype(np.uint8), cv2.DIST_L2, 5)
F31:0024     return [dict(side=side, ids=np.array(item['vertex_ids'], dtype=int), outside=outside,
F31:0025         contour=np.array([b[k] for b in item['bins'] for k in ['left', 'right']], np.float32
F31:0025+ ))
F31:0026         for side, item in report.items() if item['enabled']]
F31:0029 def main():
F31:0030     parser = argparse.ArgumentParser(description=__doc__)
F31:0031     parser.add_argument('--archive', type=Path, required=True)
F31:0032     parser.add_argument('--repo-root', type=Path, required=True)
F31:0033     parser.add_argument('--config', type=Path, required=True, help='EgoSMPLX backend model config')
F31:0034     parser.add_argument('--output', type=Path, required=True)
F31:0035     args = parser.parse_args()
F31:0036     if sha(args.archive) != REFERENCE_SHA256:
F31:0037         raise ValueError('This verifier is pinned to the exact session_hand6 result bundle')
F31:0038     if args.output.exists():
F31:0039         raise FileExistsError(args.output)
F31:0040     args.output.mkdir(parents=True)
F31:0041     os.environ['EGOSMPLX_REPO_ROOT'] = str(args.repo_root.resolve())
F31:0042     os.environ['EGOSMPLX_CONFIG'] = str(args.config.resolve())
F31:0043     os.environ['EGOSMPLX_REFERENCE_ROOT'] = str((args.output/'reference').resolve())
F31:0044     from egosmplx_pipeline.reference.support import geometry, FishEyeCameraCalibrated, smpl_x, rende
F31:0044+ r, write_obj
F31:0045     from egosmplx_pipeline.reference.contour_seam import build
F31:0046     records = []
F31:0047     with zipfile.ZipFile(args.archive) as archive:
F31:0048         def read(name):
F31:0049             return json.loads(archive.read(name))
F31:0050         def array(name):
F31:0051             with np.load(io.BytesIO(archive.read(name)), allow_pickle=False) as z:
F31:0052                 return {key:z[key].copy() for key in z.files}
F31:0053         manifest = read('ARCHIVE_MANIFEST.json')
F31:0054         for name, entry in manifest.items():
F31:0055             content = archive.read(name)
F31:0056             if len(content) != entry['bytes'] or hashlib.sha256(content).hexdigest() != entry['sha25
F31:0056+ 6']:

```

```

F31:0057         raise ValueError('Archive member hash mismatch: ' + name)
F31:0058         cases = sorted(name[len('frames/'):-len('/03_fusion/body_params.npz')])
F31:0059         for name in manifest if name.startswith('frames/') and name.endswith('/03_fus
F31:0059+ ion/body_params.npz'))
F31:0060         for case in cases:
F31:0061             # No arbitrary ZIP paths are extracted. Only these generated case paths are used.
F31:0062             condition, camera_name = case.split('/')
F31:0063             if not condition.startswith('sample_') or camera_name not in ['cam3', 'cam4']:
F31:0064                 raise ValueError('Unexpected reference case')
F31:0065             d = args.output/'frames'/condition/camera_name
F31:0066             d.mkdir(parents=True)
F31:0067             calibration = d/'calibration.json'
F31:0068             calibration.write_bytes(archive.read('calibration/'+camera_name+'.json'))
F31:0069             camera = FishEyeCameraCalibrated(calibration)
F31:0070             prefix = 'frames/'+case+'/'
F31:0071             parameters = array(prefix+'03_fusion/body_params.npz')
F31:0072             hands = array(prefix+'automatic_observations/wilor_predictions.npz')
F31:0073             hands.update(focal_length_px=np.array(25000.), image_size_wh=np.array([1280,720]))
F31:0074             matches = read(prefix+'automatic_observations/identity_recipe.json')['matches']
F31:0075             labels = np.load(io.BytesIO(archive.read(prefix+'automatic_observations/segmentation_lab
F31:0075+ els.npy'))), allow_pickle=False)
F31:0076             targets = frozen_targets(labels, read(prefix+'03_fusion/silhouette_targets.json'))
F31:0077             mesh, surface_report = build(parameters, hands, matches, camera, targets)
F31:0078             expected = array(prefix+'03_fusion/hybrid_mesh.npz')
F31:0079             if set(mesh) != set(expected):
F31:0080                 raise RuntimeError('Rebuilt mesh fields differ from archive')
F31:0081             equality = {key:bool(np.array_equal(mesh[key], expected[key])) for key in mesh}
F31:0082             maximum = float(np.max(np.abs(mesh['vertices_cam']-expected['vertices_cam'])))
F31:0083             if not all(equality.values()):
F31:0084                 raise RuntimeError('Archive reconstruction differs: '+case+' '+str(equality))
F31:0085             np.savez_compressed(d/'hybrid_mesh.npz', **mesh)
F31:0086             write_obj(d/'hybrid_mesh.obj', mesh['vertices_cam'], mesh['faces'])
F31:0087             image = cv2.imdecode(np.frombuffer(archive.read(prefix+'input.jpg'),np.uint8), cv2.IMREA
F31:0087+ D_COLOR)
F31:0088             cv2.imwrite(str(d/'projection.jpg'), render(image,mesh['vertices_cam'].astype(np.float64
F31:0088+ ),mesh['faces'],camera,.12))
F31:0089             errors = []
F31:0090             for k, side in enumerate(['left','right']):
F31:0091                 if side in matches:
F31:0092                     pixels = camera.world2camera(mesh['hand_joints_cam'][k].astype(np.float64))
F31:0093                     errors.extend(np.linalg.norm(pixels-hands['joints_2d'][matches[side]]['candidate'
F31:0093+ ]],axis=1).tolist())
F31:0094             record = dict(case=case, all_fields_bitwise_equal=True, vertex_max_abs_m=maximum,
F31:0095                     matched_hands=len(matches), hand_error_px=errors, hand_rmse_px=float(np.sq
F31:0095+ rt(np.mean(np.square(errors)))),
F31:0096                     selected_surface_candidates={s:r['selected']['alpha'] for s,r in surface_r
F31:0096+ eport.items()})
F31:0097             write(d/'verification.json', record)
F31:0098             records.append(record)
F31:0099             print('ARCHIVE_REBUILT',case,maximum,flush=True)

```

```

F31:0100     errors = [e for record in records for e in record['hand_error_px']]
F31:0101     write(args.output/'archive_verification.json', dict(reference_archive_sha256=REFERENCE_SHA256,fr
F31:0101+ ame_count=len(records),
F31:0102         verified_archive_members=len(manifest), all_mesh_fields_bitwise_equal=True,
F31:0103         hand_rmse_px=float(np.sqrt(np.mean(np.square(errors)))),frames=records,
F31:0104         verification_scope='Final meshes rebuilt from archived body parameters, WiLoR predictions,
F31:0104+ identities and frozen contour targets; networks and optimizers were not rerun'))
F31:0107 if __name__ == '__main__':
F31:0108     main()
F32:0001 from pathlib import Path
F32:0002 import os
F32:0003 import sys,shutil
F32:0004 import numpy as np
F32:0005 from egosmplx_pipeline.reference.silhouette import OUT,PREV,targets,metrics,load,read,write,geometry
F32:0005+ ,FishEyeCameraCalibrated,SIDES
F32:0006 from egosmplx_pipeline.reference.support import constraints,JI,JOINT_MAPPING,MAX_DELTA_DEG,BODY_POSE
F32:0006+ _NAMES
F32:0007 from egosmplx_pipeline.reference.support import native_projection
F32:0008 BASE=OUT.parent/'baseline'
F32:0009 def merit(p,row,lookup,w,matches,cam,oldmesh):
F32:0010     v,j=geometry(p);uv=cam.world2camera(j.astype(np.float64));rig=load(row['rigid_npz'])
F32:0011     names=[n for n in JOINT_MAPPING if lookup[n]['score']>=.3 and np.isfinite([lookup[n]['x'],lookup[n]
F32:0011+ ['y']]).all() and 0<=lookup[n]['x']<1280 and 0<=lookup[n]['y']<720]
F32:0012     bi=[JI[JOINT_MAPPING[n]] for n in names];bt=np.array([lookup[n]['x'],lookup[n]['y']] for n in name
F32:0012+ s]);bw=np.array([min(1,lookup[n]['score'])**2 for n in names]);bw/=bw.mean()
F32:0013     huber=lambda x:np.where(np.abs(x)<.003,x*x/(2*.003),np.abs(x)-.0015)
F32:0014     body=float((huber((uv[bi]-bt)/(1280,720)).mean(1)*bw).mean());wrist=[];shape=[]
F32:0015     for s,a in matches.items():
F32:0016         i=a['candidate'];k=12 if s=='left' else 13;wrist.append(huber((uv[k]-w['joints_2d'][i,0])/[1280,72
F32:0016+ 0]).mean())
F32:0017         ids=oldmesh[s+'_local_to_mesh'][oldmesh[s+'_wrist_loop_local']];q=cam.world2camera(v[ids].astype(n
F32:0017+ p.float64));q-=q.mean(0)
F32:0018         t=native_projection(w['vertices_3d_local'][i]+w['cam_t'][i])[oldmesh[s+'_wrist_loop_local']];t-=t.
F32:0018+ mean(0)
F32:0019         shape.append(np.log(np.sqrt(np.mean(np.sum(q*q,axis=1)))/max(1,np.sqrt(np.mean(np.sum(t*t,axis=1))
F32:0019+ )))**2)
F32:0020     ai=[BODY_POSE_NAMES.index(n) for n in MAX_DELTA_DEG];caps=np.deg2rad(list(MAX_DELTA_DEG.values()))
F32:0021     pp=.0008*np.mean(((p['body_pose'].reshape(21,3)[ai]-rig['body_pose'].reshape(21,3)[ai])/caps[:,None
F32:0021+ ])**2)
F32:0022     pp+=.0002*np.mean(((p['transl'].reshape(3)-rig['transl'].reshape(3)) / [.1,.1,.18])**2)+.0004*np.me
F32:0022+ an(((p['global_orient']-rig['global_orient'])/np.deg2rad(8))**2)
F32:0023     depth=.02*max(0,(.03-v[:,2].min())/0.02)**2
F32:0024     return float(body+4*np.mean(wrist)+.0008*np.mean(shape)+pp+depth)
F32:0026 for row in read(BASE/'automatic_input_manifest.json')['frames']:
F32:0027     dst=OUT/'selected_frames'/row['condition']/row['camera'];dst.mkdir(parents=True,exist_ok=True)
F32:0028     bd=BASE/'frames'/row['condition']/row['camera'];matches=read(bd/'recipe.json')['matches'];om=load(b
F32:0028+ d/'hybrid_mesh.npz');w=load(row['wilor_npz']);cam=FishEyeCameraCalibrated(row['calibration']);lookup
F32:0028+ ={'name':q for q in read(row['sapiens_json'])['keypoints']};prev=PREV/'final_frames'/row['conditi
F32:0028+ on']/row['camera'];p0=load(prev/'body_params.npz');data=targets(row,p0,matches,w,cam,dst)
F32:0029 for d in data:

```

```

F32:0030     s=d['side'];d['ids']=np.unique(np.r_[d['ids'],om[s+'_local_to_mesh'][om[s+'_wrist_loop_local']]])
F32:0031     paths={'previous':prev,'standard':OUT/'active_frames'/row['condition']/row['camera'],'strong':OUT/'
F32:0031+ strong_frames'/row['condition']/row['camera']};scores={};diag={}
F32:0032     _,j0=geometry(p0);inituv=cam.world2camera(j0.astype(np.float64))
F32:0033     for name,path in paths.items():
F32:0034         if not (path/'body_params.npz').exists():continue
F32:0035         p=load(path/'body_params.npz');constraints(p,load(row['source_npz']),load(row['rigid_npz']),cam,lo
F32:0035+ okup);v,j=geometry(p);uv=cam.world2camera(j.astype(np.float64))
F32:0036         safe=all(np.linalg.norm(uv[l2+SIDES.index(s)]-w['joints_2d'][a['candidate'],0])<=np.linalg.norm(in
F32:0036+ ituv[l2+SIDES.index(s)]-w['joints_2d'][a['candidate'],0])+3.001 for s,a in matches.items())
F32:0037         mm=metrics(v,cam,data);sil=float(np.mean([x['score'] for x in mm.values()])) if mm else 0.;base=me
F32:0037+ rit(p,row,lookup,w,matches,cam,om)
F32:0038         diag[name]=dict(merit_without_silhouette=base,silhouette=sil,wrist_guard_passed=safe)
F32:0039         if safe:scores[name]=base+.4*sil
F32:0040         choice=min(scores,key=scores.get);shutil.copyfile(paths[choice]/'body_params.npz',dst/'body_params.
F32:0040+ npz')
F32:0041         report=read(paths[choice]/'refinement.json');report.update(selected_candidate=choice,selection_scor
F32:0041+ es=scores,selection_diagnostics=diag,matches=matches,original_identity_preserved=True)
F32:0042         write(dst/'refinement.json',report);print(row['id'],choice,scores,flush=True)
F33:0001 from pathlib import Path
F33:0002 import os
F33:0003 import sys,json
F33:0004 import cv2,numpy as np,torch
F33:0005 import torch.nn.functional as F
F33:0006 OUT=Path(os.environ['EGOSMPLX_REFERENCE_ROOT'])/'contour';PREV=OUT.parent/'wrist';SEG=OUT.parent/'se
F33:0006+ gmentation'
F33:0007 from egosmplx_pipeline.reference.wrist_seam import load,read,write,geometry,smpl_x,FishEyeCameraCali
F33:0007+ brated,SIDES
F33:0009 def targets(row,p,matches,w,cam,dst):
F33:0010     v,j=geometry(p);lookup={q['name']:q for q in read(row['sapiens_json'])['keypoints']};labels=np.load
F33:0010+ (SEG/'frames'/row['condition']/row['camera']/'labels.npy');prob=np.load(SEG/'frames'/row['condition'
F33:0010+ ]/row['camera']/'probabilities.npz')['probabilities'].astype(np.float32);skin=np.isin(labels,[6,7,11
F33:0010+ ,15,16,20]);skinprob=prob[[6,7,11,15,16,20]].sum(0)
F33:0011     weights=smpl_x.layer['neutral'].lbs_weights.detach().cpu().numpy();allhands=np.concatenate(list(smp
F33:0011+ l_x.hand_vertex_idx.values()));im=cv2.imread(row['image']);data=[];reports={}
F33:0012     for side,a in matches.items():
F33:0013         si=SIDES.index(side);q=lookup[side+'_elbow'];E=np.array([q['x'],q['y']]);W=w['joints_2d'][a['candi
F33:0013+ date'],0];vec=W-E;length=np.linalg.norm(vec);u=vec/max(1,length);n=np.array([-u[1],u[0]]);half=min(7
F33:0013+ 0,max(25,length*.30));bins=[]
F33:0014         if q['score']>=.3 and length>35:
F33:0015             for t in np.linspace(.15,.88,12):
F33:0016                 offsets=np.arange(-half,half+1);xy=E+t*vec+offsets[:,None]*n;ix=np rint(xy).astype(int);inside=(
F33:0016+ ix[:,0]>=0)&(ix[:,0]<1280)&(ix[:,1]>=0)&(ix[:,1]<720);ok=np.zeros(len(ix),bool);ok[inside]=skin[ix[i
F33:0016+ nside,1],ix[inside,0]]&(skinprob[ix[inside,1],ix[inside,0]]>.55)
F33:0017                 starts=np.where(np.diff(np.r_[False,ok,False].astype(int))==1)[0];ends=np.where(np.diff(np.r_[Fa
F33:0017+ lse,ok,False].astype(int))==1)[0]
F33:0018                 candidates=[(abs((offsets[1]+offsets[r-1])/2),1,r) for 1,r in zip(starts,ends) if r-1>=8 and 1>0
F33:0018+ and r<len(offsets)]
F33:0019                 if not candidates:continue
F33:0020                 _,1,r=min(candidates);center=(offsets[1]+offsets[r-1])/2

```

```

F33:0021     if abs(center)>max(25,length*.15):continue
F33:0022     bins.append(dict(t=float(t),lo=float(offsets[1]),hi=float(offsets[r-1]),left=xy[1].tolist(),right=xy[r-1].tolist()))
F33:0023     # Only segments with repeated observations are eligible; no inference of hidden forearm.
F33:0024     if len(bins)<3:reports[side]=dict(enabled=False,reason='insufficient_visible_forearm_cross_section')
F33:0025     alpha=np.array([x['t'] for x in bins]);tmin=max(.05,alpha.min()-.025);tmax=min(.96,alpha.max()+.025)
F33:0026     elbow=j[10+si];wrist=j[12+si];axis=wrist-elbow;s=(v-elbow)@axis/(axis@axis);rad=np.linalg.norm(v-elbow-s[:None]*axis,axis=1)
F33:0027     ids=np.where((weights[:,18+si]+weights[:,20+si]>.5)&(s>=tmin)&(s<=tmax)&(rad<.075)&~np.isin(np.arange(len(v)),allhands))[0]
F33:0028     if len(ids)<12:reports[side]=dict(enabled=False,reason='too_few_model_forearm_vertices',bins=bins);continue
F33:0029     contour=np.array([x[k] for x in bins for k in ['left','right']],np.float32)
F33:0030     # Target is the six-class union, not the model's erroneous left/right semantic labels.
F33:0031     outside=cv2.distanceTransform((~skin).astype(np.uint8),cv2.DIST_L2,5)
F33:0032     data.append(dict(side=side,ids=ids,outside=outside,contour=contour,axis_origin=E,axis_vector=vec,tmin=tmin,tmax=tmax))
F33:0033     reports[side]=dict(enabled=True,bins=bins,vertex_count=len(ids),vertex_ids=ids.tolist(),tmin=tmin,tmax=tmax,identity_source='unchanged_previous_WiLoR_assignment_and_Sapiens_elbow',occlusion_handling='visible_forearm_cross_sections_only_no_hand_object_boundary_penalty')
F33:0034     for b in bins:cv2.line(im,tuple(np rint(b['left'])).astype(int),tuple(np rint(b['right'])).astype(int),(0,255,255),2)
F33:0035     for uv in cam.world2camera(v[ids].astype(np.float64)):
F33:0036         if np.isfinite(uv).all() and np.abs(uv).max()<5000:cv2.circle(im,tuple(np rint(uv).astype(int)),1,(255,100,0),-1)
F33:0037     dst.mkdir(parents=True,exist_ok=True);write(dst/'silhouette_targets.json',reports);cv2.imwrite(str(dst/'silhouette_targets.jpg'),im)
F33:0038     return data
F33:0040 def prepare(data,device):
F33:0041     for d in data:
F33:0042         d['tid']=torch.as_tensor(d['ids'],device=device);d['map']=torch.tensor(d['outside'],device=device)
F33:0043         [None,None];d['goal']=torch.tensor(d['contour'],device=device)
F33:0044     return data
F33:0045 def loss(v,cam,data):
F33:0046     terms=[]
F33:0047     for d in data:
F33:0048         uv=cam.world2camera_pytorch(v[d['tid']]);grid=uv/uv.new_tensor([1279,719])*2-1;outside=F.grid_sample(d['map'],grid.reshape(1,1,-1,2),align_corners=True,padding_mode='border').reshape(-1)
F33:0049         overflow=F.relu(-uv)+F.relu(uv-uv.new_tensor([1279,719]));a=(outside.square().mean()+overflow.square().sum(1).mean())/10000
F33:0050         coverage=torch.cdist(d['goal'][None],uv[None])[0].min(1).values.square().mean()/10000
F33:0051         terms.append(a+.5*coverage)
F33:0052     return torch.stack(terms).mean() if terms else v.sum()*0
F33:0054 def metrics(v,cam,data):
F33:0055     result={}
F33:0056     for d in data:
F33:0057         uv=cam.world2camera(v[d['ids']]).astype(np.float64).astype(np.float32);dist=cv2.remap(d['outside'],uv[:,0,None],uv[:,1,None],cv2.INTER_LINEAR,borderMode=cv2.BORDER_REPLICATE).ravel();coverage=np.linalg

```

```

F33:0057+ alg.norm(d['contour'][:,None]-uv[None],axis=2).min(1)
F33:0058 result[d['side']] = dict(outside_rms_px=float(np.sqrt(np.mean(dist**2))),boundary_coverage_rms_px=f1
F33:0058+ oat(np.sqrt(np.mean(coverage**2))),score=float((np.mean(dist**2)+.5*np.mean(coverage**2))/10000))
F33:0059 return result
F34:0001 """Project helpers for the archived algorithm: model implementations stay external."""
F34:0002 import os
F34:0003 from pathlib import Path
F34:0004 import numpy as np
F34:0005 from egosmplx_pipeline.bootstrap import setup
F34:0006 from egosmplx_pipeline.io import read, write, load as _load, arr, rmse
F34:0007 from egosmplx_pipeline.body_common import BODY_POSE_NAMES, MAX_DELTA_DEG, JOINT_MAPPING
F34:0008 from egosmplx_pipeline.matching import assign
F34:0009 from egosmplx_pipeline import fusion
F34:0010 from egosmplx_pipeline.render import write_obj, read_obj, render
F34:0011 from egosmplx_pipeline.lift import native_projection as _project
F34:0013 smpl_x, FishEyeCameraCalibrated = setup(os.environ['EGOSMPLX_REPO_ROOT'])
F34:0014 fusion.initialize(smpl_x)
F34:0015 geometry, stitch, draw_geometry = fusion.geometry, fusion.stitch, fusion.draw_geometry
F34:0016 PARAMS, SIDES, FACES, JI = fusion.PARAMS, fusion.SIDES, fusion.FACES, fusion.JI
F34:0017 FIXED = ['betas', 'jaw_pose', 'leye_pose', 'reye_pose', 'expression']
F34:0019 def load(path):
F34:0020     data = _load(path)
F34:0021     if 'vertices_3d_local' in data and 'mano_faces' in data:
F34:0022         # Exact archived WiLoR convention: 1280 x 720 and 25000 px focal.
F34:0023         if 'focal_length_px' in data:
F34:0024             if float(data['focal_length_px']) != 25000. or not np.array_equal(data['image_size_wh'],
F34:0024+ [1280,720]):
F34:0025                 raise ValueError('Reference profile requires the archived WiLoR projection conventio
F34:0025+ n')
F34:0026             data['focal_length_px'] = np.array(25000.)
F34:0027             data['image_size_wh'] = np.array([1280,720])
F34:0028     return data
F34:0030 def native_projection(points, focal=25000.):
F34:0031     return _project(points, focal, [640.,360.])
F34:0033 def constraints(p, source, rigid, cam, lookup):
F34:0034     v,j=geometry(p); _,j0=geometry(source); _,jr=geometry(rigid)
F34:0035     active=list(MAX_DELTA_DEG); ai=[BODY_POSE_NAMES.index(n) for n in active]
F34:0036     cap=np.array([MAX_DELTA_DEG[n] for n in active])
F34:0037     delta=np.rad2deg(np.linalg.norm((p['body_pose']-rigid['body_pose']).reshape(21,3)[ai],axis=1))
F34:0038     assert np.all(delta<=cap+2e-4)
F34:0039     assert all(np.array_equal(p[k],source[k]) for k in FIXED)
F34:0040     assert np.array_equal(p['body_pose'].reshape(21,3)[[9,10]],source['body_pose'].reshape(21,3)[[9,
F34:0040+ 10]])
F34:0041     assert np.linalg.norm(p['global_orient']-rigid['global_orient'])<=np.deg2rad(8)+1e-6
F34:0042     assert .05<=p['transl'].reshape(3)[2]<=3
F34:0043     floor=np.maximum(np.linalg.norm(j0[[12,13]],axis=1),np.linalg.norm(jr[[12,13]],axis=1))*0.85
F34:0044     assert np.all(np.linalg.norm(j[[12,13]],axis=1)>=floor-1e-6)
F34:0045     assert v[:,2].min()>=.02-1e-6
F34:0046     names=[n for n in JOINT_MAPPING if lookup[n]['score']>=.3 and np.isfinite([lookup[n]['x'],lookup
F34:0046+ [n]['y']]).all() and 0<=lookup[n]['x']<1280 and 0<=lookup[n]['y']<720]

```

```

F34:0047     idx=[JI[JOINT_MAPPING[n]] for n in names]; target=np.array([[lookup[n]['x'],lookup[n]['y']] for
F34:0047+ n in names])
F34:0048     before=rmse(cam.world2camera(j0.astype(np.float64))[idx],target)
F34:0049     after=rmse(cam.world2camera(j.astype(np.float64))[idx],target)
F34:0050     assert after<=before+5.001
F34:0051     return dict(passed=True,body_delta_deg=dict(zip(active,map(float,delta))),sapiens_body_rmse_px=a
F34:0051+ fter,
F34:0052         sapiens_body_source_rmse_px=before,wrist_floor_m=floor.tolist(),body_minimum_z_m=flo
F34:0052+ at(v[:,2].min()))
F35:0001     """Fixed-shape SMPL-X wrist and projected seam compatibility refinement."""
F35:0002 from pathlib import Path
F35:0003 import os
F35:0004 import sys,copy,argparse,time
F35:0005 import numpy as np
F35:0006 import torch
F35:0007 import torch.nn.functional as F
F35:0008 OUT=Path(os.environ['EGOSMPLX_REFERENCE_ROOT'])/'wrist'
F35:0009 BASE=OUT.parent/'baseline'
F35:0011 from egosmplx_pipeline.reference.support import (read,write,load,arr,geometry,constraints,PARAMS,FIX
F35:0011+ ED,SIDES,JI,MAX_DELTA_DEG,BODY_POSE_NAMES,JOINT_MAPPING,smpl_x,FishEyeCameraCalibrated)
F35:0012 from egosmplx_pipeline.reference.support import native_projection
F35:0014 def main():
F35:0015     ap=argparse.ArgumentParser();ap.add_argument('--steps',type=int,default=500);ap.add_argument('--ind
F35:0015+ ices',type=int,nargs='*');ap.add_argument('--active-projection',action='store_true');a=ap.parse_args
F35:0015+ ()
F35:0016     torch.backends.cuda.matmul.allow_tf32=False;torch.backends.cudnn.allow_tf32=False
F35:0017     torch.set_num_threads(2);dev='cuda' if torch.cuda.is_available() else 'cpu';layer=copy.deepcopy(smp
F35:0017+ l_x.layer['neutral']).to(dev).eval();layer.requires_grad_(False)
F35:0018     t=lambda x:torch.as_tensor(x, dtype=torch.float32, device=dev)
F35:0019     rows=read(BASE/'automatic_input_manifest.json')['frames']
F35:0020     for ii,row in enumerate(rows):
F35:0021         if a.indices and ii not in a.indices:continue
F35:0022         dst=OUT/('active_frames' if a.active_projection else 'frames')/row['condition']/row['camera'];dst.
F35:0022+ mkdir(parents=True,exist_ok=True)
F35:0023         if (dst/'refinement.json').exists():continue
F35:0024         bd=BASE/'frames'/row['condition']/row['camera'];p=load(bd/'body_params.npz');source=load(row['sour
F35:0024+ ce.npz']);rigid=load(row['rigid.npz']);w=load(row['wilor.npz']);oldmesh=load(bd/'hybrid_mesh.npz');m
F35:0024+ atches=read(bd/'recipe.json')['matches'];cam=FishEyeCameraCalibrated(row['calibration'])
F35:0025         lookup={q['name']:q for q in read(row['sapiens_json'])['keypoints']}
F35:0026         active=list(MAX_DELTA_DEG);ai=torch.tensor([BODY_POSE_NAMES.index(n) for n in active],device=dev);
F35:0026+ caps=t(np.deg2rad([MAX_DELTA_DEG[n] for n in active]))
F35:0027         b0=t(rigid['body_pose']).reshape(21,3);g0=t(rigid['global_orient']).reshape(3);tr0=t(rigid['transl
F35:0027+ ']).reshape(3)
F35:0028         init_path=OUT/'frames'/row['condition']/row['camera']/'body_params.npz'
F35:0029         initp=load(init_path) if a.active_projection and init_path.exists() else p
F35:0030         delta=(t(initp['body_pose']).reshape(21,3)-b0)[ai].clone().requires_grad_();gd=(t(initp['global_or
F35:0030+ rient']).reshape(3)-g0).clone().requires_grad_();tr=t(initp['transl1']).reshape(3).clone().requires_gr
F35:0030+ ad_()
F35:0031         names=[n for n in JOINT_MAPPING if lookup[n]['score']>=.3 and np.isfinite([lookup[n]['x'],lookup[n]
F35:0031+ ]['y']]).all() and 0<=lookup[n]['x']<1280 and 0<=lookup[n]['y']<720]

```



```

F35:0032 bi=torch.tensor([JI[JOINT_MAPPING[n]] for n in names],device=dev);bt=t([[lookup[n]['x'],lookup[n]
F35:0032+ 'y']] for n in names]);bw=t([min(1,lookup[n]['score'])**2 for n in names]);bw/=bw.mean()
F35:0033 ix=torch.tensor([12+SIDES.index(s) for s in matches],device=dev);goals=t([w['joints_2d'][m['candid
F35:0033+ ate'],0] for s,m in matches.items()))
F35:0034 _,src,j=geometry(source);_,rig,j=geometry(rigid);floor=t(np.maximum(np.linalg.norm(src,j[[12,13]],axi
F35:0034+ s=1),np.linalg.norm(rig,j[[12,13]],axis=1))*0.85)
F35:0035 bodylimit=float(np.sqrt(np.mean(np.sum((cam.world2camera(src,j.astype(np.float64)))[bi.cpu().numpy()
F35:0035+ ]-arr(bt))**2,axis=1))))+5
F35:0036 sv,sj=geometry(p);initial_werr=np.linalg.norm(cam.world2camera(sj.astype(np.float64))[ix.cpu().num
F35:0036+ py()-arr(goals),axis=1)
F35:0037 loopids=[];loopgoals=[]
F35:0038 for s,m in matches.items():
F35:0039 ids=oldmesh[s+'_local_to_mesh'];bv=oldmesh[s+'_wrist_loop_local'];loopids.append(torch.tensor(ids
F35:0039+ [bv],device=dev))
F35:0040 uv= native_projection(w['vertices_3d_local'][m['candidate']] + w['cam_t'][m['candidate']]);uv=uv[bv]
F35:0040+ :loopgoals.append(t(uv-uv.mean(0)))
F35:0041 def params():
F35:0042 pp={k:t(p[k]).reshape(1,-1) for k in PARAMS};body=b0.clone();body[ai]+=delta;pp.update(body_pose=
F35:0042+ body.reshape(1,-1),global_orient=(g0+gd).reshape(1,3));return pp
F35:0043 def forward():
F35:0044 o=layer(**params());v=o.vertices[0]+tr;j=o.joints[0,smpl_x.joint_idx]+tr;uv=cam.world2camera_pyto
F35:0044+ rch(j);return v,j,uv
F35:0045 def feasible(v,j,uv):
F35:0046 err=(uv[ix]-goals).norm(dim=1)
F35:0047 return bool(v[:,2].min()>=.020001 and (j[[12,13]].norm(dim=1)>=floor+1e-6).all() and torch.sqrt((
F35:0047+ uv[bi]-bt).square().sum(1).mean())<=bodylimit-1e-4 and (err<=t(initial_werr)+3).all())
F35:0048 def objective(v,j,uv):
F35:0049 body=(F.smooth_l1_loss((uv[bi]-bt)/t([1280,720]),torch.zeros_like(bt),beta=.003,reduction='none')
F35:0049+ .mean(1)*bw).mean()
F35:0050 wrists=F.smooth_l1_loss((uv[ix]-goals)/t([1280,720]),torch.zeros_like(goals),beta=.003)
F35:0051 seam=[]
F35:0052 for ids,target in zip(loopids,loopgoals):
F35:0053 q=cam.world2camera_pytorch(v[ids]);q=q-q.mean(0);radius=q.square().sum(1).mean().sqrt();rr=targe
F35:0053+ t.square().sum(1).mean().sqrt();seam.append(torch.log(radius/rr.clamp_min(1)).square())
F35:0054 shape=torch.stack(seam).mean()
F35:0055 priors=.0008*(delta/caps[:,None]).square().mean()+.0002*((tr-tr0)/t([.10,.10,.18])).square().mean
F35:0055+ )+.0004*(gd/np.deg2rad(8)).square().mean()
F35:0056 depth=.02*F.relu((.03-v[:,2].min())/0.02).square();guard=.002*F.relu((floor-j[[12,13]].norm(dim=1)
F35:0056+ )/.05).square().mean()
F35:0057 return body+4*wrists+.0008*shape+priors+depth+guard
F35:0058 opt=torch.optim.Adam([{'params':[delta],'lr':.0015},{'params':[gd],'lr':.0005},{'params':[tr],'lr'
F35:0058+ :.0008}])
F35:0059 with torch.no_grad():v,j,uv=forward();bestloss=float(objective(v,j,uv));best=[x.detach().clone() f
F35:0059+ or x in [delta,gd,tr]]
F35:0060 rejected=0;start=time.time()
F35:0061 for step in range(a.steps):
F35:0062 opt.zero_grad();v,j,uv=forward();loss=objective(v,j,uv);loss.backward()
F35:0063 previous=[x.detach().clone() for x in [delta,gd,tr]];opt.step()
F35:0064 if a.active_projection:
F35:0065 # Project infeasible Adam proposals back onto nonlinear constraint surfaces.

```

```

F35:0066     for project_step in range(12):
F35:0067         with torch.no_grad():
F35:0068             delta.mul_(torch.minimum(torch.ones_like(caps),caps/delta.norm(dim=1).clamp_min(1e-8))[:,None]
F35:0068+ ) ;gd.mul_(torch.clamp(t(np.deg2rad(8))/gd.norm().clamp_min(1e-8),max=1));tr[2].clamp_(.05,3)
F35:0069         pv,pj,pu=forward()
F35:0070         br=(pu[bi]-bt).square().sum(1).mean().sqrt()
F35:0071         violations=torch.cat([(br-bodylimit+.02).reshape(1)/5,(.02001-pv[:,2].min()).reshape(1)/.02,(f1
F35:0071+ oor+1e-5-pj[[12,13]].norm(dim=1))/.05,((pu[ix]-goals).norm(dim=1)-t(initial_werr)-2.99)/5])
F35:0072         g=violations.max()
F35:0073         if float(g)<=0:break
F35:0074         grads=torch.autograd.grad(g,[delta,gd,tr]);scales=[.0015,.0005,.0008]
F35:0075         denom=sum((grad.square().sum()*scale**2) for grad,scale in zip(grads,scales)).clamp_min(1e-12)
F35:0076         with torch.no_grad():
F35:0077             for x,grad,scale in zip([delta,gd,tr],grads,scales):x.sub_(1.05*g.detach()*grad*scale**2/denom
F35:0077+ )
F35:0078         with torch.no_grad():
F35:0079             delta.mul_(torch.minimum(torch.ones_like(caps),caps/delta.norm(dim=1).clamp_min(1e-8))[:,None]);
F35:0079+ gd.mul_(torch.clamp(t(np.deg2rad(8))/gd.norm().clamp_min(1e-8),max=1));tr[2].clamp_(.05,3)
F35:0080         ok=False
F35:0081         for trial in range(10):
F35:0082             v,j,uv=forward()
F35:0083             if feasible(v,j,uv):ok=True;break
F35:0084             for x,prev in zip([delta,gd,tr],previous):x.copy_((x+prev)*.5)
F35:0085             if not ok:
F35:0086                 rejected+=1
F35:0087                 for x,prev in zip([delta,gd,tr],previous):x.copy_(prev)
F35:0088                 v,j,uv=forward()
F35:0089                 value=float(objective(v,j,uv))
F35:0090                 if ok and value<bestloss:bestloss=value;best=[x.detach().clone() for x in [delta,gd,tr]]
F35:0091                 if step%100==0:print(row['id'],step,'wrist',arr((uv[ix]-goals).norm(dim=1)).round(2).tolist(),'re
F35:0091+ jects',rejected,flush=True)
F35:0092         with torch.no_grad():
F35:0093             for x,b in zip([delta,gd,tr],best):x.copy_(b)
F35:0094             pp=params();saved={k:x.copy() for k,x in p.items()}
F35:0095             for k,x in pp.items():saved[k]=arr(x).reshape(p[k].shape)
F35:0096             saved['transl']=arr(tr).reshape(p['transl'].shape)
F35:0097             candidate={k:x.copy() for k,x in saved.items()};cpu_alpha=1.
F35:0098             for trial in range(22):
F35:0099                 for k in ['body_pose','global_orient','transl']:saved[k]=p[k]+cpu_alpha*(candidate[k]-p[k])
F35:0100                 try:acceptance=constraints(saved,source,rigid,cam,lookup);break
F35:0101                 except AssertionError:cpu_alpha*=.9
F35:0102             else:
F35:0103                 saved={k:x.copy() for k,x in p.items()};cpu_alpha=0.;acceptance=constraints(saved,source,rigid,ca
F35:0103+ m,lookup)
F35:0104                 vv,jj=geometry(saved);local=jj-saved['transl'].reshape(3);pixels=cam.world2camera(jj.astype(np.flo
F35:0104+ at64)).astype(np.float32)
F35:0105                 saved.update(smplx_mesh_cam=vv,smplx_joint_cam=jj,smplx_joint_img=pixels,smplx_body_joints_3d_loca
F35:0105+ l=local[:25],smplx_body_joints_3d_render_cam=jj[:25],smplx_body_joints_2d_render_cam=pixels[:25],smp
F35:0105+ lx_joint_cam_root_relative=jj-jj[:1])
F35:0106                 if 'smplx_body_joints_3d_cam' in saved:saved['smplx_body_joints_3d_cam']=jj[:25]

```

```

F35:0107 if 'smplx_body_joints_2d' in saved:saved['smplx_body_joints_2d']=pixels[:25]
F35:0108 for k in FIXED+['left_hand_pose','right_hand_pose']:assert np.array_equal(saved[k],p[k])
F35:0109 # Fixed beta kinematic limb lengths (selected joints) are conserved under pose.
F35:0110 lengths=lambda j:np.array([np.linalg.norm(j[a]-j[b]) for a,b in [(8,10),(10,12),(9,11),(11,13)])])
F35:0111 boneerr=float(np.max(np.abs(lengths(jj)-lengths(sj)))):assert boneerr<1e-5
F35:0112 np.savez_compressed(dst/'body_params.npz',**saved)
F35:0113 report=dict(id=row['id'],active_constraint_projection=a.active_projection,steps=a.steps,cpu_feasib
F35:0113+ ility_step_fraction=cpu_alpha,elapsed_seconds=time.time()-start>manual_inputs=False,original_hard_bo
F35:0113+ unds_preserved=True,body_shape_fixed=True,bone_length_max_change_m=boneerr,wrist_before_px=initial_w
F35:0113+ err.tolist(),wrist_after_px=np.linalg.norm(cam.world2camera(jj.astype(np.float64))[ix.cpu().numpy()])
F35:0113+ -arr(goals),axis=1).tolist(),matches=matches,loss=bestloss,rejected_steps=rejected,acceptance=accept
F35:0113+ ance)
F35:0114 write(dst/'refinement.json',report):print('DONE',row['id'],report['wrist_after_px'],flush=True)
F35:0115 if __name__=='__main__':main()
F36:0001 """Standard-shape bounded wrist loop plus harmonic displacement on both sides."""
F36:0002 from pathlib import Path
F36:0003 import os
F36:0004 import sys,argparse
F36:0005 from collections import deque
F36:0006 import numpy as np,cv2
F36:0007 from scipy.sparse import coo_matrix
F36:0008 from scipy.sparse.linalg import spsolve
F36:0009 from scipy.spatial.transform import Rotation
F36:0010 OUT=Path(os.environ['EGOSMPLX_REFERENCE_ROOT'])/'wrist':BASE=OUT.parent/'baseline'
F36:0011 from egosmplx_pipeline.reference.support import (read,write,load,geometry,stitch,draw_geometry,FACES
F36:0011+ ,SIDES,smpl_x,FishEyeCameraCalibrated,write_obj,read_obj,constraints)
F36:0013 def adjacency(faces,n):
F36:0014 a=[set() for _ in range(n)]
F36:0015 for face in faces:
F36:0016 for x,y in zip(face,np.roll(face,-1)):a[int(x)].add(int(y)):a[int(y)].add(int(x))
F36:0017 return a
F36:0019 def distance(a,seeds):
F36:0020 d=np.full(len(a),9999, dtype=int):q=deque()
F36:0021 for v in seeds:d[v]=0;q.append(int(v))
F36:0022 while q:
F36:0023 v=q.popleft()
F36:0024 for u in a[v]:
F36:0025 if d[u]>d[v]+1:d[u]=d[v]+1;q.append(u)
F36:0026 return d
F36:0028 def harmonic(a,known,values,free):
F36:0029 """Dirichlet boundary displacements: everything outside free/known is fixed."""
F36:0030 n=len(a):disp=np.zeros((n,3)):disp[known]=values:idx=np.full(n,-1,int):idx[free]=np.arange(len(free
F36:0030+ )):rr=[]:cc=[]:vv=[]:rhs=np.zeros((len(free),3))
F36:0031 for r,v in enumerate(free):
F36:0032 rr.append(r):cc.append(r):vv.append(len(a[v]))
F36:0033 for u in a[v]:
F36:0034 if idx[u]>=0:rr.append(r):cc.append(idx[u]):vv.append(-1)
F36:0035 else:rhs[r]+=disp[u]
F36:0036 if len(free):disp[free]=spsolve(coo_matrix((vv,(rr,cc)),shape=(len(free),len(free))).tocsr(),rhs)
F36:0037 return disp

```

```

F36:0039 def normals(v,f):return np.cross(v[f[:,1]]-v[f[:,0]],v[f[:,2]]-v[f[:,0]])
F36:0040 def edges(f):return np.unique(np.sort(np.concatenate([f[:,[0,1]],f[:,[1,2]],f[:,[2,0]]]),axis=1),axi
F36:0040+ s=0)
F36:0041 def hand_metrics(v,target,f):
F36:0042     e=edges(f);rat=np.linalg.norm(v[e[:,0]]-v[e[:,1]],axis=1)/np.maximum(1e-10,np.linalg.norm(target[e[
F36:0042+ :,0]]-target[e[:,1]],axis=1))
F36:0043     n=normals(v,f);nt=normals(target,f)
F36:0044     return dict(edge_log_rms=float(np.sqrt(np.mean(np.log(np.maximum(rat,1e-8))**2))),edge_ratio_p01=f1
F36:0044+ oat(np.percentile(rat,1)),edge_ratio_p99=float(np.percentile(rat,99)),normal_reversals_vs_target=int
F36:0044+ ((np.sum(n*nt,axis=1)<0).sum()),minimum_double_area=float(np.linalg.norm(n,axis=1).min()))
F36:0046 def build(p,w,matches,cam):
F36:0047     base,reports=stitch(p,w,matches,cam);original,j=geometry(p);v=base['vertices_cam'].copy();mf=np.asa
F36:0047+ rray(w['mano_faces']);ha=adjacency(mf,778);report={}
F36:0048     neutral={k:x.copy() for k,x in p.items()}
F36:0049     for k in ['body_pose','global_orient','left_hand_pose','right_hand_pose']:neutral[k]=np.zeros_like(
F36:0049+ neutral[k])
F36:0050     nv,_=geometry(neutral)
F36:0051     for s,a in matches.items():
F36:0052         ids=base[s+'_local_to_mesh'];bv=base[s+'_wrist_loop_local'];global_b=ids[bv];target=base[s+'_lifte
F36:0052+ d_vertices'];posed_loop=original[global_b]
F36:0053         dx=posed_loop-posed_loop.mean(0);dy=target[bv]-target[bv].mean(0)
F36:0054         u,_,vt=np.linalg.svd(dx.T@dy);rotation=u@vt
F36:0055         if np.linalg.det(rotation)<0:u[:,-1]*=-1;rotation=u@vt
F36:0056         rotvec=Rotation.from_matrix(rotation.T).as_rotvec();rn=np.linalg.norm(rotvec);rotvec*=min(1,np.deg
F36:0056+ 2rad(30)/max(rn,1e-8));rotation=Rotation.from_rotvec(rotvec).as_matrix().T
F36:0057         standard_radius=float(np.linalg.norm(nv[global_b]-nv[global_b].mean(0),axis=1).mean());posed_radiu
F36:0057+ s=float(np.linalg.norm(dx,axis=1).mean());target_radius=float(np.linalg.norm(dy,axis=1).mean())
F36:0058         scale=np.clip(target_radius/standard_radius,.9,1.1)*standard_radius/posed_radius
F36:0059         center_move=target[bv].mean(0)-posed_loop.mean(0);center_move*=min(1,.015/max(np.linalg.norm(cente
F36:0059+ r_move),1e-10))
F36:0060         goal_loop=dx@rotation*scale+posed_loop.mean(0)+center_move
F36:0061         internal=np.isin(FACES,ids).all(axis=1);bodyfaces=FACES[~internal];ba=adjacency(bodyfaces,len(v));
F36:0061+ bd=distance(ba,global_b);hd=distance(ha,bv)
F36:0062         freebody=np.where((bd>0)&(bd<10))[0];freehand=np.where((hd>0)&(hd<7))[0]
F36:0063         touched_faces=bodyfaces[np.isin(bodyfaces,np.concatenate([global_b,freebody]))].any(axis=1)];be=edg
F36:0063+ es(touched_faces);bn=normals(original,touched_faces);b1=np.linalg.norm(original[be[:,0]]-original[be
F36:0063+[:,1]],axis=1)
F36:0064         before=hand_metrics(v[ids],target,mf);candidates=[]
F36:0065         for alpha in [0.,.25,.5,.75,1.]:
F36:0066             newloop=posed_loop+alpha*(goal_loop-posed_loop)
F36:0067             bdsp=harmonic(ba,global_b,newloop-posed_loop,freebody)
F36:0068             hdsp=harmonic(ha,bv,newloop-target[bv],freehand)
F36:0069             proposal=v.copy();proposal[freebody]=original[freebody]+bdsp[freebody];proposal[ids]=target+hdsp
F36:0070             ratio=np.linalg.norm(proposal[be[:,0]]-proposal[be[:,1]],axis=1)/b1
F36:0071             nn=normals(proposal,touched_faces);areas=np.linalg.norm(nn,axis=1)/np.maximum(1e-12,np.linalg.nor
F36:0071+ m(bn,axis=1))
F36:0072             lr=float(np.linalg.norm(proposal[global_b]-proposal[global_b].mean(0),axis=1).mean()/standard_rad
F36:0072+ ius)
F36:0073             ok=bool(np.isfinite(proposal).all() and proposal[:,2].min()>=.02 and ratio.min()>=.70 and ratio.m
F36:0073+ ax()<=1.40 and areas.min()>=.4 and areas.max()<=2.2 and (np.sum(nn*bn,axis=1)>0).all() and .85<=lr<=

```

```

F36:0073+ 1.15)
F36:0074     hm=hand_metrics(proposal[ids],target,mf)
F36:0075     if hm['normal_reversals_vs_target']>before['normal_reversals_vs_target']:ok=False
F36:0076     # Separate geometric acceptance from 2D visual accuracy; no manual input.
F36:0077     px=cam.world2camera((smp1_x.orig_hand_regressor[s]@proposal).astype(np.float64));goal=w['joints_2
F36:0077+ d'][a['candidate']];err=float(np.sqrt(np.mean(np.sum((px-goal)**2,axis=1))))
F36:0078     oldpx=cam.world2camera((smp1_x.orig_hand_regressor[s]@v).astype(np.float64));olderr=float(np.sqrt
F36:0078+ (np.mean(np.sum((oldpx-goal)**2,axis=1))))
F36:0079     if err>olderr+3:ok=False
F36:0080     score=hm['edge_log_rms']+.002*err+.01*hm['normal_reversals_vs_target']
F36:0081     diag=dict(alpha=alpha,accepted=ok,hand_metrics=hm,hand_rmse_px=err,forearm_edge_ratio_min=float(r
F36:0081+ atio.min()),forearm_edge_ratio_max=float(ratio.max()),forearm_area_ratio_min=float(areas.min()),fore
F36:0081+ arm_area_ratio_max=float(areas.max()),loop_radius_over_standard=1r,score=score)
F36:0082     candidates.append((diag,proposal))
F36:0083     originalpx=cam.world2camera((smp1_x.orig_hand_regressor[s]@v).astype(np.float64));originalerr=floa
F36:0083+ t(np.sqrt(np.mean(np.sum((originalpx-w['joints_2d'][a['candidate']])**2,axis=1))))
F36:0084     oldscore=before['edge_log_rms']+.002*originalerr+.01*before['normal_reversals_vs_target']
F36:0085     valid=[c for c in candidates if c[0]['accepted'] and c[0]['score']<oldscore]
F36:0086     if valid:
F36:0087         chosen,v=min(valid,key=lambda c:c[0]['score']);used=True
F36:0088     else:chosen=dict(alpha=None,hand_metrics=before,hand_rmse_px=originalerr,score=oldscore);used=False
F36:0088+ e
F36:0089     report[s]=dict(changed=used,standard_wrist_radius_m=standard_radius,posed_wrist_radius_m=posed_rad
F36:0089+ ius,lifted_mano_wrist_radius_m=target_radius,baseline_hand_metrics=before,baseline_hand_rmse_px=orig
F36:0089+ inalerr,selected=chosen,candidates=[d for d,_ in candidates],wrist_displacement_max_m=float(np.linalg
F36:0089+ g.norm(v[global_b]-original[global_b],axis=1).max()),forearm_support_vertices=int(len(freebody)),han
F36:0089+ d_transition_vertices=int(len(freehand)))
F36:0090     base['vertices_cam']=v.astype(np.float32);base['hand_joints_cam']=np.stack([smp1_x.orig_hand_regres
F36:0090+ sor[s]@v for s in SIDES]).astype(np.float32)
F36:0091     for s in matches:
F36:0092         base[s+'_legacy_blend_weights']=base.pop(s+'_blend_weights')
F36:0093     base['method']=np.array('fixed_shape_body_and_bounded_standard_wrist_harmonic_seam')
F36:0094     return base,report
F36:0096 def main():
F36:0097     ap=argparse.ArgumentParser();ap.add_argument('--verify-only',action='store_true');ap.add_argument('
F36:0097+ --frame-dir',default='frames');a=ap.parse_args();records=[]
F36:0098     for row in read(BASE/'automatic_input_manifest.json')['frames']:
F36:0099         d=OUT/a.frame_dir/row['condition']/row['camera']
F36:0100         if not (d/'refinement.json').exists():continue
F36:0101         if not a.verify_only and (d/'geometry_report.json').exists():continue
F36:0102         p=load(d/'body_params.npz');w=load(row['wilor_npz']);matches=read(d/'refinement.json')['matches'];
F36:0102+ cam=FishEyeCameraCalibrated(row['calibration'])
F36:0103         mesh,report=build(p,w,matches,cam)
F36:0104         if a.verify_only:
F36:0105             stored=load(d/'hybrid_mesh.npz')
F36:0106             for k in mesh:assert np.array_equal(mesh[k],stored[k]),k
F36:0107             v,f=read_obj(d/'hybrid_mesh.obj');assert np.array_equal(f,mesh['faces']) and np.max(np.abs(v-mesh
F36:0107+ ['vertices_cam']))<1e-7
F36:0108             lookup={q['name']:q for q in read(row['sapiens_json'])['keypoints']};check=constraints(p,load(row
F36:0108+ ['source_npz']),load(row['rigid_npz']),cam,lookup)

```

```

F36:0109     check.update(independent_process_reload=True,mesh_reload_max_abs_m=0.,mesh_minimum_z_m=float(v[:,
F36:0109+ 2].min()),bounded_geometry_passed=True,topology_unchanged_from_seam=True)
F36:0110     write(d/'acceptance.json',check);print(' VERIFIED',row['id'],flush=True);continue
F36:0111     np.savez_compressed(d/'hybrid_mesh.npz',**mesh);write_obj(d/'hybrid_mesh.obj',mesh['vertices_cam']
F36:0111+ ,mesh['faces'])
F36:0112     # Keep body-only ablation of the old seam to separate each change.
F36:0113     bm,_=stitch(p,w,matches,cam);np.savez_compressed(d/'body_refined_old_seam.npz',**bm)
F36:0114     image=cv2.imread(row['image']);cv2.imwrite(str(d/'wireframe_skeleton.jpg'),draw_geometry(image,cam
F36:0114+ ,mesh));cv2.imwrite(str(d/'body_only_wireframe.jpg'),draw_geometry(image,cam,bm))
F36:0115     rec=dict(id=row['id'],condition=row['condition'],camera=row['camera'],hands=report);write(d/'geome
F36:0115+ try_report.json',rec);records.append(rec)
F36:0116     print('SEAM',row['id'],{s:(x['changed'],x['selected'] ['alpha'],round(x['baseline_hand_metrics'] ['e
F36:0116+ dge_log_rms'],3),round(x['selected'] ['hand_metrics'] ['edge_log_rms'],3)) for s,x in report.items()}),
F36:0116+ flush=True)
F36:0117     if not a.verify_only:write(OUT/(a.frame_dir+'_summary.json'),dict(frames=[read(p) for p in sorted((
F36:0117+ OUT/a.frame_dir).glob('*/*/geometry_report.json'))],manual_inputs=False,template_shape_and_bone_leng
F36:0117+ ths_fixed=True))
F36:0118     if __name__=='__main__':main()
F37:0001     """Model-free regression checks for data safety and geometric conventions."""
F37:0002     import json
F37:0003     from pathlib import Path
F37:0004     import tempfile
F37:0005     import unittest
F37:0006     import numpy as np
F37:0007     from egosmplx_pipeline.io import load, rmse, write
F37:0008     from egosmplx_pipeline.matching import assign
F37:0009     from egosmplx_pipeline.topology import boundary_edges, graph_distance_from_boundary
F37:0010     from egosmplx_pipeline.lift import native_projection, lift_mano_to_fisheye
F37:0011     from egosmplx_pipeline.configuration import load_config, expand
F37:0014     class GeometryTests(unittest.TestCase):
F37:0015         def test_rmse_is_point_distance(self):
F37:0016             self.assertEqual(rmse([[3,4]], [[0,0]]), 5)
F37:0017             self.assertIsNone(rmse(np.empty((0,2)), np.empty((0,2))))
F37:0019         def test_ring_distance_and_disconnected_mesh(self):
F37:0020             faces = np.array([[0,1,4],[1,2,4],[2,3,4],[3,0,4]])
F37:0021             self.assertEqual(len(boundary_edges(faces)), 4)
F37:0022             np.testing.assert_array_equal(graph_distance_from_boundary(faces,[0,1,2,3],5), [0,0,0,0,1])
F37:0023             with self.assertRaises(RuntimeError):
F37:0024                 graph_distance_from_boundary(faces,[0,1,2,3],6)
F37:0026         def test_no_duplicate_hand_assignment(self):
F37:0027             observation = dict(detector_confidence=np.array([.9]), is_right=np.array([0]),
F37:0028                 joints_2d=np.tile([300.,300.], (1,21,1)))
F37:0029             lookup = {s+'_wrist':dict(x=300,y=300,score=.9) for s in ['left','right']}
F37:0030             matches = assign(observation, lookup, np.zeros((25,2)))
F37:0031             self.assertEqual(len(matches), 1)
F37:0032             self.assertEqual(matches['left'] ['candidate'], 0)
F37:0034         def test_missing_and_low_confidence_hands(self):
F37:0035             observation = dict(detector_confidence=np.array([.1]), is_right=np.array([1]),
F37:0036                 joints_2d=np.tile([300.,300.], (1,21,1)))
F37:0037             self.assertEqual(assign(observation, {}, np.zeros((25,2))), {})

```

```

F37:0038         observation['detector_confidence'] = np.empty(0)
F37:0039         self.assertEqual(assign(observation, {}, np.zeros((25,2))), {})
F37:0041     def test_lift_uses_radial_distance_and_preserves_shifted_projection(self):
F37:0042         class Camera:
F37:0043             def camera2world(self, uv, radial):
F37:0044                 ray = np.c_[(uv-[640,360])/1000, np.ones(len(uv))]
F37:0045                 return ray/np.linalg.norm(ray,axis=1)[:,None]*radial[:,None]
F37:0046             points = np.array([[.1,.2,1.],[.2,.3,1.2]])
F37:0047             w = dict(vertices_3d_local=points[None], joints_3d_local=points[None],
F37:0048                     cam_t=np.zeros((1,3)), focal_length_px=np.array(1000.), image_size_wh=np.array([128
F37:0048+ 0,720]))
F37:0049             body_wrist = np.array([.2,.3,.7])
F37:0050             result = lift_mano_to_fisheye(Camera(), w, 0, body_wrist, np.array([800,500]))
F37:0051             expected = native_projection(points, 1000, [640,360])
F37:0052             expected += [800,500]-expected[0]
F37:0053             actual = native_projection(result['vertices'], 1000, [640,360])
F37:0054             np.testing.assert_allclose(actual, expected, atol=.0001)
F37:0055             self.assertEqual(float(np.linalg.norm(result['vertices'][0])), float(np.linalg.norm(b
F37:0055+ dy_wrist)), places=6)
F37:0058     class InputTests(unittest.TestCase):
F37:0059         def test_json_roundtrip_and_nonfinite_rejection(self):
F37:0060             with tempfile.TemporaryDirectory() as tmp:
F37:0061                 p=Path(tmp)/'x.json';write(p,{'中文':1})
F37:0062                 self.assertEqual(json.loads(p.read_text()), {'中文':1})
F37:0063                 q=Path(tmp)/'x.npz';np.savez(q,x=np.array([np.nan]))
F37:0064                 with self.assertRaises(ValueError):load(q)
F37:0066         def test_unresolved_variable_rejected(self):
F37:0067             with self.assertRaises(ValueError):expand('${EGOSMPLX_MISSING_TEST_VARIABLE}', Path('.'))
F37:0069         def test_path_traversal_frame_id_rejected(self):
F37:0070             with tempfile.TemporaryDirectory() as tmp:
F37:0071                 p=Path(tmp)
F37:0072                 write(p/'frames.json', {'frames':[{'id':'../outside','image':'x.jpg','calibration':'c.js
F37:0072+ on'}}])
F37:0073                 write(p/'config.json', {'paths':{},'manifest':'frames.json','output':'out'})
F37:0074                 with self.assertRaises(ValueError):load_config(p/'config.json')
F37:0077     if __name__ == '__main__':
F37:0078         unittest.main()
F38:0001     """Camera convention and differentiable projection regression tests.
F38:0003     Run in body_python. The controller-only environment may omit PyTorch.
F38:0004     """
F38:0005     import json
F38:0006     from pathlib import Path
F38:0007     import tempfile
F38:0008     import unittest
F38:0009     import numpy as np
F38:0010     try:
F38:0011         import torch
F38:0012     except ImportError:
F38:0013         torch = None
F38:0016     @unittest.skipIf(torch is None, 'PyTorch is installed in the configured body worker environment')

```

```

F38:0017 class CameraTests(unittest.TestCase):
F38:0018     def make_camera(self, **changes):
F38:0019         from egosmplx_pipeline.camera import FishEyeCameraCalibrated
F38:0020         calibration = dict(size=[1280,720], polynomial1C2W=[-100.], polynomial1W2C=[150.,80.,2.],
F38:0021             image_center=[630,350], affine=[1.02,.01,-.02],
F38:0022             camera_axis_transform=[[0,-1,0],[1,0,0],[0,0,1]])
F38:0023         calibration.update(changes)
F38:0024         with tempfile.TemporaryDirectory() as tmp:
F38:0025             path=Path(tmp)/'camera.json';path.write_text(json.dumps(calibration))
F38:0026             return FishEyeCameraCalibrated(path)
F38:0028     def test_numpy_and_horner_projection_match(self):
F38:0029         from egosmplx_pipeline.projection import stable_world2camera
F38:0030         camera=self.make_camera()
F38:0031         points=np.array([[.1,.2,.8],[-.2,.1,.4],[.1,-.2,.7]],np.float64)
F38:0032         tensor=torch.tensor(points,dtype=torch.float64,requires_grad=True)
F38:0033         pixels=stable_world2camera(camera,tensor)
F38:0034         np.testing.assert_allclose(pixels.detach().numpy(),camera.world2camera(points),atol=1e-10)
F38:0035         pixels.square().sum().backward()
F38:0036         self.assertTrue(torch.isfinite(tensor.grad).all())
F38:0038     def test_inverse_depth_is_radial_with_axis_transform(self):
F38:0039         camera=self.make_camera()
F38:0040         pixels=np.array([[600.,400.],[700.,300.]])
F38:0041         radii=np.array([.4,.8])
F38:0042         xyz=camera.camera2world(pixels,radii)
F38:0043         np.testing.assert_allclose(np.linalg.norm(xyz,axis=1),radii,rtol=1e-6)
F38:0045     def test_singular_affine_rejected(self):
F38:0046         with self.assertRaises(ValueError):self.make_camera(affine=[1,1,1])
F38:0049 if __name__ == '__main__':
F38:0050     unittest.main()
F39:0001 #!/usr/bin/env python3
F39:0002 """Export editable DOCX and paginated PDF from the actual repository sources.
F39:0004 The full source is always retained. The deposit copy uses the first and last
F39:0005 30 pages when the full listing exceeds 60 pages. No unrelated filler code is
F39:0006 generated. Applicant facts are not inferred from the GitHub account.
F39:0007 """
F39:0008 import argparse
F39:0009 import hashlib
F39:0010 import json
F39:0011 from pathlib import Path
F39:0012 import re
F39:0014 ROOT = Path(__file__).resolve().parents[1]
F39:0017 def digest(path):
F39:0018     return hashlib.sha256(path.read_bytes()).hexdigest()
F39:0021 def chunks(items, count):
F39:0022     return [items[i:i+count] for i in range(0, len(items), count)]
F39:0025 def font_setup(font_path):
F39:0026     from reportlab.pdfbase import pdfmetrics
F39:0027     from reportlab.pdfbase.ttfonts import TTFont
F39:0028     candidates = [font_path] if font_path else [
F39:0029         Path('/usr/share/fonts/truetype/arphic-gbsn001p/gbsn001p.ttf'),

```



```

F39:0030         Path('/usr/share/fonts/truetype/droid/DroidSansFallbackFull.ttf'),
F39:0031     ]
F39:0032     path = next((p for p in candidates if p and p.is_file()), None)
F39:0033     if path is None:
F39:0034         raise FileNotFoundError('Provide —font-path with a Chinese-capable TTF font')
F39:0035     pdfmetrics.registerFont(TTFont('EgoChinese', str(path)))
F39:0036     return path
F39:0039 def wrap(text, size, width):
F39:0040     """Wrap only the printed representation; source files remain unchanged."""
F39:0041     from reportlab.pdfbase.pdfmetrics import stringWidth
F39:0042     lines, line = [], ''
F39:0043     for char in text.expandtabs(4):
F39:0044         if line and stringWidth(line + char, 'EgoChinese', size) > width:
F39:0045             lines.append(line)
F39:0046             line = ''
F39:0047             line += char
F39:0048     if line:
F39:0049         lines.append(line)
F39:0050     return lines
F39:0053 def source_listing():
F39:0054     preferred = ['egosmplx_pipeline/__init__.py', 'egosmplx_pipeline/__main__.py',
F39:0055                 'egosmplx_pipeline/configuration.py', 'egosmplx_pipeline/pipeline.py',
F39:0056                 'egosmplx_pipeline/frame.py', 'egosmplx_pipeline/bootstrap.py',
F39:0057                 'egosmplx_pipeline/io.py', 'egosmplx_pipeline/sampling.py',
F39:0058                 'egosmplx_pipeline/sapiens_pose.py', 'egosmplx_pipeline/sapiens_seg.py',
F39:0059                 'egosmplx_pipeline/wilor_predict.py', 'egosmplx_pipeline/body_common.py',
F39:0060                 'egosmplx_pipeline/body_rigid.py', 'egosmplx_pipeline/body_pose.py',
F39:0061                 'egosmplx_pipeline/projection.py',
F39:0062                 'egosmplx_pipeline/matching.py', 'egosmplx_pipeline/lift.py',
F39:0063                 'egosmplx_pipeline/topology.py', 'egosmplx_pipeline/fusion.py',
F39:0064                 'egosmplx_pipeline/render.py', 'egosmplx_pipeline/report.py']
F39:0065     discovered = sorted(str(p.relative_to(ROOT)) for folder in ['egosmplx_pipeline', 'tools', 'tests
F39:0065+ ']
F39:0066                         for p in (ROOT / folder).rglob('*.py'))
F39:0067     order = preferred + [name for name in discovered if name not in preferred] + ['run_example.sh']
F39:0068     inventory, listing = [], []
F39:0069     for number, name in enumerate(order, 1):
F39:0070         path = ROOT / name
F39:0071         lines = path.read_text(encoding='utf-8').splitlines()
F39:0072         entry = dict(file_id='F%02d' % number, path=name, sha256=digest(path),
F39:0073                     physical_lines=len(lines), nonempty_lines=sum(bool(x.strip()) for x in lines))
F39:0074         inventory.append(entry)
F39:0075         for line_number, text in enumerate(lines, 1):
F39:0076             if not text.strip():
F39:0077                 continue
F39:0078             # Prefix allows exact correspondence after a long source line wraps.
F39:0079             prefix = '%s:%04d ' % (entry['file_id'], line_number)
F39:0080             parts = wrap(text, 9, 451)
F39:0081             for continuation, part in enumerate(parts):
F39:0082                 label = prefix if not continuation else prefix.rstrip() + '+ '

```

```

F39:0083             listing.append((label + part, name))
F39:0084     return inventory, chunks(listing, 50)
F39:0087 def clean_markdown(text):
F39:0088     text = re.sub(r'!\[([^\]]*)\]\([^\)]*\)', '', text)
F39:0089     text = re.sub(r'\[([^\]]+)\]\([^\)]*\)', r'\1 (\2) ', text)
F39:0090     text = text.replace('***', '').replace('`', '')
F39:0091     return text
F39:0094 def prose_pages(path):
F39:0095     rows = []
F39:0096     for line in clean_markdown(path.read_text(encoding='utf-8')).splitlines():
F39:0097         if not line.strip():
F39:0098             continue
F39:0099         line = re.sub(r'^#\s*', '', line)
F39:0100         rows.extend((part, str(path.relative_to(ROOT))) for part in wrap(line, 11, 505))
F39:0101     return chunks(rows, 40)
F39:0104 def export_docx(path, title, subtitle, pages, source=False, original_numbers=None, figures=()):
F39:0105     from docx import Document
F39:0106     from docx.shared import Inches, Pt
F39:0107     from docx.oxml import OxmlElement
F39:0108     from docx.oxml.ns import qn
F39:0109     document = Document()
F39:0110     section = document.sections[0]
F39:0111     section.page_width, section.page_height = Inches(8.2677), Inches(11.6929)
F39:0112     section.top_margin, section.bottom_margin = Pt(50), Pt(42)
F39:0113     section.left_margin, section.right_margin = Pt(42), Pt(42)
F39:0114     section.header_distance, section.footer_distance = Pt(18), Pt(18)
F39:0115     normal = document.styles['Normal']
F39:0116     normal.font.name = 'SimSun'
F39:0117     normal._element.rPr.rFonts.set(qn('w: eastAsia'), 'SimSun')
F39:0118     normal.font.size = Pt(9 if source else 11)
F39:0119     normal.paragraph_format.space_after = Pt(0)
F39:0120     normal.paragraph_format.space_before = Pt(0)
F39:0121     normal.paragraph_format.line_spacing = Pt(14.2 if source else 17.3)
F39:0122     header = section.header.paragraphs[0]
F39:0123     header.text = title + ' | ' + subtitle
F39:0124     for run in header.runs:
F39:0125         run.font.size = Pt(8)
F39:0126     footer = section.footer.paragraphs[0]
F39:0127     footer.alignment = 2
F39:0128     footer.add_run('第 ')
F39:0129     field = OxmlElement('w:fldSimple')
F39:0130     field.set(qn('w:instr'), 'PAGE')
F39:0131     footer._p.append(field)
F39:0132     footer.add_run(' 页')
F39:0133     for page_index, page in enumerate(pages):
F39:0134         if page_index:
F39:0135             document.add_page_break()
F39:0136         for line, _ in page:
F39:0137             paragraph = document.add_paragraph()
F39:0138             paragraph.add_run(line)

```

```

F39:0139     for caption, image in figures:
F39:0140         document.add_page_break()
F39:0141         document.add_paragraph(caption)
F39:0142         document.add_picture(str(image), width=Inches(7.0))
F39:0143     document.save(path)
F39:0146 def export_pdf(path, title, subtitle, pages, source=False, original_numbers=None, figures=()):
F39:0147     from reportlab.pdfgen.canvas import Canvas
F39:0148     from reportlab.lib.pagesizes import A4
F39:0149     from reportlab.lib.utils import ImageReader
F39:0150     canvas = Canvas(str(path), pagesize=A4)
F39:0151     canvas.setTitle(title + ' ' + subtitle)
F39:0152     canvas.setAuthor('EgoSmplx project: applicant facts pending verification')
F39:0153     width, height = A4
F39:0154     total = len(pages) + len(figures)
F39:0155     for index, page in enumerate(pages):
F39:0156         canvas.setFont('EgoChinese', 8)
F39:0157         canvas.drawString(42, height-28, title)
F39:0158         canvas.drawString(42, height-41, subtitle)
F39:0159         label = '第 %d / %d 页' % (index+1, total)
F39:0160         if original_numbers is not None:
F39:0161             label += '; 完整源码第 %d 页' % original_numbers[index]
F39:0162         canvas.drawRightString(width-42, 24, label)
F39:0163         canvas.setFont('EgoChinese', 9 if source else 11)
F39:0164         spacing = 14.2 if source else 17.3
F39:0165         for line_index, (line, _) in enumerate(page):
F39:0166             canvas.drawString(42, height-64-line_index*spacing, line)
F39:0167         canvas.showPage()
F39:0168     for number, (caption, image) in enumerate(figures, len(pages)+1):
F39:0169         canvas.setFont('EgoChinese', 8)
F39:0170         canvas.drawString(42, height-28, title)
F39:0171         canvas.setFont('EgoChinese', 11)
F39:0172         canvas.drawString(42, height-62, caption)
F39:0173         reader = ImageReader(str(image))
F39:0174         iw, ih = reader.getSize()
F39:0175         scale = min((width-84)/iw, (height-130)/ih)
F39:0176         canvas.drawImage(reader, 42, height-84-ih*scale, iw*scale, ih*scale)
F39:0177         canvas.setFont('EgoChinese', 8)
F39:0178         canvas.drawRightString(width-42, 24, '第 %d / %d 页' % (number, total))
F39:0179         canvas.showPage()
F39:0180     canvas.save()
F39:0183 def main():
F39:0184     parser = argparse.ArgumentParser(description=__doc__)
F39:0185     parser.add_argument('--output', type=Path, default=ROOT/'copyright/generated')
F39:0186     parser.add_argument('--font-path', type=Path)
F39:0187     args = parser.parse_args()
F39:0188     font_setup(args.font_path)
F39:0189     args.output.mkdir(parents=True, exist_ok=True)
F39:0190     metadata = json.loads((ROOT/'software.json').read_text())
F39:0191     title = metadata['name_zh'] + ' ' + metadata['registration_version']
F39:0192     inventory, full_pages = source_listing()

```

```

F39:0193     full_numbers = list(range(1, len(full_pages)+1))
F39:0194     selected = list(range(len(full_pages))) if len(full_pages) <= 60 else list(range(30)) + list(ran
F39:0194+ ge(len(full_pages)-30, len(full_pages)))
F39:0195     deposit = [full_pages[i] for i in selected]
F39:0196     for suffix, pages, numbers in [('完整', full_pages, full_numbers), ('交存', deposit, [i+1 for i
F39:0196+ in selected])]:
F39:0197         stem = args.output / ('03_源程序文档_' + suffix)
F39:0198         export_pdf(stem.with_suffix('.pdf'), title, '源程序文档 (' + suffix + ') ', pages, True, num
F39:0198+ bers)
F39:0199         export_docx(stem.with_suffix('.docx'), title, '源程序文档 (' + suffix + ') ', pages, True, n
F39:0199+ umbers)
F39:0200     figures = [('图1 八帧三阶段输出（左：原始；中：身体校正；右：MANO 融合与局部表面调整）', ROOT/'d
F39:0200+ ocs/assets/contact_sheet.jpg')]
F39:0201     documents = [('01_登记信息填表稿', ROOT/'copyright/01_登记信息填表稿.md', []),
F39:0202                     ('02_软件设计说明书', ROOT/'docs/DESIGN_zh.md', figures),
F39:0203                     ('04_权属核对与提交清单', ROOT/'copyright/04_权属核对与提交清单.md', [])]
F39:0204     for name, path, illustrations in documents:
F39:0205         pages = prose_pages(path)
F39:0206         export_pdf(args.output/(name+'.pdf'), title, name[3:], pages, figures=illustrations)
F39:0207         export_docx(args.output/(name+'.docx'), title, name[3:], pages, figures=illustrations)
F39:0208     index = dict(software=title, files=inventory,
F39:0209                 physical_source_lines=sum(x['physical_lines'] for x in inventory),
F39:0210                 nonempty_source_lines=sum(x['nonempty_lines'] for x in inventory),
F39:0211                 printed_source_lines=sum(len(p) for p in full_pages), full_source_pages=len(full_pa
F39:0211+ ges),
F39:0212                 deposited_original_pages=[i+1 for i in selected], source_lines_per_page=50,
F39:0213                 listing_rule='Nonempty actual source lines only: long lines wrap for print with fil
F39:0213+ e/line references: no filler',
F39:0214                 ownership_status='Applicant must verify rights and third-party boundaries')
F39:0215     (args.output/'source_inventory.json').write_text(json.dumps(index,indent=2,ensure_ascii=False)+'
F39:0215+ \n')
F39:0216     (ROOT/'docs/SOURCE_INDEX.md').write_text('# 源码索引\n\n' + '\n'.join(
F39:0217         '- %s: `%s` (%d 行, 非空 %d 行)' % (x['file_id'],x['path'],x['physical_lines'],x['nonempty_
F39:0217+ lines']) for x in inventory) + '\n')
F39:0218     print(json.dumps({k:index[k] for k in ['physical_source_lines','nonempty_source_lines','printed_
F39:0218+ source_lines','full_source_pages']},ensure_ascii=False))
F39:0221 if __name__ == '__main__':
F39:0222     main()
F40:0001 #!/usr/bin/env bash
F40:0002 set -euo pipefail
F40:0003 cd — "$(dirname — "${BASH_SOURCE[0]}")"
F40:0004 exec "${EGOSMPLX_CONTROLLER_PYTHON:-python3}" -m egosmplx_pipeline "${1:-check}" —config "${EGOSMPL
F40:0004+ X_CONFIG_FILE:-configs/pipeline.example.json}" "${@:2}"

```